

Statistisches Lernen

T. Werner/P. Ruckdeschel

Oldenburg, SoSe 2019

T. Werner/P. Ruckdeschel

Oldenburg, SoSe 2019

0. Organisation

0.1 Termine

0.2 Prüfungsleistung/Übungen

■ Zeiten (Vorlesung)

Mo 10:15-11:45 Raum: W01 0-012

Mi 08:30-10:00 Raum: W01 0-012 (im Wechsel mit Übung)

■ Zeiten (Übung: 14-tägig)

Mi 08:30-10:00 Raum: W01 0-012

zu verschiebende Termine / Sonderräume

- zu verschiebende Termine
- noch nichts

■ Übungen:

- ▶ bei K. Rohmeyer
- ▶ 14-tägig
- ▶ 16 Punkte je Blatt
- ▶ zu zweit abzugeben; werden korrigiert; vorzurechnen

■ Prüfungsleistung:

- ▶ mündliche Prüfung nach Vereinbarung
- ▶ Katalog möglicher Fragen zur Orientierung
- ▶ Einstiegsfrage

1. Motivation

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 Lineare Regressionsmodelle
- 4 GLM
- 5 Klassische Klassifikationsverfahren
- 6 Gütemaße für Klassifikations- und Rankingverfahren
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

■ Klassische Verfahren:

- ▶ Regression: Lineare Modelle, generalisierte lineare Modelle
- ▶ Klassifikation: Diskriminanzanalyse, kNN, CART
- ▶ Clustering

■ Moderne Verfahren:

- ▶ Regularisierungsverfahren
- ▶ Support Vector Machines
- ▶ Ensemble-Methoden, Boosting

■ Ausblick

- ▶ Ranking
- ▶ Online-Learning

Grobe Unterteilung der Probleme des statistischen Lernens:

- Supervised Learning: Gegeben sind m Beobachtungen, die jeweils aus einer abhängigen Variable Y_i und $p \geq 1$ erklärenden Variablen $X_i = (X_{i1}, \dots, X_{ip})$ bestehen.

Daten liegen als Datenmatrix $D = (X, Y) \in \mathcal{X} \times \mathcal{Y} \subset \mathbb{R}^{m \times (p+1)}$ vor.

Drei Szenarien des überwachten Lernens (supervised learning):

- ▶ Y ist metrisch und soll erklärt werden $\rightsquigarrow$ Regression
- ▶ Y ist kategoriell und soll erklärt werden $\rightsquigarrow$ Klassifikation
- ▶ Y ist metrisch oder diskret, aber nur die Anordnung soll erklärt werden $\rightsquigarrow$ Ranking

- Unüberwachtes Lernen (unsupervised learning): Es ist $D = X$, d.h., es gibt keine zu erklärenden Variablen. Hier sollen Strukturen in den Daten erkannt werden $\rightsquigarrow$ Clustering

Hier nur ein paar Beispiele:

■ Regression: Modellierung von

- ▶ Bindungsintensitäten eines Transkriptionsfaktors an DNA-Stränge
- ▶ Produktionsraten von Riboflavin einer Bakterienkultur
- ▶ Eigenkapitalrenditen auf einem Aktienmarkt
- ▶ Immobilienwerten
- ▶ Ausfallwahrscheinlichkeit bei Krediten
- ▶ fMRT: Kartierung von Gehirnarealen

■ Ranking

- ▶ Rating
- ▶ Risk Based Auditing
- ▶ Radarechos in der Ionosphäre

■ Klassifikation:

- ▶ Automatisches Einlesen von handschriftlichen Zahlen oder Wörtern
- ▶ Spam-Filter für E-Mails
- ▶ Risikofaktoren für Krebs oder Herzerkrankungen
- ▶ Kreditwürdigkeit

■ Clustering

- ▶ Ähnlichkeiten zwischen DNA-Strängen
- ▶ Systematisierung von Pflanzen oder Tieren
- ▶ Auffinden von Krebs-Unterarten
- ▶ Identifizierung von Sternhaufen
- ▶ Ähnlichkeiten von Molekülen

- Liegen Daten der Form $\mathcal{D} = (\mathcal{X}, \mathcal{Y})$ vor und ist $\mathcal{Y}$ metrisch, wie können wir dann unterscheiden, ob wir ein Regressions- oder ein Rankingproblem lösen?
- Definiere problemadaptiert eine Funktion, die die Güte eines Modells bemessen kann
- Eine solche Funktion heißt **Verlustfunktion**. O.E. sei für Probleme des Supervised Learning hier immer die Verlustfunktion gegeben durch $L : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$.

Statistisches Lernen vs. maschinelles Lernen

- Theoretisch fundiert: Nehme unabhängig identisch verteilte Beobachtungen $(X_i, Y_i) \sim F$ an, $i = 1, \dots, m$, mit gemeinsamer Verteilung F , die Werte in $\mathcal{X} \times \mathcal{Y}$ annimmt
- Ziel: Der erwartete Verlust bzgl. F soll minimiert werden
- führt zu theoretischen Resultaten für allgemeine Datenmatrizen, deren Zeilen der Verteilung F folgen („random design“)

Statistisches Lernen vs. maschinelles Lernen

- Je nachdem, welcher Natur die Annahmen und die gewünschten theoretischen Resultate sind, unterscheidet man:
- **Statistisches Lernen:** Verteilungsannahmen an die Beobachtungen
- In der Regel Resultate vom ZGWS-Typ, sodass asymptotisch auf eine Normalverteilung geschlossen werden kann, Angabe einer Rate, mit der (in Abhängigkeit von der Zahl der Beobachtungen) das richtige Modell gefunden wird
- **Maschinelles Lernen:** Keine Verteilungsannahmen, sondern direkte Verwendung der vorliegenden Daten
- Aussage, dass das korrekte Modell gefunden werden kann, aber ohne Rate

Praktisches Vorgehen: Empirische Risikominimierung

- Seien nun Daten D und die Verlustfunktion L gegeben
- In der Praxis wird nun die empirische Verteilung $\hat{F}_m$ zugrunde gelegt
- Nähere den Erwartungswert durch ein arithmetisches Mittel, d.h., minimiere

$$L_m(Y, \hat{Y}) := \frac{1}{m} \sum_i L(Y_i, \hat{Y}_i)$$

für gefittete Werte $\hat{Y}_i$.

- Für $\hat{Y}_i = f(X_i)$ für alle i ist das gesuchte Modell also

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} \left(\frac{1}{m} \sum_i L(Y_i, f(X_i)) \right).$$

- Hier wird noch jede Funktion $f : \mathcal{X} \rightarrow \mathcal{Y}$ zugelassen, repräsentiert durch die Funktionenklasse $\mathcal{F}$. Das ist noch nicht optimal (s. Kapitel Regularisierung)

M-Schätzer und M-Funktionale

- Parametrische Modelle: Es ist $f = f_\theta$ für Parameter $\theta \in \Theta$, sodass die Optimierung nicht über eine Funktionenklasse $\mathcal{F}$, sondern über den Parameterraum Θ läuft
- Schreibe in diesem Fall $L(y, x, \theta) := L(y, f_\theta(x))$
- Ein Schätzer, der

$$\max_{\theta \in \Theta} \left(\theta \mapsto \int_{\mathcal{X} \times \mathcal{Y}} L(y, x, \theta) dF(x, y) \right) \quad \text{oder} \quad \min_{\theta \in \Theta} (...)$$

löst, heißt M-Schätzer von θ

- Sofern $\varphi = \partial_\theta L$ existiert und unter dem Integralzeichen differenziert werden darf, kann man äquivalent den Z-Schätzer

$$\text{zero}_{\theta \in \Theta} \left(\theta \mapsto \int_{\mathcal{X} \times \mathcal{Y}} \varphi(y, x, \theta) dF(x, y) \right)$$

betrachten

- $T : \mathcal{C}(\Theta) \rightarrow \Theta$ heißt **M-Funktional**, wenn T eine Nullstelle von $f \in \mathcal{C}(\Theta)$ findet, d.h., $f(T(f)) = 0$

Definition 0.1

Sei $\mathcal{F}$ eine Familie von Funktionen und sei L eine Verlustfunktion. Dann ist das **Risikofunktional** definiert durch

$$L(f) := \int L(y, f(x)) dF(x, y) = \mathbb{E}[L(X_1)] \quad (1.1)$$

und falls Daten $(x_1, y_1), \dots, (x_m, y_m)$ vorliegen, kann das **empirische Risikofunktional** geschrieben werden als

$$L_m(f) := \frac{1}{m} \sum_i L(y_i, f(x_i)). \quad (1.2)$$

Beim ERM-Prinzip wird also (1.2) statt (1.1) minimiert.

Beispiel 0.2

Bei der Maximum-Likelihood-Schätzung löst man ein empirisches Maximierungsproblem. Sei p_θ eine parametrische Dichte. Konkret berechnet man also einen empirischen M-Schätzer mit $L_\theta(\mathbf{x}) := \ln(p_\theta(\mathbf{x}))$ oder (bei ausreichender Glattheit des Modells) einen empirischen Z-Schätzer mit $\varphi_\theta(\mathbf{x}) = \partial_\theta \ln(p_\theta(\mathbf{x}))$.

Sei

$$PL := \mathbb{E}[L(Y, f(X))], \quad P_m L := \frac{1}{m} \sum_{i=1}^m L(Y_i, f(X_i)).$$

Dann ist das empirische Risiko gerade

$$L_m = P_m L.$$

Definition 0.3

Definiere die **Targetfunktion** über

$$f^* := \operatorname{argmin}_f (PL(f)) = \operatorname{argmin}_f (\mathbb{E}[L(f)]),$$

also den absoluten Minimierer des wahren Risikos. Dann ist das absolute Minimum des Risikos gerade

$$L^* := \min_f (L(f)) = L(f^*).$$

Definition 0.4

Der **Exzess (oder das Exzessrisiko)** für eine Funktion f ist gegeben durch

$$\Lambda(f) := P(L(f) - L(f^*)) = E[L(f(X_1))] - E[L(f^*(X_1))] .$$

- Ferner definiere den **empirischen Prozess** durch

$$\sqrt{m}(P_m - P)$$

- da im Allgemeinen $f \in \mathcal{F}$ gilt, wird dieser empirische Prozess zum **stochastischen Prozess**

$$\sqrt{m}(P_m - P)f = \frac{1}{\sqrt{m}} \sum_i (f(X_i) - E[f]), \quad f \in \mathcal{F} .$$

- Das **Ziel des ERM-Prinzips** ist es also, für eine gegebene Klasse $\mathcal{F}$ von Funktionen das empirische Minimum

$$f_m^* := \operatorname{argmin}_{f \in \mathcal{F}} (L_m(f))$$

zu finden.

Definition 0.5

In der Zerlegung

$$L(\mathbf{f}_m^*) - L^* = (L(\mathbf{f}_m^*) - \inf_{\mathcal{F}} L(\mathbf{f})) + (\inf_{\mathcal{F}} L(\mathbf{f}) - L^*)$$

ist der erste Summand der Schätzfehler, der zweite der Approximationsfehler (s. Kapitel Regularisierung).

Lemma 0.6

Sei $\mathbf{f}_m^$ der Minimierer des empirischen Risikofunktional L_m . Dann gilt*

$$L(\mathbf{f}_m^*) - \inf_{\mathbf{f} \in \mathcal{F}} L(\mathbf{f}) \leq 2 \sup_{\mathbf{f} \in \mathcal{F}} |L_m(\mathbf{f}) - L(\mathbf{f})|.$$

Definition 0.7

- Das ERM-Prinzip ist **konsistent**, falls gilt, dass

$$\begin{aligned} L(f_m) &\xrightarrow{P} \inf_f(L(f)) \\ L_m(f_m) &\xrightarrow{P} \inf_f(L(f)) \end{aligned}, \quad m \rightarrow \infty. \quad (1.3)$$

- Das empirische Risikofunktional **konvergiert gleichmäßig** gegen das wahre Risikofunktional, falls

$$P(\sup_f (|L(f) - L_m(f)|) > \varepsilon) \rightarrow 0. \quad (1.4)$$

Einseitige Konvergenz ist definiert durch

$$P(\sup_f (L(f) - L_m(f)) > \varepsilon) \rightarrow 0. \quad (1.5)$$

Definition 0.8

- Eine Menge $\mathcal{D}$ ist eine P -**Glivenko-Cantelli-Klasse von Mengen** für P , falls

$$\sup_{D \in \mathcal{D}} (|P_n D - PD|) = \|P_n - P\|_{\mathcal{D}} \xrightarrow{P} 0.$$

- Eine Klasse $\mathcal{F}$ von Funktionen ist eine P -**Glivenko-Cantelli-Klasse von Funktionen** für P , falls

$$\sup_{f \in \mathcal{F}} (|P_n f - Pf|) = \|P_n - P\|_{\mathcal{F}} \xrightarrow{f.s.} 0.$$

- Eine Klasse $\mathcal{F}$ von (integrierbaren) Funktionen mit $\sup_{\mathcal{F}} (|f(x) - Pf|) < \infty$ für alle x heißt P -**Donsker-Klasse**, falls

$$\sqrt{n}(P_n - P)f \rightarrow B$$

mit einer Brownschen Brücke B .

Bemerkung 0.9

Anschaulich sind die Glivenko-Cantelli-Klassen von Funktionen gerade die, bei denen das starke GGZ gleichmäßig über eine Klasse von Funktionen gilt. Analog sind die Donsker-Klassen die Funktionen, bei denen der ZGWS gleichmäßig über eine Klasse von Funktionen gilt.

Beispiel 0.10

- Die Klasse $\{I(\in [-\infty, t]) \mid t \in \mathbb{R}\}$ ist eine GC-Klasse für Mengen.
- Sei P die Gleichverteilung auf $[0, 1]$. Die Klasse aller Borel-Mengen auf $[0, 1]$ ist keine GC-Klasse für Mengen.
- Die Menge aller konvexen Teilmengen von $[0, 1]^2$ bilden eine GC-Klasse für Mengen, wenn P eine Gleichverteilung auf $[0, 1]^2$ ist.

Definition 0.11

Eine Menge $B \subset M$ in einem metrischen Raum heißt ε -**Netz in M** , falls für alle $x \in M$ ein $b \in B$ existiert, sodass $d(b, x) < \varepsilon$, d.h., es ist

$$M \subset \bigcup_{b \in B} B_\varepsilon(b).$$

B heißt **finites ε -Netz in M** , wenn B endlich viele Elemente enthält. Es heißt **minimal**, wenn B die kleinste Anzahl an Elementen enthält, sodass obige Inklusion gilt.

Definition 0.12

Die ε -**Packzahl** einer Teilmenge M eines metrischen Raumes ist die größte Zahl $D(M, \|\cdot\|, \varepsilon) =: D$, sodass D paarweise verschiedene Punkte $t_1, \dots, t_D \in M$ existieren, sodass $\|t_i - t_j\| > \varepsilon$ für alle $i \neq j$. Die Menge $T := \{t_1, \dots, t_n\}$ heißt dann **Packing** von M .

Definition 0.13

Die Zahl

$$N(M, \|\cdot\|, \varepsilon) := \min\{n \in \mathbb{N} \mid \text{Es ex. } b_1, \dots, b_n \in M : M \subset \bigcup_i B_\varepsilon(b_i)\},$$

also die Anzahl der Elemente eines minimalen ε -Netzes, heißt ε -**Überdeckungszahl** von M . Der Logarithmus der Überdeckungszahl, also $\ln(N(M, \|\cdot\|, \varepsilon))$ heißt **metrische Entropie** von M . Ferner ist

$$H(M, \|\cdot\|) := \int_0^\varepsilon \sqrt{\ln(N(M, \|\cdot\|, s))} ds$$

das **Entropie-Integral** von M .

Lemma 0.14

Es ist

$$D(M, \|\cdot\|, 2\varepsilon) \leq N(M, \|\cdot\|, \varepsilon) \leq D(M, \|\cdot\|, \varepsilon)$$

für alle $\varepsilon > 0$.

Definition 0.15

Ein **Bracket** $[l, u]$ ist definiert als die Menge aller Funktionen f , sodass $l \leq f \leq u$. Ein Bracket heißt ε -Bracket, wenn $|u - l| < \varepsilon$ ist. Die minimale Zahl an solchen ε -Brackets, die nötig sind, um $\mathcal{F}$ zu überdecken, ist die **Bracketingzahl** $N_{[]}(\mathcal{F}, \|\cdot\|, \varepsilon)$. Das **Bracketing-Entropie-Integral** ist definiert durch

$$J_{[]}(\delta, \mathcal{F}, \|\cdot\|) := \int_0^\delta \sqrt{\ln(N_{[]}(\mathcal{F}, \|\cdot\|, \varepsilon))} d\varepsilon.$$

Satz 0.16

- Jede Klasse $\mathcal{F}$ messbarer Funktionen mit **endlicher Bracketing-Zahl** $\mathcal{N}_{[]}(\varepsilon, \mathcal{F}, L_1(P))$ ist eine P -Glivenko-Cantelli-Klasse.
- Jede Klasse $\mathcal{F}$ messbarer Funktionen, sodass das Bracketing-Integral endlich ist, ist eine P -Donsker-Klasse.
- Die Klasse

$$\mathcal{F} := \{f_t := I_{]-\infty, t]} \mid t \in \mathbb{R}\}$$

ist eine GC-Klasse und die korrespondierende Klasse bestehend aus den empirischen Prozessen

$$\mathbb{G}_n f_t = \sqrt{n}(F_n(t) - F(t))$$

ist eine Donsker-Klasse.

Wie kann man die Qualität eines berechneten Modells $\hat{f}$ bemessen?

- Berechnung des sich ergebenden Verlustes auf D wäre unfair, da die Y -Werte bereits beim Fitten verwendet worden sind
- Man benötigt also Daten, die aus der gleichen Verteilung stammen und an denen das Modell getestet werden kann
- Trenne daher die Daten in Trainingsdaten D^{train} und Testdaten D^{test} auf
- Sei o.E. im Folgenden immer n die Anzahl der Beobachtungen, die zum Trainieren verwendet werden und $n_{test} := m - n$

- Konkret ist für ein Modell $\hat{f}$

$$err^{train}(\hat{f}) = \frac{1}{n} \sum_i L(Y_i^{train}, \hat{f}(X_i^{train}))$$

der **Trainingsfehler**

- Ein geringer Trainingsfehler ist i.A. kein ausreichendes Kriterium für ein gutes Modell!
- Analog ist

$$err^{test}(\hat{f}) = \frac{1}{n_{test}} \sum_i L(Y_i^{test}, \hat{f}(X_i^{test}))$$

der **Testfehler (Generalisierungsfehler)**

- Ein geringer Testfehler sagt aus, dass das Modell gut auf allen anderen Daten, die aus der gleichen Verteilung wie die Trainingsdaten stammen, funktioniert
- Konvention: $L(u, u) = 0$ für jede Verlustfunktion

- Wie können wir den Testfehler eines Modells schätzen?
- Woher bekommen wir Testdaten?
- Insbesondere: Wie können wir sicherstellen, dass die Testdaten aus der gleichen Verteilung stammen wie die Trainingsdaten?

Bekannte Methode zur Generierung neuer Stichproben: Statistisches Bootstrap

- Erzeuge B neue Stichproben mit gleicher Länge wie die Originalstichprobe durch Ziehen mit Zurücklegen
- Berechne auf jeder generierten Stichprobe den Schätzer
- Varianz des Schätzers kann so genähert werden

Alternative: Unterstichproben durch Ziehen ohne Zurücklegen (Subsampling)

Frage: Kann man diese Methode auf Probleme des statistischen Lernens übertragen?

Analoge Idee:

- Partitioniere D mehrfach in Trainingsdaten $D^{(b,train)}$ und Testdaten $D^{(b,test)}$
- Dadurch folgen Trainings- und Testdaten jeweils der gleichen Verteilung
- Berechne jeweils auf $D^{(b,train)}$ das Modell $\hat{f}^{(b)}$ und auf $D^{(b,test)}$ den Testfehler $err^{(b,test)}(\hat{f}^{(b)})$
- Dadurch kann der erwartete Testfehler durch den **kreuzvalidierten Fehler**

$$err^{CV}(\hat{f}) := \frac{1}{B} \sum_b err^{(b,test)}$$

approximiert werden

Das Vorgehen nennt sich Kreuzvalidierung (CV)

Konkrete mögliche Vorgehensweisen bei der Kreuzvalidierung:

- Randomisierte Kreuzvalidierung: Ziehe zufällig jeweils Trainingsdaten der Länge n . Die übrigen $m - n$ Beobachtungen sind die Testdaten
- K -fache Kreuzvalidierung: Zerteile D in K Unterdatensätze D_k und wähle im k -ten Szenario D_k als Testdaten und den Rest als Trainingsdaten
- Leave-one-out-CV (LOOCV): Im k -ten Szenario nur die k -te Beobachtung von D als Testdatensatz, alle anderen $m - 1$ Beobachtungen als Trainingsdaten. LOOCV ist also m -fache Kreuzvalidierung

2. Lineare Regressionsmodelle

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 **Lineare Regressionsmodelle**
- 4 GLM
- 5 Klassische Klassifikationsverfahren
- 6 Gütemaße für Klassifikations- und Rankingverfahren
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

Definition 1.1

Sei $p < n \in \mathbb{N}$. Ein lineares Modell für n reellwertige Beobachtungen Y_i und einen unbekannten Regressionsparameter $\beta \in \mathbb{R}^p$ und unbekanntem Skalenparameter $\sigma > 0$ besteht aus

- Beobachtungen $(Y, X)_i$, $i = 1, \dots, n$; dabei heißt
 - ▶ Y_i (mit Werten in $\mathbb{R}$) abhängige Variable/ zu erklärende Variable
 - ▶ X_i (mit Werten in $\mathbb{R}^p$) Regressor/unabhängige Variable
- Fehler-/Störgrößen ε_i , wobei $E[\varepsilon_i] = 0$, $\text{Var}(\varepsilon_i) = 1$ und ε_i unkorreliert

Die ε_i und die Y_i fassen wir zu Vektoren $\varepsilon, Y \in \mathbb{R}^n$ zusammen, die X_i zu einer Designmatrix $X \in \mathbb{R}^{n \times p}$ zusammen. Dann lautet die Modellgleichung

$$Y = X\beta + \sigma\varepsilon \quad (1)$$

Das zugehörige statistische Modell lautet mit $\vartheta^\tau = (\beta^\tau, \sigma)$

$$\mathcal{P} = \left\{ P_\vartheta \mid P_\vartheta = \mathcal{L}_\vartheta(Y) \right\} \quad (2)$$

auf dem Stichprobenraum $(\Xi, \mathcal{B}) = (\mathbb{R}^n, \mathbb{B}^n)$.

Proposition 1.2

Seien $p, q \in \mathbb{N}$ und $A \in \mathbb{R}^{p \times q}$. Dann existiert eine eindeutige Matrix $A^- \in \mathbb{R}^{q \times p}$ so dass

$$AA^- = (AA^-)^T \quad (3)$$

$$A^-A = (A^-A)^T \quad (4)$$

$$AA^-A = A, \quad A^-AA^- = A^- \quad (5)$$

A^- heißt generalisierte Inverse.

Lemma 1.3 (Projektionslemma)

Seien $p, q \in \mathbb{N}$, $y \in \mathbb{R}^p$ und $A \in \mathbb{R}^{p \times q}$. Dann ist jede Lösung zum Problem

$$|y - Ax| = \min! \quad x \in \mathbb{R}^q \quad (6)$$

zugleich auch Lösung der Normalengleichung

$$A^T Ax = A^T y \quad (7)$$

und umgekehrt. Die kürzeste aller Lösungen ist gerade $\hat{x} = A^-y$.

Bemerkung 1.4

Sei $\mathbf{x}^\natural$ eine Lösung zu Problem (6) im Projektionslemma 1.3. Dann ist offenbar

$$\hat{\mathbf{y}} = \mathbf{A}\mathbf{x}^\natural \quad (8)$$

die (eindeutige) orthogonale Projektion von $\mathbf{y}$ in den Spaltenraum $\mathbf{L}$ von $\mathbf{A}$. Im Beweis zu Proposition 1.2 haben wir aber gezeigt, dass diese Projektion durch die Matrix $\mathbf{A}\mathbf{A}^-$ dargestellt wird, so dass

$$\hat{\mathbf{y}} = \mathbf{A}\mathbf{A}^- \mathbf{y} \quad (9)$$

Im Folgenden schreiben wir für den Projektor $\mathbf{A}\mathbf{A}^-$ auch $\Pi_{\mathbf{A}}$.

Im linearen Modell können wir nun diese Projektionstechniken verwenden, um einen Schätzer für ϑ zu konstruieren.

Definition 1.5

Wir betrachten das lineare Modell.

- Der Schätzer β^{LS} definiert als

$$\beta^{LS} = X^{-}Y \quad (10)$$

heißt Kleinste-Quadrate-Schätzer für β .

- die Zufallsvariablen $r_i := Y_i - (X\beta^{LS})_i$ heißen (Kleinste-Quadrate-)Residuen.

Bemerkung 1.6

Nach (10) gilt

$$X\beta^{LS} = XX^{-}Y = \Pi_X Y \quad (11)$$

Definition 1.7

Sei $\mathcal{P} = \{\mathbf{P}_\beta \mid \beta \in \beta\}$ ein parametrisches Modell und $\mathbf{S}$ ein Schätzer für $\tau(\beta)$. Dann heißt $\mathbf{S}$ BLUE, falls $\mathbf{S}$ linear in den Beobachtungen und erwartungstreu ist, und für jeden weiteren linearen und erwartungstreuen Schätzer $\mathbf{S}'$ gilt: $\text{Cov}_\beta \mathbf{S} \preceq \text{Cov}_\beta \mathbf{S}'$.

Satz 1.8 (Gauß-Markov)

Wir betrachten das lineare Modell. Sei $\mathbb{N} \ni \mathbf{s} \leq \mathbf{p}$, $\mathbf{B} \in \mathbb{R}^{\mathbf{p} \times \mathbf{s}}$ und $\beta_B := \mathbf{B}^\tau \beta$.

- [Existenz] Es gibt einen linearen, erwartungstreuen Schätzer für β_B genau dann, wenn es eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times \mathbf{s}}$ gibt so dass

$$\mathbf{A}^\tau \mathbf{X} = \mathbf{B}^\tau \quad (12)$$

- [Verbesserung durch Projektion] Sei $\mathbf{S}(\mathbf{Y}) = \mathbf{A}^\tau \mathbf{Y}$ ein linearer und erwartungstreuer Schätzer für β_B . Dann ist auch $\hat{\mathbf{S}}(\mathbf{Y}) = \mathbf{A}^\tau \Pi_{\mathbf{X}} \mathbf{Y}$ erwartungstreu, und es gilt

$$\text{Cov}_\beta \hat{\mathbf{S}} \preceq \text{Cov}_\beta \mathbf{S} \quad \forall \beta \quad (13)$$

mit “=” genau dann wenn $\mathbf{A}^\tau = \mathbf{A}^\tau \Pi_{\mathbf{X}}$.

- [Eindeutigkeit der Verbesserung] Seien $\mathbf{S}_i(\mathbf{Y}) = \mathbf{A}_i^\tau \mathbf{Y}$, $i = 1, 2$ lineare und erwartungstreue Schätzer für β_B , sowie entsprechend $\hat{\mathbf{S}}_i(\mathbf{Y}) = \mathbf{A}_i^\tau \Pi_{\mathbf{X}} \mathbf{Y}$. Dann gilt

$$\mathbf{A}_1^\tau \Pi_{\mathbf{X}} = \mathbf{A}_2^\tau \Pi_{\mathbf{X}} \quad (14)$$

Mit anderen Worten $\hat{\mathbf{S}}_1 = \hat{\mathbf{S}}_2$ ist BLUE.

Bemerkung 1.9

Nach Konstruktion gilt bei Lösbarkeit von (12)

$$\hat{S}(Y) = A^T \Pi_X Y \stackrel{(11)}{=} A^T X \beta^{LS} = B \beta^{LS} \quad (15)$$

Sei nun im linearen Modell (2) $\varepsilon_i \sim \mathcal{N}(\mathbf{0}, \sigma^2)$ i.i.d. und sei $rkX = p$. Dann gilt folgende Aussage:

Sei nun im linearen Modell (2) $u_i \sim \mathcal{N}(\mathbf{0}, \sigma^2)$ i.i.d. und sei $\text{rk}X = p$. Dann gilt

Proposition 1.10

- $(\vartheta^{ML})^\tau := (\beta^{LS}, \frac{n-p}{n} \sigma_{LS}^2)$ ist ML-Schätzer.
- $\beta^{LS} \sim \mathcal{N}_p(\beta, \sigma^2 (X^\tau X)^{-1})$
- $\hat{Y} := X\beta^{LS} \sim \mathcal{N}_n(X\beta, \sigma^2 \Pi_X)$
- $r = Y - \hat{Y} \sim \mathcal{N}_n(\mathbf{0}, \sigma^2 \bar{\Pi}_X)$
- $\frac{n-p}{\sigma^2} \sigma_{LS}^2 \sim \chi_{n-p}^2$
- $|\hat{Y} - X\beta|^2 / \sigma^2 \sim \chi_p^2$ stochastisch unabhängig von σ_{LS}^2 und

$$\frac{|\hat{Y} - X\beta|^2}{p\sigma_{LS}^2} \sim F_{p, n-p} \quad (16)$$

- Sei L der Spaltenraum von X und $H \subset L$ ein linearer Teilraum mit $\dim H = q < p$ und Π_H die orthogonale Projektion auf H . Gilt dann $X\beta \in H$, so folgt $|(\Pi_X - \Pi_H)(Y)|^2 / \sigma^2 \sim \chi_{p-q}^2$, stochastisch unabhängig von σ_{LS}^2 . Für die Fisher-Statistik $F_{H,L}$ definiert als

$$F_{H,L} := \frac{n-p}{p-q} \frac{|(\Pi_X - \Pi_H)(Y)|^2}{|\bar{\Pi}_X(Y)|^2} = \frac{|X\beta^{LS} - \Pi_H(Y)|^2}{(p-q)\sigma_{LS}^2} \quad (17)$$

gilt $F_{H,L} \sim F_{p-q, n-p}$.

In der Regel befinden sich in Daten immer kategorielle Prädiktoren, d.h., Prädiktoren mit diskreten Ausprägungen (Beispiel: `Species` im `iris`-Datensatz)

- Idee: Schätze für jede Kategorie einen eigenen Achsenabschnitt
- Definiere zufällig eine Kategorie als Basis-Kategorie
- Der Achsenabschnitt β_0 ist derjenige für die Basis-Kategorie, alle anderen spezifischen Achsenabschnitte sind relativ zu β_0 zu verstehen
- Konkret: Ist X_p kategoriell mit 3 Ausprägungen **A**, **B**, **C**, so ist das Modell

$$Y = \beta_0 + X_{\cdot,1:(p-1)}\beta + \beta_B I(X_p = B) + \beta_C I(X_p = C).$$

Kategorielle Variablen

Dies lässt sich so nur sehr umständlich formulieren, wenn viele Variablen kategoriell sind

- Lösung: Bilde eine Modellmatrix $\tilde{X}$. Dazu wird für jede Nicht-Basis-Kategorie eine Spalte generiert, die 1 ist, wenn die jeweilige Kategorie vorliegt, und Null sonst
- Im vorherigen Beispiel werden also die Spalten $I(X_p = B)$ und $I(X_p = C)$ hinzugefügt
- Für K kategorielle Variablen mit $d_k, k = 1, \dots, K$, Ausprägungen hat die Modellmatrix $\tilde{X}$ also die Dimension $n \times (p + d_1 + d_2 + \dots + d_K - K)$; ggf. noch eine 1-Spalte für den Intercept
- Dann lässt sich das Modell kompakt schreiben als

$$Y = \tilde{X}\tilde{\beta},$$

wobei $\tilde{\beta} \in \mathbb{R}^{p+d_1+\dots+d_K-K}$.

- Der ML-Schätzer ist dann

$$\hat{\beta} = ((\tilde{X})^T \tilde{X})^{-1} (\tilde{X})^T Y.$$

- Die kategoriellen Variablen sind in **R** als Faktoren codiert

Regression mit Basisfunktionen

- So weit bekannt: Polynomiale Regression, d.h., wir verwenden o.E. nur Variable X_1 aus, aber lassen Potenzen von X_1 bis zur Ordnung D in das Modell eingehen, d.h.,

$$Y = \sum_{d=0}^D X_1^d \beta_d.$$

Dadurch bekommen wir nichtlineare Effekte (für X_1), die allerdings nicht mehr direkt interpretierbar sind, sondern nur noch die Richtung der Effekte

- Anstelle von Polynomen können wir beliebige Funktionen $h_j : \mathcal{X} \rightarrow \mathbb{R}, j = 1, \dots, p$ verwenden und Modelle der Form

$$Y = \beta_0 + \sum_j \beta_j h_j(X)$$

erzeugen.

- Die Funktionen h_j nennt man auch Basisfunktionen. Das obige Modell ist ein Spezialfall eines generalisierten additiven Modells
- Für die Berechnung der Koeffizienten wird wiederum eine Modellmatrix $\tilde{X}$ gebildet, mit Einträgen $(\tilde{X})_{ij} = (h_j(X_{ij}))_{i,j}$ und ggf. noch einer 1-Spalte für den Intercept, vgl. Vorgehen für kategoriale Variablen

■ Lin-log-Modell:

$$Y = \beta_0 + \ln(X)\beta.$$

Für kleine Änderungen von X_j kann man approximativ sagen, dass eine Veränderung von 1% von X_j zu einer Veränderung von Y von $0.01\beta_j$ führt.

■ Log-Lin-Modell:

$$\ln(Y) = \beta_0 + X\beta.$$

Hier gibt es keine Einschränkungen an Y , d.h., wir haben i.A. hier kein GLM vorliegen (vgl. Poisson-Modell, später), sondern betrachten genauer Y auf einer logarithmierten Skala. Approximativ verursacht eine Veränderung von X_j um 1 eine Veränderung von Y von $\beta_j \cdot 100\%$.

■ Log-Log-Modell:

$$\ln(Y) = \beta_0 + \ln(X)\beta.$$

Veränderung von X_j um 1% verursacht approximativ eine Änderung von Y um $\beta_j\%$.

Interaktionen zwischen Variablen

- Bisher wurden lediglich die Effekte beschrieben, die durch Änderung einer einzigen Variablen hervorgehen, wenn alle anderen festgehalten werden („ceteris paribus“). Was ist aber, wenn sich mehrere Variablen gleichzeitig ändern? Gibt es gemeinsame Effekte?
- Diese werden durch Interaktionsterme beschrieben
- Beispiel: Eine Interaktion zwischen X_j und X_k ersten Grades manifestiert sich durch den Koeffizienten $\beta_{j \cdot k}$ im Modell

$$Y = \beta_0 + X\beta + (X_{ij}X_{ik})_i\beta_{j \cdot k}.$$

- Eine Erhöhung von X_j und X_k gemeinsam um eine Einheit erhöht Y also um

$$\beta_j + \beta_k + \beta_{j \cdot k}.$$

Interaktionen zwischen Variablen

- Für die Berechnung wird eine Modellmatrix $\tilde{X}$ erstellt, die für jede Interaktion eine Extraspalte beinhaltet. Konkret wird für jede Interaktion zwischen Variablen $X_{j_1}, \dots, X_{j_l}$ (eine Interaktion $(l - 1)$ -ten Grades) die Spalte mit Einträgen $(\prod_{k=1}^l X_{ij_k})_i$ generiert.
- In R werden Interaktionen über das `formula`-Argument kenntlich gemacht, beispielsweise durch `X1*X2`
- Interaktionen zwischen kategoriellen, metrischen, aber auch transformierten Variablen sind erlaubt
- In R: Codierung über `model.matrix`
- Eine noch raffiniertere Methode zur Abbildung von Interaktionen zwischen nichtlinearen Effekten in GAM's sind die variierenden Koeffizienten

- Bisher hatten wir Modelle der Form

$$Y_i = f(X_i) + \varepsilon_i$$

mit $\varepsilon_i \sim F$ mit $\mathbf{E}[\varepsilon_i] = \mathbf{0}$ und $\text{Var}(\varepsilon_i) = \sigma^2 \in]0, \infty[$, wobei alle ε_i unabhängig identisch verteilt sind

- Konkret haben wir Homoskedastizität der Fehler angenommen
- Falls $\text{Var}(\varepsilon_i) = \sigma_i^2 \in]0, \infty[$ (i.A. aber immer noch zentriert), so spricht man von heteroskedastischen Fehlern
- Zusätzlich können die Fehler im heteroskedastischen Modell noch abhängig von den X_i sein

- Diverse Tests auf Heteroskedastizität: Breusch-Pagan-Test, Goldfeld-Quandt-Test,...
- KQ-Schätzer bleibt auch bei Heteroskedastizität unverzerrt, jedoch werden die Konfidenzintervalle verzerrt und daher unbrauchbar
- Gewichtete KQ-Schätzung: Idee, durch geeignete Gewichtung der Zeilen wieder im homoskedastischen Fall zu landen (Aitken-Schätzer, s. nächste Folie)
- Eicker-Huber-White-Schätzer: Versuche, durch geeignete Gewichtung der Zeilen die Kovarianzmatrix korrekt zu schätzen

Gewichteter KQ-Schätzer

Angenommen, es ist $\text{Cov}(\varepsilon) = \sigma^2 \mathbf{W}$ für $\mathbf{W}$ positiv definit, aber immer noch $\mathbf{E}[\varepsilon_i] = \mathbf{0}$ für alle i .

Definition 1.11

Im obigen Modell ist

$$\hat{\beta}^{\mathbf{W}} := (\mathbf{X}^T \mathbf{W}^{-1} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{-1} \mathbf{Y}$$

*der **gewichtete KQ-Schätzer**.*

Satz 1.12

- *Der gewichtete KQ-Schätzer $\hat{\beta}^{\mathbf{W}}$ ist erwartungstreu.*
- *Es ist*
$$\text{Cov}(\hat{\beta}^{\mathbf{W}}) = \sigma^2 (\mathbf{X}^T \mathbf{W}^{-1} \mathbf{X})^{-1}.$$
- *Es ist*
$$\text{Cov}(\hat{\beta}^{\mathbf{W}}) \preceq \text{Cov}(\hat{\beta})$$
für alle linearen, erwartungstreuen Schätzer $\hat{\beta}$.

- Bisher haben wir immer die quadratische Verlustfunktion $L_2(y, \hat{y}) := (y - \hat{y})^2$ betrachtet
- Erinnerung: Es ist

$$\operatorname{argmin}_a (\mathbb{E}[(X - a)^2]) = \mathbb{E}[X].$$

- Analog wird das Problem

$$\operatorname{argmin}_{f \in \mathcal{F}} ((Y - f(X))^2)$$

für die Klasse $\mathcal{F}$ aller messbaren Funktionen $f : \mathcal{X} \rightarrow \mathcal{Y}$ durch den bedingten Erwartungswert $f = \mathbb{E}[Y|X]$ gelöst

- D.h., Regression mit dem quadratischen Verlust modelliert den bedingten Erwartungswert der Response-Variable gegeben den Regressoren
- Andere Verlustfunktionen führen daher zu anderen Koeffizienten, anderen Bias-Varianz-Tradeoffs, anderen Robustheitseigenschaften der Schätzer, ...

- Kann man analog den bedingten Median einer Verteilung modellieren?
- Wie würde eine Verlustfunktion aussehen, die ein beliebiges Quantil modelliert?
- $\rightsquigarrow$ Bekannt aus der Statistik I:

$$\operatorname{argmin}_a (\mathbb{E}[|a - X|]) = \operatorname{median}(X).$$

Also führt die Verlustfunktion $L_1(y, \hat{y}) := |y - \hat{y}|$ zur **Median-Regression** (oder L_1 -Regression), bei der $\operatorname{median}(Y|X)$ gefittet wird

Definition 1.13

Definiere den **Check-Verlust**

$$L_\tau(u) := u(\tau - I(u \leq 0)).$$

Offensichtlich ist $\tau = 0.5$ ebenfalls den Median als Lösung.

Bemerkung 1.14

*Die Verwendung des Check-Verlustes führt zur sogenannten **Quantilsregression**, bei der bedingte τ -Quantile von Y gegeben X modelliert werden. Die Lösung hat allerdings keine geschlossene Form.*

- Durch Einführung von Schlupfvariablen kann das durch die Check-Verlustfunktion gegebene Optimierungsproblem auf ein lineares Programm mit Nebenbedingungen (die einen Polyeder definieren) zurückgeführt werden
- Für die Lösung werden Optimierungsverfahren wie Innere-Punkte-Verfahren oder Simplex-Verfahren verwendet
- Quantilsregression ist verteilungsfrei

- Konkret löst man bei der Quantilsregression also

$$\min_{\beta} \left(\frac{1}{n} \sum_i \tau(y_i - \mathbf{x}_i^T \beta)_+ + (1 - \tau)(y_i - \mathbf{x}_i^T \beta)_- \right).$$

- Das Problem

$$\min_{\beta} \left(\frac{1}{n} \sum_i \tau(y_i - \mathbf{x}_i^T \beta)_+^2 + (1 - \tau)(y_i - \mathbf{x}_i^T \beta)_-^2 \right)$$

führt auf die sogenannte **Expektil-Regression**.

- Expektil-Regression wird durch einen IRWLS-Algorithmus (s. Kapitel GLM) gelöst und ist daher computationell einfacher als Quantilsregression

- Quantilsregression besitzt gewisse Robustheitseigenschaften, und zwar sind die Schätzer weniger anfällig gegenüber Ausreißern in den Responses als KQ-Regression
- Robust gegenüber solchen Ausreißern ist auch die Huber-Regression, die die Verlustfunktion

$$L_{\delta}(u) := \begin{cases} u^2/2, & |u| \leq \delta \\ \delta(|u| - \delta/2), & |u| > \delta \end{cases}$$

minimiert

3. GLM

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 Lineare Regressionsmodelle
- 4 GLM
- 5 Klassische Klassifikationsverfahren
- 6 Gütemaße für Klassifikations- und Rankingverfahren
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

Exponentialfamilien

- Bisher hatten wir $\mathcal{Y} \subset \mathbb{R}$ angenommen ohne weitere Restriktionen
- Was ist, wenn die Responses zwangsläufig Werte z.B. in $\mathbb{R}_{\geq 0}$ annehmen (Länge, Gewicht, Alter etc.) oder einer anderen unendlichen Teilmenge wie $\mathbb{N}_0$ (Anzahlen)?
- Verwendet man ungeeigneten Schätzer, so besteht die Gefahr, dass man z.B. negative Längen vorhersagt
- Mathematisch korrekt gesprochen müssen wir die Verteilung $\mathcal{L}(Y|X)$ vernünftig approximieren
- Dazu brauchen wir folgende Definition:

Definition 2.1

Ein parametrisches Modell $\mathcal{P} := \{P_\theta \mid \theta \in \Theta\}$ auf einem Messraum $(\Xi, \mathcal{B})$ heißt **einparametrische Exponentialfamilie**, falls ein dominierendes, sigma-finites Maß $\mu, \xi : \Theta \rightarrow \mathbb{R}, \mathbf{b} : \Theta \rightarrow \mathbb{R}, \mathbf{c} : (\Xi, \mathcal{B}) \rightarrow (\mathbb{R}, \mathbb{B})$ und $\mathbf{a} \in \mathbb{R} > 0$ existieren, sodass

$$P_\theta(dx) = \exp\left(\frac{x\theta - \mathbf{b}(\theta)}{\mathbf{a}} + \mathbf{c}(x)\right) \mu(dx).$$

Der Parameter $\mathbf{a}$ wird auch als Dispersionsparameter bezeichnet.

Lemma 2.2

In Exponentialfamilien dürfen Integration und Differentiation beliebig vertauscht werden.

Lemma 2.3

In einer Exponentialfamilie gilt

$$E[X] = b'(\theta),$$

$$\text{Var}(X) = ab''(\theta).$$

Beispiele für Exponentialfamilien:

- Normalverteilungsfamilien $\{\mathcal{N}_d(\mu, \Sigma) \mid \mu \in \mathbb{R}^d, \Sigma \in \mathbb{R}^{d \times d} \text{ s.p.d.}\}$
- Binomialfamilien $\{\mathcal{B}(n, \mathbf{p}) \mid \mathbf{p} \in]0, 1[\}$
- Poissonfamilien $\{\mathcal{P}(\lambda) \mid \lambda > 0\}$
- Gammafamilien $\{\Gamma(r, \lambda) \mid r, \lambda > 0\}$

Keine Exponentialfamilien:

- Binomialfamilien, in denen $\mathbf{p} \in \{0, 1\}$ möglich ist
- Generalisierte Pareto-Verteilungs-Familien
- Gleichverteilungsfamilien $\{\mathcal{U}([0, \theta]) \mid \theta \in \Theta\}$

- Angenommen, alle Y_i sind unabhängig identisch gemäß einer Exponentialfamilie verteilt
- Sei $h : \mathbb{R} \rightarrow \mathbb{R}$ eine bijektive und zweimal differenzierbare Funktion
- Modelliere den bedingten Erwartungswert der Y_i gegeben den X_i durch die Beziehung

$$\mu_i := E[Y_i | X_i] = h(\eta_i) \quad (18)$$

mit $\eta_i := X_i \beta$

- Ein Modell der Form 18 heißt **generalisiertes lineares Modell (GLM)**. In dieser Situation heißen h **Responsefunktion** und $g := h^{-1}$ **Linkfunktion**

ML-Schätzung im GLM

Im GLM ist

$$L(\theta) := \prod_{i=1}^n p_{\theta}(y_i)$$

die Likelihood.

Lemma 2.4

Die ML-Methode führt zu dem Z-Schätzer

$$\text{zero}_{\beta} \left(\beta \mapsto s(\beta) := \sum_i \mathbf{X}_i (y_i - \mu_i) \frac{\partial_{\eta_i} h(\eta_i)}{\text{Var}(Y_i | X_i)} \right).$$

Die Funktion s im Lemma können wir kompakt als

$$s(\beta) = \mathbf{X}^T \mathbf{D} \Sigma^{-1} (\mathbf{Y} - \boldsymbol{\mu})$$

mit $\boldsymbol{\mu} := (\mu_i)_i$, $\Sigma := \text{diag}(\text{Var}(Y_i | X_i))$ und einer entsprechenden Diagonalmatrix $\mathbf{D}$ schreiben. Allerdings lässt sich die Z-Gleichung i.A. nicht analytisch lösen.

- Erinnerung: Newton-Verfahren zur Näherung einer Nullstelle einer stetig differenzierbaren Funktion f . Sei $\mathbf{x}_0$ ein Startwert. Dann iteriere

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{f(\mathbf{x}_k)}{f'(\mathbf{x}_k)}$$

- Iterative Berechnung eines ML-Schätzers erfolgt durch den **Newton-Raphson-Algorithmus**, welcher die Iterationen

$$\hat{\beta}^{(k+1)} = \hat{\beta}^{(k)} + \mathbf{F}^{-1}(\hat{\beta}^{(k)})\mathbf{s}(\hat{\beta}^{(k)})$$

ausführt, wobei $\mathbf{s}$ wieder die Funktion sei, deren Nullstelle zu finden ist und $\mathbf{F}$ die **Fisher-Information**

$$\mathbf{F}(\beta) := \mathbf{E}[\mathbf{s}(\beta)\mathbf{s}^T(\beta)].$$

- Die Anwendung des Newton-Raphson-Algorithmus zur Näherung des ML-Schätzers im GLM führt zu einer iterativ gewichteten KQ-Schätzung („iteratively reweighted least squares“; IRWLS)

Beispiel 2.5

Für $\mathbf{g} = \mathbf{h} = \text{id}$ erhalten wir wieder das gewöhnliche lineare Modell.

Bemerkung 2.6

Die Gleichungen in Lemma 2.3 motivieren, die Linkfunktion so zu wählen, dass $\eta_i = \theta_i$ gilt. In diesem Fall spricht man von der **kanonischen oder natürlichen Linkfunktion**.

Bemerkung 2.7

Unter Regularitätsannahmen kann man asymptotische Normalität (und damit Konsistenz) des ML-Schätzers $\hat{\beta}^{\text{GLM}}$ im GLM zeigen, konkret

$$F^{1/2}(\beta)(\hat{\beta}_n - \beta) \rightarrow \mathcal{N}(\mathbf{0}, I).$$

Fahrmeir, Kaufmann, 1985: Consistency and asymptotic normality of the maximum likelihood estimator in generalized linear models

- Im Falle von z.B. zu vorhersagenden Längen muss sichergestellt sein, dass die gefitteten Responses positiv sind
- Eine logische Wahl einer Response-Funktion ist somit $h(\eta) := \exp(\eta)$, also $g(\mu) = \ln(\mu)$. Dies führt auf das multiplikative Modell

$$E[Y_i|X_i] = \exp(\eta_i) = \exp(X_i\beta).$$

Dieses Verfahren nennt man auch **Gamma-Regression**

- Bei Zähldaten, d.h., wenn $Y_i \in \mathbb{N}_0$ für alle i , bedienen wir uns des folgenden Tricks: Wir nehmen an, dass $\mathcal{L}(Y_i|X_i) \sim P(\lambda_i)$ i.id. ist und modellieren die Rate durch ein lineares Modell, also

$$\lambda_i = \exp(X_i\beta)$$

- In R: Funktion `glm` im Paket MASS

4. Klassische Klassifikationsverfahren

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 Lineare Regressionsmodelle
- 4 GLM
- 5 Klassische Klassifikationsverfahren**
- 6 Gütemaße für Klassifikations- und Rankingverfahren
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

- Bisher: Vorhersage der Y –Werte, indem ein M-Schätzer (lineare Modelle: Minimierer des empirischen Risikos, GLMs: Maximierer der Likelihood) berechnet wurde
- In diesem Abschnitt: ML- und Distanz-basierte Verfahren zur Klassifikation
- Allerdings wird eine spezielle Gütefunktion verwendet, um die Qualität des resultierenden Modells zu bewerten
- Moderne(re) Verfahren, die explizit das ERM-Prinzip nutzen, folgen in späteren Abschnitten (Bäume, SVM, Boosting)
- Die Struktur unserer Daten D bleibt wie gehabt, lediglich der Wertebereich von Y ändert sich im Vergleich zur Regression

- Die Y_i nehmen o.E. Werte in $\{1, \dots, K\}$ an
- Diese Durchnummerierung der Klassen wird in **R** durch Codierung als `factor`-Variablen, analog wie bei kategoriellen Merkmalen
- Die statistische Modellierung erfolgt hier über klassenspezifische Dichten, d.h., wir nehmen an, dass ein Regressor $\mathbf{X}$ der Klasse k einer Dichte f_k folgt
- Bei der Diskriminanzanalyse machen wir einen Bayes'schen Ansatz
- Wir bezeichnen $\pi_k, k = 1, \dots, K$, als die Priorwahrscheinlichkeiten, dass eine zufällig ausgewählte Beobachtung der Klasse k angehört
- In der Praxis sind die π_k die relativen Anteile der Beobachtungen der Klasse k an den gesamten Beobachtungen

Satz 3.1 (Wiederholung: Satz von Bayes)

Sei $(\Omega, \mathcal{A}, P)$ ein Wahrscheinlichkeitsraum. Für $A \subset \Omega$ und einer Partition $(B_i)_{i=1}^n$ von Ω gilt

$$P(B_i|A) = \frac{P(A|B_i)P(B_i)}{\sum_j P(A|B_j)P(B_j)}.$$

- Wir wollen nun die Posterior-Wahrscheinlichkeiten $P(Y = k|X)$ berechnen
- **LDA-Annahme:** Es ist f_k jeweils die Dichte einer $\mathcal{N}_p(\mu_k, \Sigma)$ –Verteilung mit $\mu_k \in \mathbb{R}^p, \Sigma \in \mathbb{R}^{p \times p}$
- Beachte: Σ hängt nicht von k ab!

Lineare Diskriminanzanalyse (LDA)

Vorgehen für den Fall $K = 2$:

- Vergleiche die Posterior-Wahrscheinlichkeiten durch Quotientenbildung. Um die Rechnungen zu vereinfachen, bilde den logarithmierten Quotienten

$$\ln \left(\frac{P(Y = 1|X)}{P(Y = 2|X)} \right).$$

- Nun werden die Parameter μ_k und Σ anhand der Daten geschätzt und der Quotient konkret ausgewertet. Ist er größer als Null, d.h., ist die Posterior-Wahrscheinlichkeit, dass Y der Klasse 1 angehört größer als eine Zugehörigkeit zur Klasse 2, klassifiziere die Beobachtung nach Klasse 1 und vice versa

Lineare Diskriminanzanalyse (LDA)

Allgemeiner Fall:

Satz 3.2

Es seien

$$\delta_k(\mathbf{x}) := \ln(\pi_k) - \frac{1}{2}\mu_k^T \Sigma^{-1} \mu_k + \mathbf{x}^T \Sigma^{-1} \mu_k \quad (19)$$

die **lineare Diskriminanzfunktionen**. Dann ist die **Trennungsgerade** zwischen Klassen $k \neq l$ gegeben durch

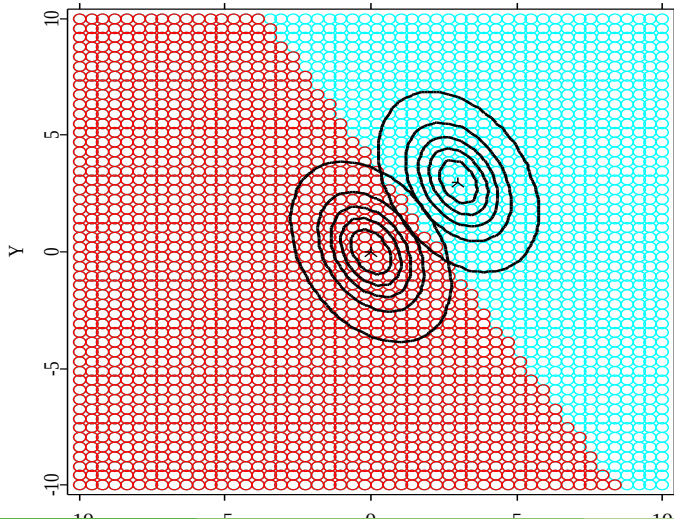
$$\{\mathbf{x} \in \mathbb{R}^p \mid \delta_k(\mathbf{x}) = \delta_l(\mathbf{x})\}.$$

Der **LDA-Klassifizierer** ist dann

$$\hat{Y}_i = \operatorname{argmax}_k(\delta_k(\mathbf{X}_i)). \quad (20)$$

- Vorteil: Einfache Berechnung des Klassifizierers
- Nachteil: Gleiche Varianzstruktur in allen Klassen ist evtl. nicht realistisch
- In R: Funktion `lda` im Paket MASS

Lineare Diskriminanzanalyse (LDA)



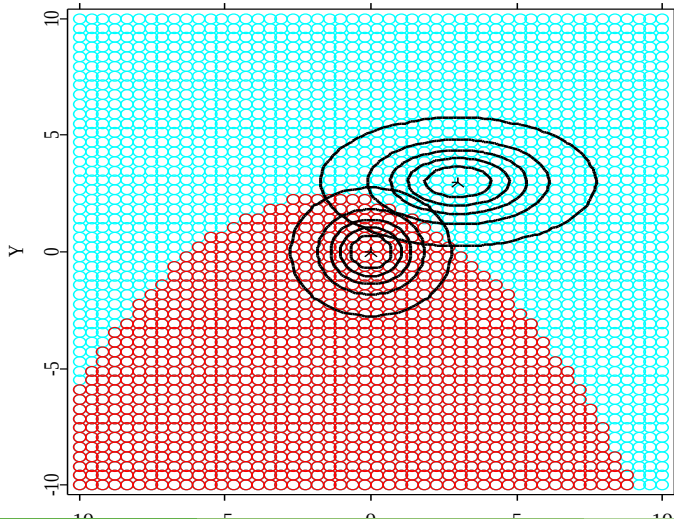
Quadratische Diskriminanzanalyse (QDA)

- Bei der QDA nehmen wir immer noch klassenspezifische Normalverteilungen an, jedoch mit klassenspezifischen Kovarianzmatrizen Σ_k .
- Dies führt auf **quadratische Diskriminanzfunktionen**

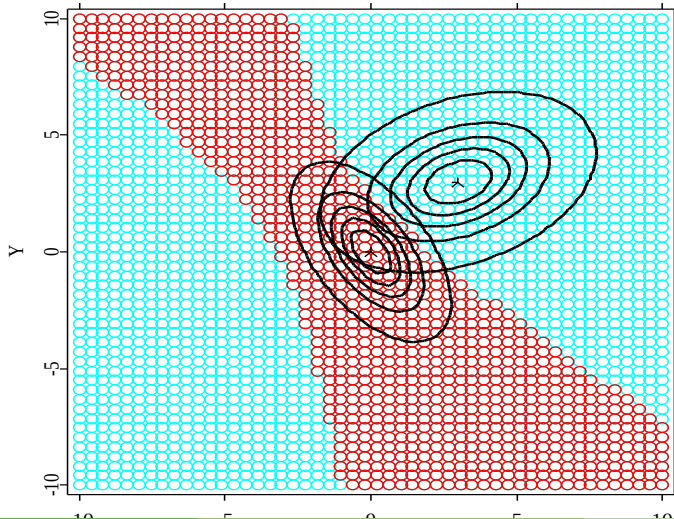
$$\delta_k(\mathbf{x}) = -\frac{1}{2} \ln(\det(\Sigma_k)) + \ln(\pi_k) - \frac{1}{2} (\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k)$$

- Vorteil gegenüber LDA: Die Trennungsgrenzen sind nicht mehr notwendigerweise linear wie bei der LDA, d.h., QDA kann deutlich flexiblere Strukturen abbilden

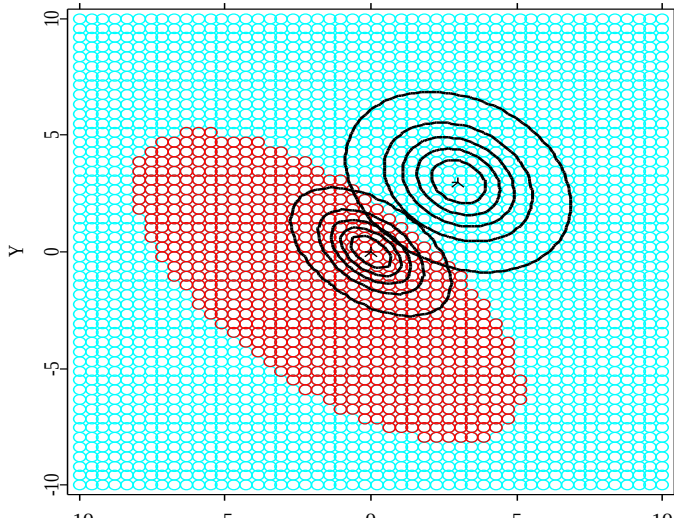
Quadratische Diskriminanzanalyse (QDA)



Quadratische Diskriminanzanalyse (QDA)



Quadratische Diskriminanzanalyse (QDA)



- Wir hatten bereits die Poisson-Regression, um $\mathbb{N}_0$ –wertige Daten zu modellieren
- GLMs können durch geeignete Linkfunktionen und Diskretisierung sogar für Klassifikation eingesetzt werden
- Grundidee: Transformiere die $\mathbb{R}$ –wertige Größe $X\beta$ bijektiv auf ein kompaktes Intervall, in der Regel $[0, 1]$
- Führe einen Cutoff ein und klassifiziere eine Instanz nach Klasse 1, wenn ihr transformierter Wert größer als der Cutoff ist
- Für zwei Klassen gibt es zwei populäre Modelle: Das Logit- und das Probit-Modell

- Annahme: $Y_i|X_i$ sind $B(1, p_i)$ –verteilt mit einem unbekannten Parameter p_i
- Damit ist $\mu_i = E[Y_i|X_i] = p_i$
- Idee: Verwende ein GLM, um die μ_i zu erklären
- Der natürliche Link ist hier der **Logit-Link**, also

$$\ln \left(\frac{p_i}{1 - p_i} \right) = X_i \beta.$$

Die linke Seite bezeichnet man auch als Log-Odds

- Damit ist

$$p_i = \frac{\exp(X_i \beta)}{1 + \exp(X_i \beta)}.$$

Die Inverse der Logit-Funktion nennt sich auch **Expit-Funktion**

- Das Logit-Modell wird manchmal als logistische Regression bezeichnet

- Allgemein für K Klassen haben wir jeweils $p_{ik}(x) := P(Y_i = k|X = X_i)$ für alle k
- Somit folgt $Y_i|X_i$ jeweils einer Multinomialverteilung
- Die Strategie ist nun, $(K - 1)$ verschiedene Log-Odds zu modellieren, jeweils mit Referenzklasse K , also

$$\ln \left(\frac{P(Y_i = k|X = X_i)}{P(Y_i = K|X = X_i)} \right) = X_i \beta_k \quad \forall k = 1, \dots, K - 1.$$

- Das führt zu

$$P(Y_i = k|X = X_i) = \frac{\exp(X_i \beta_k)}{1 + \sum_{l=1}^{K-1} \exp(X_i \beta_l)}.$$

- Die Vorhersage ist damit

$$\hat{Y}_i = \operatorname{argmax}_k (P(Y_i = k | X_i)).$$

- Im Fall $K = 2$ klassifizieren wir nach Klasse 1, wenn $p = P(Y = 1) > 0.5$ ist, oder äquivalent, wenn die Log-Odds größer ist als Null
- Der Logit-Link ist zwar die natürliche Link-Funktion, aber prinzipiell ist jede stetig differenzierbare Funktion $h : \mathbb{R} \rightarrow [0, 1]$ erlaubt
- Eine Alternative ist die Inverse der Verteilungsfunktion Φ der Standardnormalverteilung. Das führt auf ein **Probit-Modell**, für $K = 2$ ist das

$$P(Y = 1 | X) = \Phi(X\beta).$$

- Auch hier klassifizieren wir nach Klasse 1, wenn $\Phi(X\beta)$ größer als 0.5 ist oder äquivalent, wenn $X\beta > 0$ ist
- Das Probit-Modell lässt sich analog für $K > 2$ Klassen erweitern

- Da es sich bei beiden Modellen um GLMs handelt, wird zur Berechnung der Koeffizienten der Newton-Raphson-Algorithmus verwendet
- Bei beiden Modellen können die Koeffizienten nicht direkt interpretiert werden, lediglich deren Vorzeichen
- Logit- und Probit-Modell liefern unterschiedliche Koeffizienten, die Modelle sind gemessen an ihrer Performance aber i.A. sehr ähnlich
- Wesentlicher Unterschied zwischen LDA und Logit: Logit braucht keine Normalverteilungs-Annahme der klassenbedingten Dichten
- In der Praxis ist dennoch nicht klar, ob LDA oder Logit besser ist

k nearest neighbors (kNN)

- Idee: Gegeben sei ein neuer Regressor X^{neu} . Finde den Regressor aus den Trainingsdaten, dem X^{neu} am nächsten ist und sage dessen Klasse für Y^{neu} voraus
- Wir benötigen also eine Metrik d , die einen Abstand zwischen Regressoren berechnet
- Besser: Benutze k nächste Nachbarn von X^{neu} und aggregiere deren Klassen geeignet
- Sei $\mathcal{N}_k(X^{neu})$ die Menge dieser k nächsten Nachbarn
- Dann mache die Vorhersage

$$\hat{Y}^{neu} = \underset{l}{\operatorname{argmax}} \left(\sum_{X_i \in \mathcal{N}_k(X^{neu})} I(Y = l) \right).$$

Tatsächlich auch für Regression möglich:

$$\hat{Y}^{neu} = \frac{1}{k} \sum_{X_i \in \mathcal{N}_k(X^{neu})} Y_i$$

k nearest neighbors (kNN)

- Bei kNN wird also kein Modell auf den Trainingsdaten gefittet!
- Die Wahl der Anzahl k der Nachbarn stellt einen Bias-Varianz-Tradeoff dar
- Das optimale k wird in der Praxis über Kreuzvalidierung ermittelt
- Es gibt zahlreiche Verbesserungen des kNN
- Der gewichtete kNN gibt den Regressoren, die näher an X^{neu} liegen, auch mehr Gewicht

- Wächst die Dimension p des Raumes, in dem die Regressoren Werte annehmen, so kann der nächste Nachbar bereits sehr weit entfernt sein
- Unvorteilhafte Phänomene durch die wachsende Dimension des Regressors werden oft unter **curse of dimensionality** zusammengefasst
- Man versucht daher, adaptiv zu den Daten eine geeignete Metrik zu wählen
- Mit den empirischen Klassenhäufigkeiten π_k , den Mittelwerten $\bar{x}_k$ innerhalb der Klassen, den Varianzen $\mathbf{W}_k$ innerhalb der Klassen und der Varianz $\mathbf{B}$ zwischen den Klassen verwendet man

$$D(\mathbf{x}, \mathbf{x}') = (\mathbf{x} - \mathbf{x}')^T \mathbf{W}^{-1/2} [\mathbf{W}^{-1/2} \mathbf{B} \mathbf{W}^{-1/2} + \varepsilon \mathbf{I}] \mathbf{W}^{-1/2} (\mathbf{x} - \mathbf{x}'),$$

mit $\mathbf{W} = \sum_k \pi_k \mathbf{W}_k$ und einem zu wählenden Parameter ε

- In R: knn und kknns

5. Gütemaße für Klassifikations- und Rankingverfahren

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 Lineare Regressionsmodelle
- 4 GLM
- 5 Klassische Klassifikationsverfahren
- 6 Gütemaße für Klassifikations- und Rankingverfahren**
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

- Wesentlicher Unterschied bisher zu den Regressionsmethoden: Es wurde bei den Klassifikationsverfahren bislang keine Verlustfunktion betrachtet, lediglich Likelihoods und Distanzen
- Wie kann man konkurrierende Klassifikationsverfahren vergleichen?
- Es sind klassische Güte- und Verlustzahlen verfügbar, die jeweils einzeln aber nicht unbedingt informativ genug sind
- Sehr illustrativ und auch für das Tuning von Klassifikationsverfahren verwendbar: Graphische Tools, insbesondere die ROC-Kurve (s. unten)

Einfache Gütebemessungen eines Klassifikators

- Universell einsetzbar: Fehlklassifikationsrate, d.h., der Anteil der falsch klassifizierten Instanzen:

$$(L_{0/1})_n(\mathbf{Y}, \hat{\mathbf{Y}}) := \frac{1}{n} \sum_i L_{0/1}(Y_i, \hat{Y}_i) := \frac{1}{n} \sum_i I(Y_i \neq \hat{Y}_i)$$

für die Anzahl n der Komponenten von $\mathbf{Y}$ und $\hat{\mathbf{Y}}$

- Vorteil: Sehr leicht zu berechnen für jede beliebige Zahl K von Klassen
- Nachteil: Symmetrie, d.h., bei $K = 2$ werden ggf. Fehler 1. und 2. Art gleichwertig behandelt, wenn Klasse 1 o.E.: die „interessante“ und Klasse (-1) die „uninteressante“ ist
- Für den speziellen Fall $K = 2$ gibt es eine ganze Reihe an anderen Gütequantifikationen, die sich oft leicht an der **Konfusionsmatrix** ablesen lassen

Konfusionsmatrix

	Positive	Negative
Predicted positive	✓ (True positive; TP)	False positive (FP; Type I error)
Predicted negative	False negative (FN; Type II error)	✓ (True negative; TN)

- Sei n die Anzahl aller Instanzen und n_+ , n_- die wahre Anzahl der Instanzen aus Klasse 1 bzw. (-1)
- Die Fehlklassifikationsrate ist somit $(FP + FN)/n$
- True positive rate (TPR, Sensitivität): TP/n_+
- True negative rate (TNR, Spezifizität): TN/n_-
- False positive rate (FPR): FP/n_-
- False negative rate (FNR): FN/n_+
- False discovery rate (FDR): $FP/(FP + TP)$

Receiver Operation Characteristic (ROC)

- Eine Score ist eine Funktion $s : \mathcal{X} \rightarrow \mathbb{R}$
- Bei Logit- und Probit-Modellen haben wir jeder Instanz eine Score (in $[0, 1]$) zugewiesen
- Per Default war bisher 0.5 der Cutoff für die Klassifikation
- Wenn die Klassen aber asymmetrisch sind, also Fehler 1. Art gravierender sind als Fehler 2. Art, kann man dann den Cutoff geeignet anders wählen?
- Idee: Versuche, eine möglichst große TPR auf Kosten einer möglichst kleinen FPR zu erhalten
- Dazu zeichne die **empirische ROC-Kurve**

Receiver Operation Characteristic (ROC)

Vorgehen für das Zeichnen der ROC-Kurve:

Ordne die Scores $s_1, \dots, s_n$, die den Instanzen zugewiesen wurden, an ;

Erhalte $s_{(1)} \leq s_{(2)} \leq \dots \leq s_{(n)}$;

for $i=1, \dots, n$ **do**

 Wähle $s_{(i)}$ als Cutoff ;

 Klassifiziere $\hat{Y}_{(k)} = I(k \geq i)$ für alle $k = 1, \dots, n$;

 Berechne die TPR und die FPR und nenne sie TRP_i, FPR_i

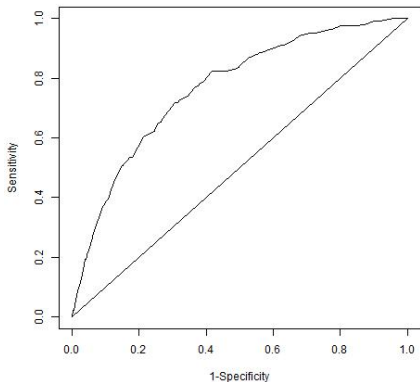
end

Plote den Vektor $(0, (TRP_i)_i, 1)$ gegen den Vektor $(0, (FPR_i)_i, 1)$;

Interpoliere zwischen den Punkten linear ;

Receiver Operation Characteristic (ROC)

Man erhält dann einen Plot der Form



Die Menge aller annehmbaren Punkte (FPR , TPR) wird auch **ROC-Raum** genannt

- Anschaulich ist klar, dass der Punkt der ROC, der am nächsten an $(0, 1)$ liegt, den besten Tradeoff zwischen hoher TPR und niedriger FPR liefert
- Optimal wäre eine ROC aus den Punkten $(0, 0)$, $(0, 1)$, $(1, 1)$. Ein solcher Klassifizierer könnte alle positiven Instanzen erwischen, ohne eine einzige fälschlicherweise als positiv zu klassifizieren
- „Random guessing“, d.h., man ordnet zufällig mit Wahrscheinlichkeit 0.5 jeder Instanz eine der beiden Klassen zu, führt theoretisch auf die Winkelhalbierende als ROC
- Die ROC-Kurve liefert somit Informationen für die Güte des Klassifizierers für jeden sinnvollen Cutoff
- Diese gesamte Information kann durch eine einzige Zahl ausgedrückt werden, nämlich die **AUC (Area under the ROC curve)**

- Annahme: $s(X)|Y$ folgt den Verteilungen F_- für $Y = -1$ und F_+ für $Y = 1$
- Dann ist die theoretische AUC (wenn keine Ties vorliegen) gegeben durch

$$AUC(s) = \int F_+(s) dF_-(s) = P((s(X)|Y = 1) > (s(X)|Y = -1)).$$

- Die empirische AUC lässt sich durch die Mann-Whitney-Statistik schätzen, nämlich

$$\widehat{AUC}(s) := \frac{1}{n_+ n_-} \sum_{i \in \mathcal{N}_+} \sum_{k \in \mathcal{N}_-} I(s(X_i) > s(X_k)) + \frac{1}{2} I(s(X_i) = s(X_k))$$

mit den Mengen $\mathcal{N}_*$ der Zeilenindices der Instanzen mit Klassenvorzeichen $* \in \{+, -\}$

- Wenn keine Ties vorliegen, so fallen alle Summanden mit Faktor $\frac{1}{2}$ weg

Binäre Klassifikation und binäres Ranking

- Sehr eng verwandt: Binäre Klassifikation und binäres Ranking
- Konkret lässt sich ein binäres Ranking oft als gewichtete binäre Klassifikation auffassen
- Die Scorefunktionen s , die z.B. das Logit-Modell ausgibt, sind bereits **Rankingfunktionen**, d.h., Funktionen, die Regressoren (in $\mathcal{X}$) einen Wert in einer geordneten Menge, hier o.E. $\mathbb{R}$, zuweisen
- O.E. ranken wir X_i höher als X_j genau dann, wenn $s(X_i) > s(X_j)$ ist
- Während bei der binären Klassifikation „nur“ die richtige Klasse vorhergesagt werden sollte, soll beim binären Rankingproblem die richtige stochastische Ordnung vorhergesagt werden, d.h., wenn $P(Y = 1|X_i) > P(Y = 1|X_j)$ ist, dann soll auch $s(X_i) > s(X_j)$ sein
- Techniken, wie die Score-Funktion so bestimmt werden kann, dass sie möglichst gute Rankings liefert, siehe später

Konzentrationsungleichungen

Lemma 4.1

Sei X eine Zufallsvariable mit $E[X] = 0$ und $P(X \in [a, b]) = 1$. Dann gilt für alle $s > 0$, dass

$$E[\exp(sX)] \leq \exp\left(\frac{s^2(b-a)^2}{8}\right).$$

Satz 4.2 (Hoeffdings Ungleichung)

Seien $X_1, \dots, X_n$ unabhängige Zufallsvariablen mit $P(X_i \in [a_i, b_i]) = 1$ für alle i (o.E. $E[X_i] = 0 \forall i$). Mit $S_n := \sum_i X_i$ gilt

$$P(|S_n - E[S_n]| \geq \varepsilon) \leq \exp\left(\frac{-2\varepsilon^2}{\sum_i (b_i - a_i)^2}\right).$$

Konzentrationsungleichungen

Satz 4.3 (Azumas Ungleichung)

Sei $(X_i)_i$ eine Folge von Zufallsvariablen und dazu $(V_i)_i$ eine Folge von Martingaldifferenzen bzgl. $(X_i)_i$. Seien $(Z_i)_i = (Z_i(X_1, \dots, X_{i-1}))_i$ und $c_i > 0$ so, dass

$$P(Z_i \leq V_i \leq Z_i + c) = 1. \quad (21)$$

Dann gilt für alle $\varepsilon > 0$ und alle n , dass

$$P\left(\left|\sum_i V_i\right| \geq \varepsilon\right) \leq \exp\left(\frac{-2\varepsilon^2}{\sum_i c_i^2}\right).$$

Satz 4.4 (McDiarmids Ungleichung)

Seien $X_1, \dots, X_n$ i.i.d. Zufallsvariablen mit Werten in $\mathcal{X}_1, \dots, \mathcal{X}_n$. Sei $s : \bigotimes_i \mathcal{X}_i \rightarrow \mathbb{R}$ so, dass

$$\sup_{x_i \in \mathcal{X}_i, x'_k \in \mathcal{X}_k} (|s(x_1, \dots, x_n) - s(x_1, \dots, x_{k-1}, x_k, x_{k+1}, \dots, x_n)|) \leq c_k \quad \forall k \quad (22)$$

Dann gilt für alle $\varepsilon > 0$, dass

$$P(|s(x_1, \dots, x_n) - \mathbb{E}[s(X_1, \dots, X_n)]| \geq \varepsilon) \leq 2 \exp\left(\frac{-2\varepsilon^2}{\sum_k c_k^2}\right).$$

Genauigkeit der empirischen AUC

Lemma 4.5

Die Mann-Whitney-Statistik ist ein unverzerrter Schätzer für die Ranking-Genauigkeit

$$A(s) := \mathbb{E}_{X \sim F_+, X' \sim F_-} \left[I(s(X) > s(X')) + \frac{1}{2} I(s(X) = s(X')) \right].$$

Agarwal et. al., 2005: Generalization bounds for the area under the ROC curve

Satz 4.6

Sei $s : \mathcal{X} \rightarrow \mathbb{R}$ eine Scoring-Funktion und seien $\hat{Y}_1, \dots, \hat{Y}_N \in \{-1, 1\}$ Vorhersagen. Sei $\hat{n}_-, \hat{n}_+$ jeweils die Anzahl der negativen bzw. positiven Vorhersagen. Dann ist

$$P(|\widehat{AUC}(s) - A(s)| \geq \varepsilon) \leq 2 \exp \left(\frac{-2n_+n_- \varepsilon^2}{n_+ + n_-} \right).$$

Agarwal et. al., 2005: Generalization bounds for the area under the ROC curve

- Sei x der Cutoff für die binäre Klassifikation, d.h., $\hat{Y}_i = I(s(X_i) \geq x)$
- Seien

$$TPR_s(x) := P(s(X) \geq x \mid Y = 1),$$

$$FPR_s(x) := P(s(X) \geq x \mid Y = -1)$$

- Betrachte $t \mapsto q_{s,\alpha} := \inf\{t \in]0, 1[\mid FPR_s(x) \leq \alpha\}$.
- Wir können die ROC also äquivalent durch Plotten von $\beta_s(\alpha) := TPR_s(q_{s,\alpha})$ gegen α darstellen. Das ist nützlich bei Asymmetrie, da man so einen Graphen der empirischen Macht gegen das Niveau erhält

Ordered Discrete Choice Model

- Situation: n Instanzen, deren Response-Variable $K < n$ geordnete Ausprägungen hat, o.E. seien die Klassenlabels $1, \dots, K$ und tatsächlich als $1 \leq 2 \leq \dots \leq K$ zu verstehen
- Das ist spezieller als bei K -facher Klassifikation, da dort die Klassen nicht geordnet sein mussten
- Es ist schwächer als Ranking, da Instanzen mit gleichem Klassenlabel nicht unterschieden werden
- Zur Vorhersage der Klassen unter Ausnutzung der Ordnung wird ein Threshold-Modell verwendet, d.h., man definiert Hilfs-Responses

$$Y' = X\beta + \varepsilon$$

für $\varepsilon \sim F_\varepsilon$

- Aus Vorhersagen $\hat{Y}'$ werden Vorhersagen $\hat{Y}$ abgeleitet durch

$$\hat{Y} = r \Leftrightarrow Y' \in]\gamma_{r-1}, \gamma_r]$$

für Thresholds

$$-\infty =: \gamma_0 < \gamma_1 < \dots < \gamma_K := \infty.$$

Ordered Discrete Choice Model

- Dies wird in ein Logit-Modell übersetzt, indem man sich die Wahrscheinlichkeiten $P(Y \leq k)$ ansieht
- Offenbar gilt

$$P(Y \leq k) = P(Y' \leq \gamma_k) = P(\varepsilon \leq -X\beta + \gamma_k) = F_\varepsilon(\gamma_k - X\beta).$$

- Der Koeffizient β kann durch ML-Schätzung berechnet werden
- Nimmt man an, dass F_ε eine spezielle logistische Verteilung mit der Expit-Funktion als Verteilungsfunktion ist, erhält man das

Ordered-Logit-Modell

$$\ln \left(\frac{P(Y \leq r|X)}{P(Y > r|X)} \right) = \gamma_r - X\beta.$$

Definition 4.7

Seien

$$F_{s,k}(t) := P(s(x) \leq t \mid Y = k).$$

Sei

$$\Delta := \{t \in \mathbb{R}^{K-1} \mid t_1 < \dots < t_{K-1}\}.$$

Dann ist die **ROC-Fläche** definiert als die stetige Erweiterung der Abbildung

$$\Delta \rightarrow [0, 1]^K, t \mapsto (F_{s,1}(t_1), F_{s,2}(t_2) - F_{s,1}(t_1), \dots, 1 - F_{s,K}(t_{K-1})).$$

- Stetige Erweiterung heißt hier lineare Interpolation zwischen den Punkten
- Liegt die Score zwischen den Thresholds t_k und t_{k+1} , $k = 2, \dots, K-2$, so ordne die Instanz der Klasse $(k+1)$ zu, und wenn t_1 unter- bzw. t_{K-1} überschritten wird, dann der Klasse 1 bzw. K
- Die k -te Komponente quantifiziert jeweils die Wahrscheinlichkeit, ein Objekt, dass aus Klasse k stammt, auch in Klasse k zu klassifizieren

- Die ROC-Fläche kann als gemeinsamer Plot der klassenweisen TPR's aufgefasst werden
- Man kann die ROC-Fläche empirisch schätzen
- Das Pendant zur AUC ist hier das Volumen unter der ROC-Fläche (VUS (Volume under the ROC surface))
- Für „random guessing“ und $K = 3$ erhalten wir ein theoretisches VUS von $1/6$

6. Clustering

- 1 Organisation
 - Termine
 - Prüfungsleistung/Übungen
- 2 Motivation
- 3 Lineare Regressionsmodelle
- 4 GLM
- 5 Klassische Klassifikationsverfahren
- 6 Gütemaße für Klassifikations- und Rankingverfahren
- 7 Clustering
- 8 Modellwahl und Regularisierung
- 9 Bäume
- 10 Support Vector Machines
- 11 Boosting
- 12 Ensemble-Methoden
- 13 Ranking
- 14 Online Learning

- Gegeben sei eine Datenmatrix $D = X \in \mathbb{R}^{n \times p}$
- Mangels Responsevariable muss man anderweitig Ähnlichkeiten zwischen den Regressoren, also den Zeilen X_i , finden
- Gebe eine Zahl $K > 1$ vor und versuche, die Beobachtungen so in Cluster $C_1, \dots, C_K$ zu partitionieren, sodass innerhalb der Cluster möglichst große Ähnlichkeit herrscht
- Wir schreiben $C(i) = k$, wenn die Beobachtung X_i ins Cluster C_k eingruppiert wird
- Offenbar muss jedes Cluster mindestens eine Beobachtung enthalten und jede Beobachtung fällt in genau ein Cluster

Partitionierendes Clustern

- Sei d ein Abstandsmaß zwischen zwei Punkten
- d muss keine Metrik sein, denn Symmetrie und Dreiecksungleichung sind nicht erforderlich, lediglich $d(x, y) \geq 0$ mit $d(x, y) = 0 \Leftrightarrow x = y$ sind notwendig
- Dann ist die innere Varianz („within cluster point scatter“)

$$W(C) := \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=k} \sum_{C(i')=k} d(X_i, X_{i'})$$

und die äußere Varianz („between cluster point scatter“)

$$B(C) = \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=k} \sum_{C(i') \neq k} d(X_i, X_{i'}).$$

- Die totale Varianz ist $T(C) = B(C) + W(C)$, d.h., die Minimierung von $W(C)$ ist äquivalent zur Maximierung von $B(C)$.

Bemerkung 5.1

Die Anzahl an möglichen Partitionen der n Beobachtungen in K Cluster ist

$$\frac{1}{K!} \sum_{k=1}^K (-1)^{K-k} \binom{K}{k} k^n.$$

- Daher haben wir es mit einem kombinatorischen Problem zu tun
- Generell würde man versuchen, eine geeignete Teilmenge der Menge aller möglichen Partitionen zu finden und diese auszuprobieren
- Alternativ kann man versuchen, iterativ die Zuordnungen so zu ändern, dass $W(\mathbf{C})$ in jeder Iteration kleiner wird. Ist das nicht möglich, ist der Algorithmus beendet
- Problem: Man läuft in lokale Optima und diese können sehr weit vom globalen Optimum entfernt sein

Lemma 5.2

Sei $d(\mathbf{x}, \mathbf{y}) := \|\mathbf{x} - \mathbf{y}\|_2^2$. Dann gilt, dass

$$W(\mathbf{C}) = \frac{1}{2} \sum_{k=1}^K \sum_{\mathbf{C}(i)=k} \sum_{\mathbf{C}(i')=k} \|\mathbf{x}_i - \mathbf{x}_{i'}\|_2^2 = \sum_{k=1}^K \mathcal{N}_k \sum_{\mathbf{C}(i)=k} \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|_2^2, \quad (23)$$

mit $\mathcal{N}_k := |\mathbf{C}_k|$ für alle k und

$$\bar{\mathbf{x}}_k = (\bar{x}_{k1}, \dots, \bar{x}_{kp}) \quad \text{mit} \quad \bar{x}_{kj} = \frac{1}{\mathcal{N}_k} \sum_{\mathbf{C}(i)=k} x_{ij}.$$

- Durch das obige Lemma ist also unser Optimierungsproblem nun

$$\min_{\mathbf{C}} \left(\sum_k \mathcal{N}_k \sum_{\mathbf{C}(i)=k} \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|_2^2 \right).$$

- Aber: Immer noch K^n Möglichkeiten, eine Partition zu bilden

Betrachte das vergrößerte Optimierungsproblem

$$\min_{C, m_1, \dots, m_K} \left(\sum_k \mathcal{N}_k \sum_{C(i)=k} \|X_i - m_k\|_2^2 \right)$$

Dieses lässt sich durch folgenden Algorithmus iterativ lösen:

Verteile zufällig die Indices $1, \dots, K$ auf die Beobachtungen ;

for $i = 1, \dots, n$ **do**

 Löse

$$C(i) = \operatorname{argmin}_k (\|X_i - m_k\|_2^2)$$

end

for $k = 1, \dots, K$ **do**

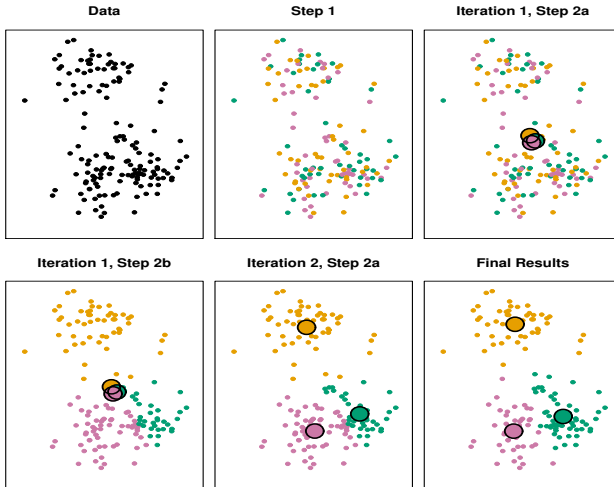
 Berechne $m_k = \bar{X}_k$

end

Ende, sobald sich die Zuordnungen nicht mehr ändern

K-Means

Beispiel für $K = 3$



6 Durchläufe von K –Means mit unterschiedlichen Prototypen im ersten Schritt



- Wird ein Cluster durch eine Beobachtung repräsentiert, so nennt man diese Cluster-Prototyp oder Zentroid
- Im ersten Schritt werden also zufällig Prototypen verteilt
- Anschließend werden die Beobachtungen dem Cluster mit dem am nächsten gelegenen Prototypen zugeordnet
- Computationell wesentlich aufwändiger, aber robuster, ist die Verwendung der l_1 –Norm. Der entsprechende Algorithmus heißt dann **K–Medoids**
- In R: `kmeans (stats)`, Paket `cluster`

- K –Means kann auch für Klassifikationsprobleme verwendet werden. Ist K die Anzahl der Klassen, verwende K Cluster mit jeweils R Prototypen. Klassifiziere anschließend eine neue Beobachtung in die Klasse, die zum Cluster gehört, in dem der nächste Prototyp liegt
- Es gibt zahlreiche Ansätze zur Beschleunigung von K –Means
- Anschauliche Bedeutung von Robustheit beim Clustern: Ein Ausreißer in den Daten sollen nicht dazu führen, dass um diesen ein Cluster gebildet wird, das also nur diese Beobachtung enthält
- Weder K –Means noch K –Medoids sind robust in diesem Sinne

Hard vs. soft clustering

- Hard/crisp clustering: Jede Beobachtung wird genau einem Cluster zugeordnet
- Beispiel: K –Means, K –Medoids
- Soft/fuzzy/probabilistic clustering: Zu jeder Beobachtung wird für jedes Cluster eine Wahrscheinlichkeit der Zugehörigkeit angegeben
- Allgemein ist modellbasiertes Clustern soft clustering (hier nicht)
- Die Anwendung des EM-Algorithmus führt zu einer probabilistischen Version von K –Means
- Lege dazu ein Gauß'sches Misch-Modell zugrunde, d.h., die Dichte von $\mathbf{X}$ ist gegeben durch

$$f(\mathbf{x}) = \sum_{k=1}^K \pi_k f_k(\mathbf{x})$$

mit Normalverteilungs-Dichten f_k

- Im E-Schritt werden die Zugehörigkeits-Wahrscheinlichkeiten aufdatiert und im M-Schritt basierend auf diesen die Parameter jeder Mischkomponente aufdatiert

Hierarchisches Clustern

- Beim partitionierenden Clustern haben wir versucht, direkt ein Optimierungsproblem zu lösen. Davon abgesehen startete der K –Means-Algorithmus im ersten Schritt schon mit K (zufälligen) Prototypen
- Die Zahl K der Cluster musste im Vorfeld festgelegt werden. Das ist in der Praxis oft eine Schwierigkeit
- Kann man Strategien finden, in denen man die Zahl der Cluster sukzessive erhöht oder verringert?
- $\leadsto$ Hierarchisches Clustern
- Man bezeichnet die Situation, in der ein einziges Cluster vorliegt, welches alle Beobachtungen enthält, als oberste Stufe, und diejenige, in der jede Beobachtung ein eigenes Cluster bildet, als unterste Stufe
- Idee: Starte mit der obersten Stufe und trenne die Cluster geeignet auf (**divisiv**) oder starte mit der untersten Stufe und vereinige geeignet die Cluster (**agglomerativ**)

Hierarchisches Clustern

- Problem: Für beide Strategien muss man einen Abstand zwischen Clustern definieren
- Zunächst: Abstand von einem Punkt zu einer Gruppe

Definition 5.3

Sei G eine Menge und sei

$$d'(x, G) := \{d(x, y) \mid y \in G\}.$$

Dann gibt es folgende Distanzen zwischen x und G :

1 single linkage / nearest neighbor

$$d_S(x, G) := \min(d'(x, G)),$$

2 complete linkage / furthest neighbor

$$d_C(x, G) := \max(d'(x, G)),$$

3 average linkage

$$d_A(x, G) := \text{mean}(d'(x, G)),$$

4 centroid linkage

$$d_P(x, G) := (d(x, P)),$$

mit einem Zentroiden P von G .

- Die Definition wird erweitert auf Abstände zwischen Gruppen durch

$$d''(F, G) := \{d_*(x, G) \mid x \in F\}$$

für $* \in \{S, C, A, P\}$. Anschließend wird mit dieser Menge genau so verfahren wie in der Definition mit $d'(x, G)$.

- Agglomeratives Clustern funktioniert also folgendermaßen:
 - i) Berechne alle paarweisen Abstände d_* zwischen den Clustern
 - ii) Fusioniere die beiden Cluster mit dem niedrigsten Abstand
- Die alternierende Ausführung von i) und ii) geschieht so lange, bis nur noch ein Cluster vorliegt, man also auf der höchsten Stufe angekommen ist

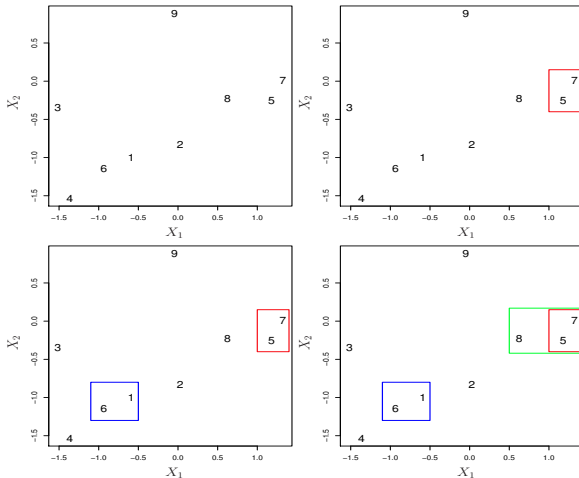
- Divisives Clustern funktioniert umgekehrt:
 - i) Berechne für jede Beobachtung die Distanz d_* zu dem Cluster, dem sie aktuell angehört
 - ii) Wähle die Beobachtung mit der größten in i) berechneten Distanz. Diese bildet ein neues Cluster
 - iii) Transferiere geeignet Beobachtungen aus dem ursprünglichen Cluster in das neue Cluster
- Das genaue Vorgehen in iii) hängt vom konkreten Algorithmus ab
- Bei beiden Clusterverfahren hat man für jede Stufe eine Partition der Beobachtungen erhalten
- Diese kann als **Dendrogramm** visualisiert werden, bei dem die oberste Stufe die Wurzel und die unterste Stufe die Blätter sind (s. auch Kapitel Bäume)

Hierarchisches Clustern

- Auch beim hierarchischen Clustern ist die Wahl von K nicht klar und lässt sich auch aus dem Dendrogramm nicht ablesen
- Die Wahl der konkreten Distanz zwischen den Gruppen ist auch nicht klar
- Hierarchisches Clustern ist nicht robust
- Je nach Art der Daten macht ein hierarchisches Clustern eventuell keinen Sinn, insbesondere bei kategoriellen Variablen
- In R: Pakete `hclust`, `cluster`

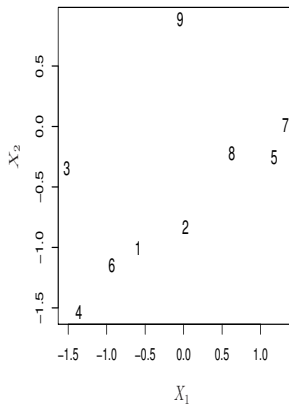
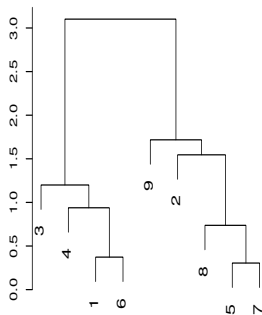
Hierarchisches Clustern

Beispiel: Die ersten drei Schritte beim agglomerativen Clustern mit neun Beobachtungen



Hierarchisches Clustern

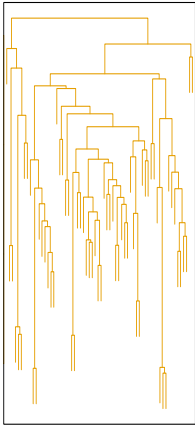
Die Prozedur lässt sich als folgenden Baum darstellen



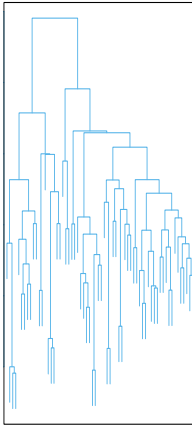
Hierarchisches Clustern

Größeres Beispiel mit Average Linkage (links), Complete Linkage (Mitte) und Single Linkage (rechts) mit jeweils euklidischer Distanz

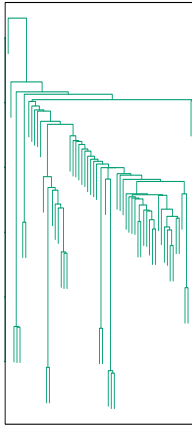
Average Linkage



Complete Linkage



Single Linkage



7. Modellwahl und Regularisierung

- Definition 0.5 könnte dazu verleiten, große Funktionenklassen $\mathcal{F}$, also potentiell komplexe Modelle, zu bevorzugen, um den Approximationsfehler klein zu halten
- Genau das ist allerdings keine gute Strategie
- I.A. ist der Testfehler für weniger komplexe Modelle kleiner als für komplexe Modelle. Das lässt i.d.R. auch theoretisch zeigen
- Statistik I: AIC und BIC als Komplexitätsmaße für ML-Modelle
- Problem: Wie bemisst man generell die Komplexität eines Modells?

Wiederholung: AIC und BIC

- Je höher die log-Likelihood eines Modells, also je niedriger die negative log-Likelihood, desto besser ist das Modell zu bewerten
- Da die Anzahl der verwendeten Prädiktoren allerdings nicht berücksichtigt wird, wäre das ein unfairer Vergleich zwischen Modellen mit verschieden großer Prädiktorzahl
- Zwei verbreitete Kriterien sind AIC (Akaike information criterion) und BIC (Bayes information criterion), nämlich

$$AIC(\hat{\theta}) = - \left(\sum_{i=1}^n \ln(p_{\hat{\theta}}(Y_i)) \right) + 2p,$$

$$BIC(\hat{\theta}) = - \left(\sum_{i=1}^n \ln(p_{\hat{\theta}}(Y_i)) \right) + \ln(n)p.$$

- Für $n \geq 8$ bestraft BIC die Anzahl der Prädiktoren bereits stärker als AIC, daher verwendet man AIC in der Regel dann, wenn man mehr Wert auf die Prädiktionsgüte, BIC dann, wenn man mehr Wert auf die Modellgüte legt

Klassische Modellwahl-Strategien für Regression

- **Best Subset Selection:** Berechne alle $2^p - 1$ möglichen Regressionsmodelle (bis auf dasjenige, das nur aus dem Intercept besteht) und wähle das beste Modell bzgl. AIC, BIC oder vergleichbaren Kriterien aus
- Offensichtlich ist dieses Vorgehen für großes p nicht sinnvoll
- Alternativen: **Forward / Backward Stepwise Selection (FSS/BSS)**
- FSS startet mit dem Intercept und fügt iterativ diejenige Variable hinzu, die am meisten mit der Response-Variable korreliert ist
- BSS startet mit dem vollen Modell und löscht iterativ diejenige Variable mit dem größten p -Wert
- In R: Paket `leaps`

Probleme der klassischen Modellwahl

- FSS und BSS liefern zwar nur p verschiedene Modelle, jedoch kann für großes p auch das noch zu aufwändig sein
- BSS kann nur für $n \geq p$ verwendet werden
- Sehr viel verlorene Rechenzeit, da viele Modelle berechnet werden, von denen am Ende nur eines gewählt wird, daher auch für komplexe nichtlineare Regressions- oder Rankingmodelle keine Option
- FSS und BSS können suboptimal sein, da sie durch das iterative Vorgehen das beste Modell gar nicht erwischen müssen

- VC-Dimension als generelles Komplexitätsmaß (nächste Folien)
- Regularisierte Regression durch explizite Strafterme (dieses Kapitel)
- Regularisierte Regression und Klassifikation (Kapitel Bäume, SVM, Boosting)
- Stabile Modellwahl (Kapitel Bagging)
- Regularisiertes Ranking (Kapitel Ranking)
- Hier nicht: Clustering und Modellwahl

Entropie von Funktionenklassen

Definition 6.1

Sei $\mathcal{F}$ eine Klasse von Indikator-Funktionen und sei

$$\mathbf{f}_n := (f(x_1), \dots, f(x_n))$$

ein n -dimensionaler binärer Vektor. Dann beschreibt $N_{\mathcal{F}}(\mathbf{x}_1, \dots, \mathbf{x}_n)$ die Anzahl aller verschiedenen Vektoren $\mathbf{f}_n$, die durch die Klasse $\mathcal{F}$ und die Daten $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ gebildet werden können.

Bemerkung 6.2

Es gilt $N_{\mathcal{F}}(\mathbf{x}_1, \dots, \mathbf{x}_n) \leq 2^n$.

Definition 6.3

Die **zufällige Entropie** der Klasse $\mathcal{F}$ auf den Daten $\mathbf{x}_1, \dots, \mathbf{x}_n$ ist definiert durch

$$H_{\mathcal{F}}(\mathbf{x}_1, \dots, \mathbf{x}_n) := \ln(N_{\mathcal{F}}(\mathbf{x}_1, \dots, \mathbf{x}_n)).$$

Die **Entropie** ist

$$H_{\mathcal{F}}(n) := \mathbb{E}[H_{\mathcal{F}}(X_1, \dots, X_n)] = \int H_{\mathcal{F}}(\mathbf{x}_1, \dots, \mathbf{x}_n) dF_X(\mathbf{x}_1, \dots, \mathbf{x}_n)$$

für $X_1, \dots, X_n \sim F$ i.i.d..

Satz 6.4

Gleichmäßige Konvergenz im Sinne von Def. 0.7 ist genau dann gegeben, wenn

$$\frac{H_{\mathcal{F}}(n)}{n} \xrightarrow{n \rightarrow \infty} 0.$$

Vapnik: The Nature of Statistical Learning Theory, 1995, Thm. 2.3

Vapnik, Statistical Learning Theory, 1998, Thm. 3.3

- Für Klassen reellwertiger Funktionen haben wir bereits Entropiemaße kennengelernt, nämlich die Covering- und die Packingzahl
- Die in Def. 0.13 eingeführte metrische Entropie und das Entropie-Integral kann man als Analoga zu der zufälligen Entropie und der Entropie in der vorigen Definition betrachten
- Analog gilt gleichmäßige Konvergenz genau dann, wenn das Entropie-Integral langsamer wächst als n (Thm. 2.3/3.4 ebenda)

Entropie von Funktionenklassen

- Im obigen Satz hat man gesehen, dass eine langsamer als n wachsende Entropie bereits notwendig und hinreichend für Konsistenz der Lernstrategie ist
- Allerdings macht dies noch keine Aussage über eine Konvergenzrate

Definition 6.5

Sei $\mathcal{F}$ eine Klasse von Indikator-Funktionen.

a) Die Größe

$$H_{\mathcal{F}}^{\text{ann}}(n) := \ln(\mathbb{E}[\mathcal{N}_{\mathcal{F}}(X_1, \dots, X_n)])$$

heißt **annealed entropy**.

b) Die **Wachstums-Funktion** ist definiert durch

$$G_{\mathcal{F}}(n) = \ln(\sup_{x_1, \dots, x_n} (\mathcal{N}_{\mathcal{F}}(x_1, \dots, x_n))) .$$

Bemerkung 6.6

Es gilt immer: $H_{\mathcal{F}}(n) \leq H_{\mathcal{F}}^{\text{ann}}(n) \leq G_{\mathcal{F}}(n)$.

- Man kann zeigen (Vapnik: Statistical Learning Theory), dass die Bedingung

$$\frac{H_{\mathcal{F}}^{\text{ann}}(n)}{n} \xrightarrow{n \rightarrow \infty} 0$$

hinreichend für exponentiell schnelle Konvergenz ist

- Ebenso:

$$\frac{G_{\mathcal{F}}(n)}{n} \xrightarrow{n \rightarrow \infty} 0$$

ist eine **hinreichende und notwendige Bedingung für verteilungsfreie Konsistenz der Lernprozedur und exponentiell schnelle Konvergenz**

- Analog für reellwertige Funktionen, indem $\mathcal{N}_{\mathcal{F}}$ durch die Überdeckungs-Zahl ersetzt wird. In dem Fall muss die Konvergenz des entsprechenden Quotienten gegen Null für alle $\varepsilon > 0$ erfüllt sein

Entropie von Funktionenklassen

- Problem: Zwar können Ungleichungen vom Markov-Typ gezeigt werden bzw. Ungleichungen, die das wahre Risiko nach oben begrenzen, jedoch hängen die rechten Seiten von $H_{\mathcal{F}}^{ann}$ oder der Wachstumsfunktion ab
- Beide sind nicht ohne Weiteres auf gegebenen Daten und gegebenen Funktionenklassen auszuwerten, daher bezeichnet Vapnik die Ungleichungen als „nonconstructive“

Satz 6.7

Sei $\mathcal{F}$ eine Klasse von Indikatorfunktionen. Dann gilt entweder $G_{\mathcal{F}}(n) = n \ln(2)$ für alle n , oder es ist

$$G_{\mathcal{F}}(n) \begin{cases} = n \ln(2), & n \leq h \\ \leq h \left(1 + \ln\left(\frac{n}{h}\right)\right), & n > h \end{cases} \quad (24)$$

*In dieser Situation heißt h die **Vapnik-Chervonenkis-Dimension (VC-Dimension)** von $\mathcal{F}$.*

Die VC-Dimension h im vorigen Satz wird durch folgende äquivalente Definitionen handhabbar:

Definition 6.8

Sei $\mathcal{F}$ eine Klasse von Indikatorfunktionen. Sei

$$S(\mathcal{F}, n) := \max_{x_1, \dots, x_n \in \mathcal{X}} (|N_{\mathcal{F}}(x_1, \dots, x_n)|)$$

der **Shatter-Koeffizient** von $\mathcal{F}$, also die Anzahl aller möglichen Klassifikationen, die $\mathcal{F}$ für eine Stichprobe $(x_1, \dots, x_n)$ vornehmen kann.

a) Die VC-Dimension ist $VC(\mathcal{F}) = \max\{n \mid S(\mathcal{F}, n) = 2^n\}$,

b) Äquivalent:

$$VC(\mathcal{F}) = \max\{n \mid \forall z \in \{0, 1\}^n \exists f_z \in \mathcal{F} : (f_z(x_1), \dots, f_z(x_n)) = z\}.$$

VC-Dimension: Beispiele

- Anschaulich: Wie viele Beobachtungen können durch die Funktionenklasse in allen möglichen Varianten in zwei Klassen eingeteilt werden? $\leadsto$ Die VC-Dimension ist genau die maximale Anzahl.
- Für allgemeine Funktionen: Gehe über zur Familie $\mathcal{G}$ bestehend aus den Subgraphen $\{(\mathbf{x}, t) \mid f(\mathbf{x}) \geq t\}$. Dadurch erhält man wieder eine Familie aus Indikatorfunktionen

Beispiel 6.9

Seien $\mathbf{x}_1, \dots, \mathbf{x}_n$ Punkte in allgemeiner Lage.

a) Die VC-Dimension der Klasse

$$\{f \mid f(\mathbf{x}) = 2I(\mathbf{x} \in]-\infty, b]) - 1, b \in \mathbb{R}\}$$

ist 1.

Beispiel 6.10

b) Die VC-Dimension der Klasse

$$\{f \mid f(x) = 2I(x \in]a, b]) - 1, a, b \in \mathbb{R}, a < b\}$$

ist 2.

c) Die VC-Dimension der Familie von Sphären im d -dimensionalen Raum mit dem Ursprung als Zentrum ist 1.

d) Die **VC-Dimension der Familie linearer trennender Hyperebenen**, also

$$\mathcal{F} := \{f(x) = \text{sign}(w^T x + b) \mid x \in \mathbb{R}^n, w, b \in \mathbb{R}\},$$

ist $(n + 1)$.

e) Die VC-Dimension der Familie

$$\{f \mid f(a, x) = (\sin(ax))_+, a \in \mathbb{R}\}$$

ist unendlich.

Definition 6.11

Eine Klasse mit endlicher VC-Dimension heißt **Vapnik-Chervonenkis-Klasse** oder kurz **VC-Klasse**.

Lemma 6.12

- a) Ist F eine VC-Klasse, so auch F^C .
- b) Ist F eine VC-Klasse, so auch $E \subset F$.
- c) Seien F und G VC-Klassen. Dann sind auch $G \cup F$, $G \cap F$, $G \times F$ VC-Klassen.

Giné, Nickl, 2015: Mathematical Foundations of Infinite-Dimensional Statistical Models, Prop. 3.6.7

Satz 6.13

Sei $\mathcal{F} := \{f_\theta \mid \theta \in \Theta\}$ eine Klasse von Indikatorfunktionen mit endlicher VC-Dimension h . Dann gilt

$$\begin{aligned} P(\sup_{\theta \in \Theta} (|L(\theta) - L_n(\theta)|) > \varepsilon) &< \\ 4 \exp \left(n \left(\frac{h(1 + \ln(2n/h))}{n} - \left(\varepsilon - \frac{1}{n} \right)^2 \right) \right) \\ &= \text{const} \left(\frac{2n}{h} \right)^h \exp \left(- \left(\varepsilon - \frac{1}{n} \right)^2 \right) \end{aligned}$$

für das Risiko $L(\theta) := L(f_\theta)$ wie in Def. 0.1, L_n analog. Man kann damit zeigen, dass mit Wahrscheinlichkeit $1 - \eta$ die Ungleichung

$$L(\hat{\theta}_n) < L_n(\hat{\theta}_n) + \frac{c_n}{2} \left(1 + \sqrt{1 + \frac{4L_n(\hat{\theta}_n)}{c_n}} \right)$$

gilt, wobei

$$c_n = 4 \frac{h(\ln(2n/h) + 1) - \ln(\eta/4)}{n}.$$

Satz 6.14

Für binäre Klassifikation (o.E. sind die Klassen hier 0 und 1) und Verlustfunktion $L_{0/1}(\mathbf{u}, \mathbf{u}') := I(\mathbf{u} \neq \mathbf{u}')$ und eine Klasse $\mathcal{F}$ von Funktionen $f : \mathbb{R}^p \rightarrow \{0, 1\}$ gilt die Abschätzung

$$P(\sup_{f \in \mathcal{F}} (|L_n(f) - L(f)|) > \varepsilon) \leq 8\mathcal{S}(\mathcal{F}, n)e^{-n\varepsilon^2/32}$$

sowie

$$E[\sup_{f \in \mathcal{F}} (|L_n(f) - L(f)|)] \leq 2\sqrt{\frac{\ln(\mathcal{S}(\mathcal{F}, n)) - \ln(2)}{n}}.$$

Devroye, Györfli, Lugosi, 1996: A Probabilistic Theory of Pattern Recognition, Thm.12.5

- Theoretische Resultate zeigen, dass (zu) komplexe Funktionenklassen zu einer schlechten Generalisierungs-Fähigkeit (d.h., der Testfehler ist groß) der Lernstrategie führt
- Sinnigerweise müssen die betrachteten Funktionenklassen VC-Klassen sein
- Da die VC-Dimension nirgends die Verteilung der Beobachtungen berücksichtigt, sind also alle VC-Resultate verteilungsfrei
- Im Folgenden wollen wir uns der Fragestellung widmen, wie man effizient und automatisiert geeignete Funktionen erhält (also ohne die Funktionenklasse vorher konkret festzulegen)

Strukturierte Risikominimierung

- Bisher: ERM, d.h., das empirische Risiko soll minimiert werden, ungeachtet der Komplexität des Modells
- Idee von Vapnik: Betrachte eine genestete Sequenz von Funktionenklassen $\mathcal{F}_1 \subset \mathcal{F}_2 \subset \dots$ mit jeweils endlichen VC-Dimensionen $h_k, k \geq 1$. Wähle das Element der genesteten Sequenz, welches zur kleinsten Grenze für den Fehler führt. Das Vorgehen nennt sich **strukturierte Risikominimierung (SRM)**
- Die Subset-Selection-Strategien können als Realisierung des SRM-Prinzips angesehen werden
- Im Folgenden beschäftigen wir uns mit der Frage, wie man Modelle mit Hilfe des SRM-Prinzips in der Praxis bekommt, ohne alle möglichen Modelle ausprobieren zu müssen

- Betrachte das Optimierungsproblem

$$\min_{\theta}(L_n(\theta)) \quad \text{s.t.} \quad J(\theta) \leq c_{\lambda}$$

für eine von λ abhängige Konstante $c_{\lambda} > 0$ und $J : \Theta \rightarrow \mathbb{R}_{\geq 0}$

- In **Lagrange-Form** lässt sich das Problem äquivalent umformulieren in

$$\min_{\theta}(\mathcal{L}(\theta, \lambda) := L_n(\theta) + \lambda J(\theta)).$$

- L_n ist wieder unser gewöhnliches empirisches Risiko, während λJ einen Strafterm darstellt, der die Komplexität des Modelles bestraft. Daher ist i.A. $J(\theta) > 0$ für alle $\theta \neq \theta_p$.
- Geeignete Verlustfunktionen sind bereits von den klassischen Algorithmen bekannt. Für regularisierte Verfahren bedarf es einer geeigneten Wahl von J und λ

Wiederholung: Karush-Kuhn-Tucker-Bedingungen

- Betrachte sehr allgemeine Optimierungsprobleme der Form

$$\min(f(x)) \quad \text{s.t.} \quad g_i(x) \leq 0, h_j(x) = 0 \quad \forall i, j,$$

für $i = 1, \dots, u, j = 1, \dots, g$ und Funktionen $f, g_i, h_j : \mathbb{R}^p \rightarrow \mathbb{R}$

- In Lagrange-Form ist dies äquivalent zur Minimierung von

$$L(x, \lambda, \mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x)$$

- $(\hat{x}, \hat{\lambda}, \hat{\mu})$ heißt KKT-Punkt des Problems, falls die **KKT-Bedingungen** erfüllt sind, d.h.,

$$\begin{aligned} \nabla_x L(\hat{x}, \hat{\lambda}, \hat{\mu}) &= 0, \quad g_i(\hat{x}) \leq 0 \quad \forall i, \quad h_j(\hat{x}) = 0 \quad \forall j \\ \hat{\lambda}_i &\geq 0 \quad \forall i, \quad \hat{\lambda}_i g_i(\hat{x}) = 0 \quad \forall i \end{aligned}$$

- Die letzte Bedingung wird auch als komplementäre Schlupfbedingung bezeichnet

Wiederholung: Compressed Sensing

- Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $m < n$. Ziel ist das Auffinden einer sparsen Lösung für das unterbestimmte Problem $\mathbf{Ax} = \mathbf{b}$ für $\mathbf{A}, \mathbf{b}$ bekannt
- Die Probleme (P_0) und (P_1) sind gegeben durch
$$\min_x (\|\mathbf{x}\|_k) \quad \text{s.t.} \quad \mathbf{Ax} = \mathbf{b}, \quad k = 0, 1$$

- Letzteres wird auch als **Basis Pursuit** bezeichnet
- Mit Techniken der Numerik lässt sich zeigen, wann im Falle der Existenz einer Lösung zu $\mathbf{Ax} = \mathbf{b}$ diese sowohl die eindeutige Lösung zu (P_0) als auch die eindeutige Lösung zu (P_1) ist, d.h., wann man äquivalent (P_1) anstelle von (P_0) lösen kann (siehe Elad: Sparse and Redundant Representations, 2010, Kap. 4)
- Lässt man eine Fehlertoleranz zu, so genügt es, die approximativen Probleme (P_k^ϵ) mit

$$\min(\|\mathbf{x}\|_k) \quad \text{s.t.} \quad \|\mathbf{b} - \mathbf{Ax}\|_2 \leq \epsilon$$

zu lösen, $k = 0, 1$

- (P_1^ϵ) lässt sich äquivalent schreiben als das **BPDN-Problem (Basis Pursuit Denoising Problem)**

$$\min_x (\lambda \|\mathbf{x}\|_1 + \|\mathbf{b} - \mathbf{Ax}\|_2^2)$$

Definition 6.15

a) Ein Schätzer $\hat{\beta}_n$ für β heißt **konsistent**, falls $\hat{\beta}_n \xrightarrow{P} \beta$.

b) Ein Schätzer $\hat{\beta}_n$ für β heißt **$\sqrt{n}$ -konsistent**, falls $\sqrt{n}(\hat{\beta}_n - \beta) = \mathcal{O}(1)$.

Definition 6.16

a) Sei β^0 der wahre Parameter in dem Modell $Y = X\beta$. Dann ist $S^0 := \{j \mid \beta_j^0 \neq 0\}$ und $s^0 := |S^0|$.

b) Eine Modellwahl-Prozedur hat die **Screening-Eigenschaft**, falls

$$P(\hat{S} \supset S^0) \rightarrow 1$$

für $\hat{S} := \{j \mid \hat{\beta}_n \neq 0\}$.

c) Eine Modellwahl-Prozedur ist **variablenselektions-konsistent**, falls

$$P(\hat{S} = S^0) \rightarrow 1$$

- Die Komplexität eines parametrischen Modells ist eigentlich gegeben durch $||\beta||_0 := \#\{j \mid \beta_j \neq 0\}$
- Diese Norm ist allerdings nicht stetig, sodass eine Optimierung einer Funktion mit Strafterm $\lambda ||\beta||_0$ numerisch nicht handhabbar wäre
- Idee: Wähle $J(\beta) := ||\beta||_2^2$ als Strafterm. Dieser ist konvex und immer noch glatt. Außerdem ist das Argumentminimum von $||\beta||_2^2$ das gleiche wie für $||\beta||_0$, also 0_p . Die l_2 -Norm kann somit als **Surrogate** für die l_0 -Norm betrachtet werden
- Dies führt auf die **Ridge-Regression**

$$\beta^{ridge} = \underset{\beta \in \mathbb{R}^p}{\operatorname{argmin}} \left(\frac{1}{2} \sum_i (Y_i - X_i \beta)^2 + \lambda \sum_j \beta_j^2 = \frac{1}{2} ||Y - X\beta||_2^2 + \lambda ||\beta||_2^2 \right)$$

- Die Lösung ist gegeben durch

$$\beta^{ridge} = (X^T X + \lambda I_p)^{-1} X^T Y.$$

- Nachteil: Die Koeffizienten werden bei der Ridge-Regression zwar geschrumpft, aber in der Regel nicht auf Null gesetzt
- Betrachte die l_1 -Norm:

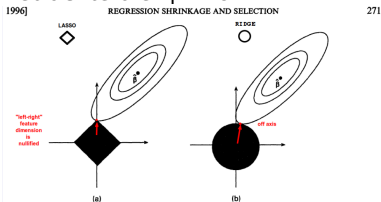


Fig. 2. Estimation picture for (a) the lasso and (b) ridge regression

Tibshirani: Regression Shrinkage and Selection via the Lasso, Abb. 2

- Der l_1 -Strafterm führt auf das **Lasso** (Least Absolute Shrinkage and Selection Operator)

$$\beta^{Lasso} = \underset{\beta}{\operatorname{argmin}} \left(\frac{1}{2} \|Y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right)$$

- Das Lasso realisiert sparsame Modellwahl. Genauer ist es äquivalent zum BPDN-Problem
- Noch besser für sparsame Modellwahl (da noch näher an der l_0 -Norm) wären Strafterme der Form $\|\beta\|_q$ für $0 < q < 1$. Diese wären aber nicht mehr konvex, sodass selbst die Existenz einer Lösung nicht gegeben sein muss
- Allgemeiner heißen Schätzer der Form

$$\beta^{Bridge} = \underset{\beta}{\operatorname{argmin}} (\|Y - X\beta\|_2^2 + \lambda \|\beta\|_q^q)$$

für $q > 0$ **Bridge-Schätzer**

Im Fall p konstant kann man zeigen, dass für $\lambda = \lambda_n = \mathcal{O}(n)$ (d.h., $\lambda_n/n \rightarrow \lambda_0$) der Bridge-Schätzer $\mathcal{O}(n^0)$ konsistent ist, für eine Rate brauchen wir folgenden Satz

Satz 6.17

Sei

$$\Sigma_n := \frac{1}{n} \sum X_i^T X_i \rightarrow \Sigma, \quad \frac{1}{n} \max_i (X_i X_i^T) \rightarrow 0$$

für eine positiv semidefinite Matrix Σ .

a) Sei $\lambda = \lambda_n = \mathcal{O}(\sqrt{n})$. Dann sind alle Bridge-Schätzer für $q \geq 1$ (insbesondere also das Lasso) $\sqrt{n}$ -konsistent.

b) Sei $\lambda = \lambda_n = \mathcal{O}(n^{q/2})$. Dann sind alle Bridge-Schätzer für $q < 1$ $\sqrt{n}$ -konsistent.

Knight, Fu: Asymptotic of Lasso-type Estimators, Thm. 2.2+Thm. 2.3

Achtung: Klingt die Sequenz $(\lambda_n)_n$ zu langsam ab (konkret: $\lambda_n/n \rightarrow 0$, aber $\lambda_n/\sqrt{n} \rightarrow \infty$), so ist das Lasso i.A. nicht variabelselektions-konsistent

Satz 6.18

Seien die Spalten der Regressormatrix o.E. so umgeordnet, dass

$$\Sigma_n = \begin{pmatrix} (\Sigma_n)_{1,1} & (\Sigma_n)_{1,2} \\ (\Sigma_n)_{2,1} & (\Sigma_n)_{2,2} \end{pmatrix}, \quad (\Sigma_n)_{1,1} \in \mathbb{R}^{s_0 \times s_0}.$$

Eine notwendige Bedingung für Variablenselektions-Konsistenz des Lasso ist die **schwache Irrepräsentabilitäts-Bedingung**

$$\|(\Sigma_n)_{2,1}(\Sigma_n)_{1,1}^{-1} \text{sign}(\beta_1^0, \dots, \beta_{s_0}^0)\|_\infty \leq 1.$$

Lasso und Variablenselektions-Konsistenz

- Man kann zeigen (Bühlmann, van de Geer, 2011: Statistics for High-Dimensional Data, Kap. 7.5), dass die **Irrepräsentabilitäts-Bedingung** (d.h., die schwache IB gilt mit < 1) ein hinreichendes Kriterium an die Regressormatrix für Variablenselektions-Konsistenz ist, sofern sogenannte „Beta-min-Bedingungen“ erfüllt sind (d.h., die wahren Nichtnull-Koeffizienten sind „groß genug“)
- Frage: Gibt es einen besseren Algorithmus als das Lasso, welcher sowohl schätz- als auch variablenselektions-konsistent ist?
- Idee: Gewichte die Summanden des Strafterms geeignet. Das führt auf gewichtete Lasso der Form

$$\beta^{WLasso} := \operatorname{argmin}_{\beta} \left(\frac{1}{2} \|Y - X\beta\|_2^2 + \lambda \sum_j \frac{1}{w_j} |\beta_j| \right)$$

mit Gewichten $w_j \geq 0$

- Wählt man die Gewichte adaptiv, spricht man vom **adaptiven Lasso**. Dazu setzt man z.B. $w_j := |\hat{\beta}_j^{LS}|$ für den KQ-Schätzer $\hat{\beta}^{LS}$ oder $w_j := |\hat{\beta}_j^{Lasso}|$. Letztere Wahl hat den Vorteil, dass diejenigen Variablen, die das Lasso nicht wählt, auch vom Adaptiven Lasso nicht gewählt werden

Satz 6.19

Seien $w_j = |\hat{\beta}_j|^q$ mit einem $\sqrt{n}$ -konsistenten Schätzer $\hat{\beta}_n$. Wenn $\lambda = \lambda_n$ mit $\lambda_n/\sqrt{n} \rightarrow 0$, aber $\lambda_n n^{(q-1)/2} \rightarrow \infty$, dann ist das Adaptive Lasso variabelenselektions-konsistent und asymptotisch normal.

Zou: The Adaptive Lasso and its Oracle Properties

- Die Irrepräsentabilitäts-Bedingung lässt sich zu einer gewichteten IB erweitern
- Sofern man einen Algorithmus hätte, der das Lasso berechnet, könnte man diesen mit entsprechenden Gewichten direkt verwenden, um das Adaptive Lasso zu berechnen
- Das Problem bei der numerischen Optimierung der Lasso-Zielfunktion ist aber die **Nicht-Differenzierbarkeit des Lasso-Strafterms!**

Algorithmen für das Lasso

- Nicht mehr aktuell, da nicht effizient genug für hochdimensionale Daten: LARS (Efron: Least Angle Regression, 2004)
- Idee hinter LAR: Starte mit dem leeren Modell und finde die Variable j_1 , die am meisten mit $\mathbf{r} := \mathbf{Y} - \bar{\mathbf{Y}}$ korreliert ist. Berechne den KQ-Schätzer basierend auf dieser einen Variablen
- Lasse den aktuellen Koeffizienten von $\hat{\beta}_{j_1}$ gegen den KQ-Koeffizienten gehen, aber nur so lange, bis eine Variable j_2 stärker mit dem aktuellen Residuum korreliert ist
- Berechne den **gemeinsamen KQ-Schätzer** bzgl. beider Variablen und bewege beide aktuellen Koeffizienten gegen den KQ-Schätzer, bis wieder eine andere Variable stärker mit dem aktuellen Residuum korreliert ist als die beiden usw.
- Nach höchstens $\min(p, n - 1)$ Schritten erreichen wir so die KQ-Lösung des gesamten Modells
- Lasso-Modifikation (LAR $\rightarrow$ LARS): Sobald ein Koeffizient beim Aufdatieren wieder die Null erreicht, bleibt der Koeffizient Null
- In R im Paket `lars` realisiert

Lemma 6.20

Falls X orthonormal ist, d.h., $X^T X = I_p$, oder $p = 1$, hat das Lasso die explizite Lösung

$$\hat{\beta}_j^{\text{Lasso}} = \text{sign}(\hat{\beta}_j^{\text{LS}})(|\hat{\beta}_j^{\text{LS}}| - \lambda)_+.$$

Allgemein kann man nur folgendes Resultat formulieren:

Lemma 6.21

Es sei $G(\beta) := -2X^T(Y - X\beta)/n = \partial_\beta \|Y - X\beta\|_2^2/n$. Dann ist $\hat{\beta}$ eine Lösung des Lasso-Problems genau dann, wenn

$$G(\beta) = \begin{cases} \hat{\beta}_j = -\text{sign}(\hat{\beta}_j)\lambda, & \hat{\beta}_j \neq 0 \\ \hat{\beta}_j = 0 & |\hat{\beta}_j| \leq \lambda \end{cases}$$

Algorithmen für das Lasso

- Im R-Paket `glmnet` implementiert: **Coordinate-Descent (CD; auch: Koordinaten-Schießverfahren)**
- Gehe iterativ zyklisch durch die Spaltenindices $j = 1, \dots, p$, d.h., auf eine Iteration mit $j = p$ folgt eine Iteration mit $j = 1$ und betrachte alle Koeffizienten $\hat{\beta}_k, k \neq j$ als konstant
- Berechne daher jeweils das partielle Residuum

$$r^{(j)} := Y - \sum_{k \neq j} X_{:,k} \hat{\beta}_k$$

und löse

$$\hat{\beta}_j^{Lasso} := \operatorname{argmin}_{\beta_j \in \mathbb{R}} (||r^{(j)} - X_{:,j} \beta_j||_2^2 + \lambda |\beta_j|)$$

wie in Lemma 6.20

- Der Algorithmus kann direkt für das Adaptive Lasso und das Elastic Net (nächste Folie) angewendet werden
- λ wird per CV gewählt

- Als Kompromiss zwischen Lasso- und Ridge-Regression wurde das **Elastic Net** eingeführt (Zou, Hastie 2005), konkret

$$\hat{\beta}^{EN} := \mathop{\text{argmin}}_{\beta} (\|Y - X\beta\|_2^2 + \alpha\|\beta\|_1 + (1 - \alpha)\|\beta\|_2^2)$$

- Die Lösung des EN-Problems kann als Lösung eines Lasso-Problems identifiziert werden. Für X orthonormal gibt es eine analoge explizite Lösung
- Für Variablenselektions-Konsistenz gibt es ein Analogon zur Irrepräsentabilitäts-Bedingung (Jia, Yu, 2010)
- In der Praxis ist es oft so, dass die EN-Koeffizienten von gruppierten Variablen (nächste Folie) das gleiche Vorzeichen haben und auch betragsmäßig ähnlich sind
- In R: Eingabe von `alpha` in `glmnet` (Default ist 1 (Lasso))

Gruppierte Variablen

- Bei kategoriellen Variablen erzeugen wir bei der Codierung zusätzliche Spalten. Bei sparsen Modellen ist es natürlich sinnvoll, dass entweder alle Koeffizienten, die zu diesen Spalten korrespondieren, Null oder von Null verschieden sind (sonst würde man nur bestimmte Ausprägungen der Variablen in das Modell aufnehmen). Analog z.B. für polynomiale Regression
- In der Praxis kommen gruppierte Variablen darüber hinaus z.B. bei genetischen Anwendungen vor
- Das **Grouped Lasso** (Yuan, Lin, 2007) soll möglichst wenige Gruppen auswählen und löst daher

$$\hat{\beta}^{GrLasso} = \underset{\beta}{\operatorname{argmin}} \left(\frac{1}{2} \left\| Y - \sum_{l=1}^m X^{(l)} \beta^{(l)} \right\|_2^2 + \lambda \sum_l \sqrt{p_l} \|\beta^{(l)}\|_2 \right)$$

mit den zur l -ten Gruppe gehörenden p_l Variablen in Form des Vektors $\beta^{(l)}$ und den zugehörigen Spalten $X^{(l)}$ von X

- Lässt sich ebenfalls durch den CD-Algorithmus lösen, indem man zyklisch durch die Blöcke geht, daher nennt sich der Algorithmus hier **Blockwise Coordinate Descent** (BlockCD)

- Nachteil des Grouped Lasso: Es werden zwar nur wenige Gruppen von Variablen ausgewählt, aber dann jede Variable aus diesen Gruppen
- Um auch noch innerhalb der Gruppen sparsame Modellwahl zu realisieren, löst man das **Sparse-Group Lasso** (Simon et. al. 2013)
- Ähnlich wie beim EN verwendet man einen kombinierten Strafterm aus der l_1 – und der l_2 –Norm, konkret

$$(1 - \alpha)\lambda \sum_I \sqrt{p_I} \|\beta^{(I)}\|_2 + \alpha\lambda \|\beta\|_1.$$

- Das Sparse-Group Lasso kann auch durch einen BlockCD-Algorithmus gelöst werden
- In R: Paket SGL

Weitere Anwendungen von Lasso-Straftermen

- Erinnerung: Bei der ML-Schätzung wurde die log-Likelihood maximiert
- Wenn wir $\rho_{\beta}(\mathbf{X}_i, \mathbf{Y}_i)$ als die negative logarithmierte Dichte an der Stelle $(\mathbf{X}_i, \mathbf{Y}_i)$ schreiben, so können wir einen sparsen ML-Schätzer durch

$$\hat{\beta}^{ML-Lasso} = \underset{\beta}{\operatorname{argmin}} \left(\frac{1}{n} \sum_i \rho_{\beta}(\mathbf{X}_i, \mathbf{Y}_i) + \lambda \|\beta\|_1 \right)$$

erhalten

- Insbesondere interessant für GLMs, zumal Dichten aus Exponentialfamilien i.A. log-konkav bzgl. des Parameters sind
- Lasso-Strafterme finden darüber hinaus z.B. auch Verwendung bei der sparsen Schätzung einer Kovarianz- oder Präzisionsmatrix oder bei Clustering-Modellen mit Variablenselektion (hier nicht)

- Wie bereits erwähnt, würden Strafterme der Form $||\beta||_q^q$ für $0 < q < 1$ noch sparsamere Modell liefern
- In Kombination mit einer konvexen Verlustfunktion ist aber i.A. nicht einmal die Existenz eines Optimums geklärt
- Populäre nicht-konvexe Strafterme sind darüber hinaus SCAD (Fan, Li, 2001) und MCP (Zhang, 2010)
- Zu nicht-konvexen Straftermen und nicht-konvexen Verlustfunktionen gibt es bereits zahlreiche Arbeiten (hier nicht)
- **Multistep Adaptive Lasso (MSALasso)**: Wähle $\lambda_1, \dots, \lambda_K$ adaptiv per CV und verwende im k -ten Schritt den Strafterm $\lambda^{(k)} \sum_j (\mathbf{w}_j)^{-1} |\beta_j|$ mit $\mathbf{w}_j := |\hat{\beta}^{(k-1)} \lambda^{(k-1)}|$. Asymptotisch (für $K \rightarrow \infty$) realisiert man dadurch die Minimierung von

$$||Y - X\beta||_2^2 + \lambda \sum_j \ln(|\beta_j|)$$

- Der Strafparameter λ wird per CV gewählt. Um die Information aus den berechneten Modellen für verschiedene Werte von λ zu nutzen, werden in **R Regularisierungspfade** ausgegeben, realisiert durch einen gemeinsamen Plot aller Koeffizienten in Abhängigkeit von λ oder $\|\hat{\beta}\|_1$
- Sei λ_{max} das kleinste $\lambda > 0$, sodass $\hat{\beta}(\lambda_{max}) = \mathbf{0}_p$
- Problem bei der Konstruktion: Wir müssten eigentlich die Koeffizienten für alle $\lambda \in]0, \lambda_{max}]$ berechnen
- Günstig ist es, wenn die Koeffizientenpfade **stückweise linear** sind, d.h., die Koeffizienten aufgefasst als $\hat{\beta} = \hat{\beta}(\lambda)$, sind stückweise lineare Funktionen von λ

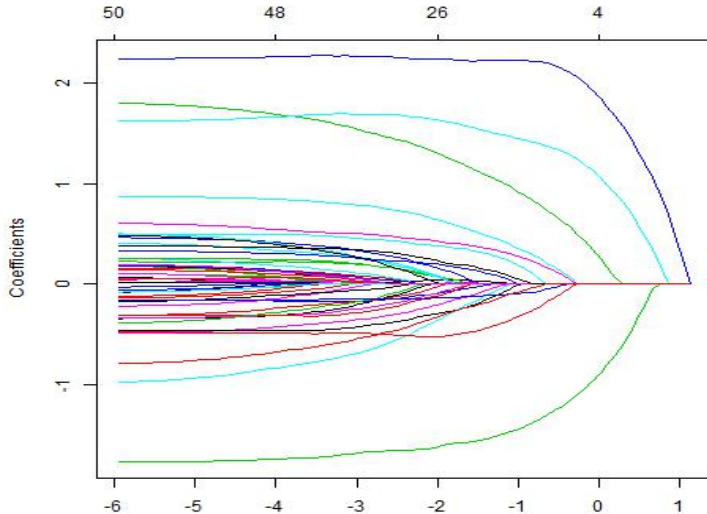
Regularisierungspfade

- (Efron et. al. 2004) haben bereits gezeigt, dass Lasso-Regularisierungspfade stückweise linear sind und diese durch den LARS-Algorithmus berechnet
- Allgemeiner (Details: Rosset, Zhu, 2007): Die Regularisierungspfade sind stückweise linear, wenn die Verlustfunktion stückweise quadratisch ist und der Strafterm stückweise linear. Dieser wird von ihrem CD-Algorithmus berechnet
- (Park und Hastie, 2007): GLM-Regularisierungspfade sind i.A. nicht linear. Die Pfade konnten mit einem Prädiktor-Korrektor-Verfahren berechnet werden, inzwischen funktioniert auch dies mit dem CD-Algorithmus
- In `glmnet` sowohl für GLMs als auch für EN-Schätzer: Implementiert als `plot.glmnet`, d.h., man wendet die Funktion `plot` auf ein Objekt der Klasse `glmnet` an

Regularisierungspfade

- Dann können die Regularisierungspfade konstruiert werden, indem die Koeffizienten für ein Gitter von Strafparametern bestimmt werden. I.A. betrachtet man eine streng monoton fallende Sequenz von Strafparametern, oft äquidistant auf der log-Skala. Die vorige Lösung kann oft als „warm start“ verwendet werden
- Regularisierungspfade sind insbesondere interessant für die Modellwahl: Aus den Regularisierungspfaden ist ersichtlich, ab welchem λ Variablen aus dem Modell entfernt werden. Für eine CV braucht man also nur diejenigen Lösungen für eben diese Strafparameter betrachten
- Die Regularisierungspfade sind nicht nur ein sehr hilfreiches graphisches Tool zur Visualisierung, sondern können auch zur Wahl des optimalen Strafparameters herangezogen werden
- Beispiel: Füge dem Datensatz p_1 Störvariablen hinzu. Wähle dann als obere Schranke für λ denjenigen Wert, ab dem alle künstlich hinzugefügten Störvariablen einen Koeffizienten von Null bekommen

Regularisierungspfade



Aussagen für $p \gg n$

- Bislang haben wir gesehen, dass klassische Verfahren wie Subset Selection bei $p > n$ aufgrund des computationellen Aufwands keine Option sind
- Kleinste-Quadrate-Lösungen sind nicht mehr eindeutig für $p > n$. Um überhaupt einen Koeffizienten schätzen zu können, muss der p -dimensionale Parameter zudem linear in einen niedriger-dimensionalen Raum überführt werden
- Man kann obendrein zeigen: Das Lasso kann auf einem Datensatz mit $X \in \mathbb{R}^{p \times n}$ maximal n Nichtnull-Koeffizienten produzieren
- Das alles gibt aber noch keine Aussage darüber, wie groß p im Vergleich zu n sein darf oder genauer, **wie stark p , aufgefasst als $p(n)$, wachsen darf**
- Interessant ist die Frage, für welche Verhältnisse p/n noch eine konsistente Schätzung möglich ist

Aussagen für $p \gg n$

Definition 6.22

Seien $X \in \mathbb{R}^{n \times p}$, $Y \in \mathbb{R}^n$ deterministisch und $\varepsilon \sim \mathcal{N}_n(\mathbf{0}, \sigma I_n)$. Legen wir das lineare Modell zugrunde, also $Y = X\beta + \varepsilon$, so definieren wir im Folgenden:

a) $\hat{\beta}$ ist ein **konsistenter Schätzer** für β , wenn

$$\frac{\|X(\hat{\beta} - \beta)\|_2^2}{n} \xrightarrow{P} 0.$$

b) Den **Prädiktionsfehler**

$$\frac{\|X(\hat{\beta} - \beta)\|_2^2}{\sigma^2}.$$

c) Für den quadratischen Verlust nennen wir

$$2\varepsilon^T \frac{X(\hat{\beta} - \beta)}{n}$$

den **Empirischen-Prozess-Teil**.

Aussagen für $p \gg n$

Lemma 6.23

Es ist

$$2|\varepsilon^T X(\hat{\beta} - \beta)| \leq (\max_j (2|\varepsilon^T X_{\cdot j}|)) \|\hat{\beta} - \beta\|_1.$$

Bühlmann, van de Geer: Statistics for High-Dimensional Data, 2011, Ungleichung (6.2)

Lemma 6.24

Sei $\hat{\Sigma} := X^T X / n$ und $\hat{\sigma}_j^2 := (\hat{\Sigma})_{j,j}$. Falls $\hat{\sigma}_j^2 = 1$ für alle j , so ist für alle $t > 0$

$$P\left(\max_j \left(2 \frac{|\varepsilon^T X_{\cdot j}|}{n}\right) > \lambda_0\right) \leq 2e^{-t^2/2}$$

für

$$\lambda_0 = 2\sigma \sqrt{\frac{t^2 + 2\ln(p)}{n}}.$$

Bühlmann, van de Geer: Statistics for High-Dimensional Data, 2011, Lemma 6.2

Aussagen für $p \gg n$

Corollary 6.1

Seien $\hat{\sigma}_j^2 = 1$ für alle j und sei für $t > 0$

$$\hat{\lambda} := 4\hat{\sigma} \sqrt{\frac{t^2 + 2\ln(p)}{n}}$$

für einen Schätzer $\hat{\sigma}$ für σ . Dann ist

$$P\left(2 \frac{\|X(\hat{\beta} - \beta)\|_2^2}{n} > 3\hat{\lambda} \|\beta\|_1\right) \leq 2e^{-t^2/2} + P(\hat{\sigma} \leq \sigma).$$

Bühlmann, van de Geer: Statistics for High-Dimensional Data, 2011, Cor. 6.1

- Das Kleinste-Quadrate-Schätzer ist konsistent im Sinne von 6.22, falls $\frac{p \ln(p)}{n} \rightarrow 0$, d.h., p muss langsamer wachsen als n .
- Zum Vergleich: Das Lasso ist konsistent für $\ln(p)/n \rightarrow 0$, d.h., für **eine exponentiell wachsende Anzahl von Variablen!**

Definition 6.25

Für $\mathbf{s}^0$ gilt die **Kompatibilitätsbedingung**, falls $\varphi_0 > 0$ existiert, sodass für alle β mit $\|\beta_{(\mathbf{s}^0)^c}\|_1 \leq 3\|\beta_{\mathbf{s}^0}\|_1$ gilt, dass

$$\|\beta_{\mathbf{s}^0}\|_1^2 \leq \beta^T \hat{\Sigma} \beta \frac{\mathbf{s}^0}{\varphi_0^2}.$$

Bühlmann, van de Geer: Statistics for High-Dimensional Data, 2011, Ungleichung 6.4

- Norm-Ungleichungen liefern sofort $\|\beta_{\mathbf{s}^0}\|_1^2 \leq \mathbf{s}^0 \|\beta_{\mathbf{s}^0}\|_2^2$
- Dies wird in Abschnitt 6.2 ebenda mit einer Bedingung an den kleinsten Eigenwert von $\hat{\Sigma}$ identifiziert. Beides lässt sich in der Praxis nicht prüfen, wohl aber (sofern $\mathbf{s}^0$ bekannt wäre) durch Prüfung der Ungleichung aller $\mathbf{s}^0$ -elementigen Teilmengen von $\{1, \dots, p\}$. Dies nennt sich **restringierte Eigenwert-Bedingung**
- Die θ -**uniforme Irrepräsentabilitätsbedingung**

$$\max_{\|\tau_S\|_\infty \leq 1} (|(\Sigma_n)_{2,1}(\Sigma_n)_{1,1}^{-1} \tau_S|_\infty) \leq \theta$$

für $0 < \theta < 1$ impliziert die Kompatibilitätsbedingung (Abschnitt 7.5.4, ebenda)

Aussagen für $p \gg n$

Satz 6.26 (Orakelungleichung für das Lasso)

Sei $\hat{\sigma}_j^2 = 1$ für alle j und es gelte die Kompatibilitätsbedingung für S^0 . Sei $\hat{\sigma}^2$ ein Schätzer für σ^2 und sei

$$\hat{\lambda} := 4\hat{\sigma} \sqrt{\frac{t^2 + \ln(p)}{n}}.$$

Dann gilt

$$P\left(\frac{\|X(\hat{\beta} - \beta)\|_2^2}{n} + \hat{\lambda} \|\hat{\beta} - \beta\|_1 > 4\lambda^2 \frac{s^0}{\varphi_0^2}\right) \leq 2e^{-t^2/2} + P(\hat{\sigma} \leq \sigma).$$

Bühlmann, van de Geer: Statistics for High-Dimensional Data, 2011, Cor. 6.2

■ Über die **Signal-to-Noise-Ratio (SNR)**

$$SNR := \frac{\|X\beta\|_2}{\sqrt{n}\sigma}$$

lässt sich der Term $P(\hat{\sigma} \leq \sigma)$ durch $2e^{-t^2/2}$ abschätzen

- Es gibt eine ähnliche Orakelungleichung für den Fall, dass gar kein lineares Modell zugrunde liegt, sondern wir das wahre Modell nur linear approximieren (Abschnitt 6.2.3, ebenda)

- Es gibt ähnliche Konzepte für allgemeine Schätzer der Form

$$\hat{\beta} = \underset{\mathcal{F}}{\operatorname{argmin}} \left(\frac{1}{n} \sum_i L(Y_i, \hat{f}_{\beta}(X_i)) + \lambda \|\beta\|_1 \right)$$

für konvexe Verlustfunktionen L und einer parametrischen Klasse $\mathcal{F} = \{f_{\beta} \mid \beta \in \Theta\}$, sodass jeweils $\beta \mapsto f_{\beta}$ linear ist (und noch allgemeinere Fälle, siehe Kap. 6, ebenda)

- Es gibt ähnliche Konzepte für das adaptive Lasso (Kap. 6, ebenda)
- Sei beim gruppierten Lasso wieder m die Anzahl der Gruppen und $p_l = \bar{p}$ die Anzahl der Variablen in Gruppe l .
- Für den Schätzfehler lässt sich zeigen, dass

$$\sum_l \|\hat{\beta}^{(l)} - \beta^{(l)}\|_2 = \mathcal{O} \left(\frac{s^0 \sqrt{\max(\ln(m), \bar{p})}}{\sqrt{n} \varphi^2} \right),$$

mit der Kompatibilitäts-Konstanten φ (Kap. 4 und 8, ebenda)

- Klassische Lernverfahren sind im Fall $p > n$ nicht angemessen, da sie zu komplex sind
- Die Komplexität eines Modells lässt sich sehr allgemein durch die VC-Dimension quantifizieren. Modelle sollten immer aus VC-Klassen stammen. Das führt vom ERM- zum SRM-Prinzip
- I_1 –Regularisierung der Zielfunktion realisiert die strukturelle Risikominimierung in der Praxis
- Für I_1 –regularisierte Verfahren gibt es theoretische Resultate, die Schätz- und Variablenselektions-Konsistenz absichern

8. Bäume

- Bei der Diskriminanzanalyse mit K Klassen haben wir den Raum $\mathcal{X}$ durch die Trennungsflächen in K Regionen partitioniert
- Fällt eine neue Beobachtung in eine der Regionen, so wird die korrespondierende Klasse vorhergesagt
- Bisher: Wir brauchten genau K Regionen, um K Klassen abbilden zu können. Die DA war zudem Likelihood-basiert und benötigte strikte Annahmen über die klassenspezifischen Verteilungen
- Nun: Partitioniere $\mathcal{X}$ in M Regionen R_m und weise allen Beobachtungen der Region die gleiche Score c_m zu. Neben der Score unterliegt aber hier sogar die Anzahl der Regionen einer Optimierung
- Vorteil gegenüber DA: Die Regionen gehen jeweils durch Splitten an einer einzigen Variablen hervor, sind also sehr einfach und benötigen nicht alle Variablen; vergleiche auch mit dem Kapitel Modellwahl

Idee und Bezeichnungen

- Bäume können sowohl für Klassifikation als auch für Regression verwendet werden. Dabei werden verschiedene Splitting-Kriterien verwendet
- Bäume sind, wie hierarchisches Clustering, als Dendrogramme repräsentierbar
- Die Wurzel des Baumes (root) ist im Dendrogramm immer oben
- In dieser Veranstaltung: An den Knoten (nodes) werden binäre Splits vollzogen, d.h., es gehen aus jedem Knoten zwei abstammende Knoten (children nodes) hervor. In dieser Situation wird der Knoten, aus dem die beide children nodes hervorgingen, als parent node bezeichnet
- Knoten, von denen keine weiteren Knoten mehr abstammen, heißen Blätter (leafs)
- Betrachtet man, von einem anderen Knoten als der Wurzel ausgehend, die gesamte abstammende Struktur, so wird diese als Teilbaum (subtree) bezeichnet

- Wir machen für eine Beobachtung X_i die Vorhersage

$$\hat{Y}_i = \hat{f}(X_i) := \sum_{m=1}^M c_m I(X_i \in R_m). \quad (25)$$

- Bei der Regression ist die Residuenquadratsumme ein geeignetes Gütemaß
- Mache durch $R_m = R_m(j, s)$ jeweils die Split-Variable sowie den Cutoff klar, d.h., es ist

$$R_{m,1}(j, s) := \{X \mid X_j \leq s\}, \quad R_{m,2}(j, s) := \{X \mid X_j > s\}.$$

Die Nummerierung der Regionen wird später angepasst, sobald deren Gesamtanzahl bekannt ist

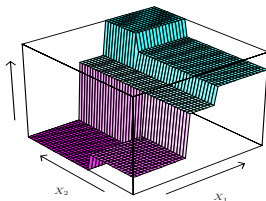
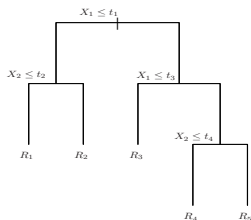
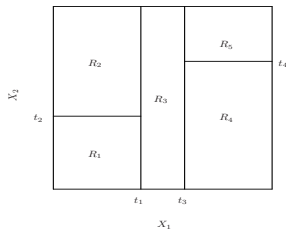
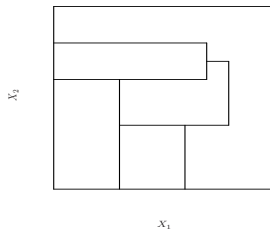
- Für die erste Split-Variable und dem zugehörigen Cutoff sowie der Konstante müssen wir also folgendes Optimierungsproblem lösen:

$$\min_{j,s} \left(\min_{c_1} \left(\sum_{i: X_i \in R_1(j,s)} (Y_i - c_1)^2 \right) + \min_{c_2} \left(\sum_{i: X_i \in R_2(j,s)} (Y_i - c_2)^2 \right) \right).$$

- Das Ausprobieren aller möglichen Partitionen von $\mathcal{X}$ in M Regionen wäre computationell nicht möglich, das Vorgehen, jeweils an einer Variable zu splitten, aber schon
- Der Algorithmus geht alle p Prädiktoren durch, wobei sich jeweils die Anzahl der sinnvollen Cutoffs auf eine endliche Menge reduziert. Für n Beobachtungen sind die einzigen sinnvollen Cutoffs nämlich $X_{(1).j}, \dots, X_{(n-1).j}$ für $X_{(1).j} \leq X_{(2).j} \leq \dots \leq X_{(n).j}$
- Erinnerung: $\operatorname{argmin}_a (\mathbb{E}[(X - a)^2]) = \mathbb{E}[X]$, d.h., die inneren Minimierungsprobleme werden durch die arithmetischen Mittel innerhalb der Regionen gelöst, d.h.,

$$\hat{c}_k = \frac{1}{n_k} \sum_{i: X_i \in R_k} Y_i, \quad n_k := \#\{i \mid X_i \in R_k\}, \quad k = 1, 2.$$

Beispiel: Ein einfacher Baum



■ Probleme:

- ▶ Welche der beiden neuen Regionen soll wieder gesplittet werden?
- ▶ Brauchen wir noch alle vorangegangenen Splits oder kann man ganze Teilbäume wieder löschen?
- ▶ Wann endet der Algorithmus?
- ▶ Wie geht man mit kategoriellen Variablen um?

Komplexität von Bäumen

- Bias-Varianz-Tradeoff: Große (komplexe) Bäume haben eine hohe Varianz und neigen zum Overfitten, während kleine Bäume oft die Daten nicht gut genug abbilden und daher einen Bias haben
- Idee: Vergrößere den Baum nur, solange durch den neuen Split die RSS signifikant geringer wird
- Das ist allerdings eine Vorwärts-Methode, die gute Split-Variablen benachteiligen könnte
- Besser: Berechnen eines komplexen Baumes und nachträgliches Stutzen (Pruning)
- Da aber der Vergleich des kreuzvalidierten Fehlers aller Teilbäume computationell zu aufwändig wäre, wird ein geeigneter Strafterm definiert

Komplexität von Bäumen

- Sei $n_m := \#\{i \mid X_i \in R_m\}$ und $\hat{c}_m := \frac{1}{n_m} \sum_{i: X_i \in R_m} Y_i$.

- Definiere für einen Baum T

$$Q_m(T) := \frac{1}{n_m} \sum_{i: X_i \in R_m} (Y_i - \hat{c}_m)^2.$$

- Dann ist das regularisierte Kriterium für den Vergleich von Bäumen gegeben durch

$$C_\lambda(T) := \sum_{m=1}^{|T|} n_m Q_m(T) + \lambda |T|.$$

Man nennt dies **cost complexity criterion**, das ganze Vorgehen **cost complexity pruning**

- Praktisches Vorgehen: Starte mit dem vollen Baum T_0 . Entferne sukzessive denjenigen Knoten (und damit den daran hängenden Teilbaum, erhalte also einen neuen Baum T_1), für den die Differenz von $\sum_m n_m Q_m(T_0)$ und $\sum_m n_m Q_m(T_1)$ am geringsten ist. Fahre dann mit T_1 ebenso fort. Man kann zeigen, dass der optimal-gestutzte Baum, der C_λ minimiert, so gefunden werden kann. λ wird über CV bestimmt.

Regressionsbäume: Algorithmus

Berechne einen komplexen Baum durch binäre Splits. Terminiere, wenn ein weiterer Split eine Mindestanzahl von Beobachtungen pro Region unterschreiten lassen würde $\leadsto$ Baum T_0 ;

Cost complexity pruning $\leadsto T_0 \supset T_1 \supset \dots$;

Partitioniere die Trainingsdaten in V Unterdatensätze ;

Wähle ein Gitter λ_{grid} ;

for $l = 1, \dots, |\lambda_{grid}|$ **do**

for $v = 1, \dots, V$ **do**

 Führe die Schritte oben auf die um den v -ten Unterdatensatz verminderten Trainingsdatensatz aus ;

 Berechne den (nicht regularisierten!) Vorhersage-Fehler des besten Teilbaumes (bzgl. λ) auf dem v -ten Unterdatensatz

end

 Erhalte den kreuzvalidierten Fehler bzgl. λ

end

Wähle das optimale λ ;

Wähle das opt. Element aus der Sequenz $T_0, T_1, \dots$ als finalen Baum ;

Klassifikationsbäume

- Die Tatsache, dass es sich oben um Regressionsbäume handelte, erkannte man ausschließlich an der Verlustfunktion
- Für Klassifikation mit K Klassen kommen drei Verlustfunktionen in Frage. Sei zunächst

$$p_{mk} := \frac{1}{n_m} \sum_{i: X_i \in R_m} I(Y_i = k).$$

- Dann wähle für $Q_m(T)$ entweder

$$\frac{1}{n_m} \sum_{i: X_i \in R_m} I(Y_i \neq k(m)) \quad (\text{Fehlklassifikationsrate}),$$

$$\sum_{k \neq k'} \hat{p}_{mk} \hat{p}_{mk'} = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) \quad (\text{Gini-Index})$$

oder

$$- \sum_k \hat{p}_{mk} \ln(\hat{p}_{mk}) \quad (\text{Kreuz-Entropie}),$$

wobei $k(m) := \operatorname{argmax}_k (\hat{p}_{mk})$

- Der Rest erfolgt analog

Probleme beim Fitten eines Baumes

- Ein Cutoff zum Splitten macht bei kategoriellen Prädiktoren keinen Sinn, stattdessen müssen die Ausprägungen in zwei Gruppen aufgespalten werden
- Der Vergleich aller möglichen Aufteilungen wäre zu aufwändig
- Für binäre Responses gibt es Strategien, indem man die kategoriellen Prädiktoren über die Wahrscheinlichkeit der Regressoren, der Klasse 1 anzugehören, ordnet
- Bäume sind in der Regel sehr instabil, jedoch kann man durch geeignete Kombination verschiedener Bäume dieses Problem vollständig lösen (siehe Kapitel Bagging)

Vor- und Nachteile von Bäumen

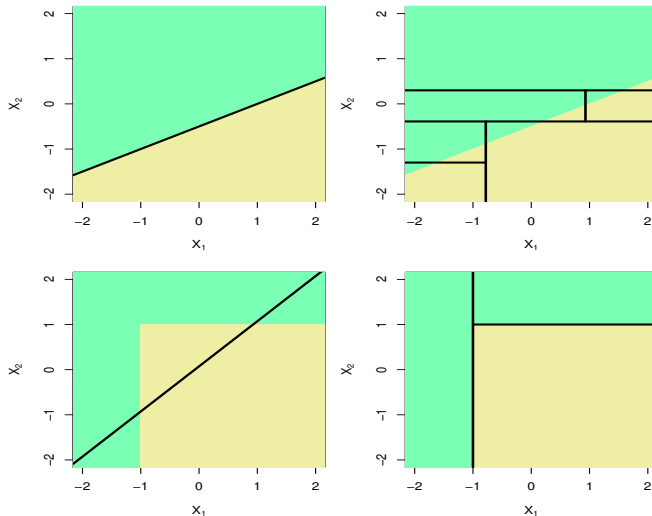
■ Vorteile:

- ▶ Ergebnis ist (auch Laien) sehr einfach zu erklären
- ▶ Sehr gut visualisierbar
- ▶ Kann bei Nichtlinearität schon in einfachen Situationen Regression schlagen (s. Graphik nächste Folie)

■ Nachteile:

- ▶ In der Regel können Bäume im Hinblick auf die Genauigkeit der Vorhersagen nicht mit Regressionsmodellen konkurrieren
- ▶ Bei Regression wird eine stückweise konstante Funktion vorhergesagt
- ▶ Bäume sind nicht robust
- ▶ Bäume liefern keine quantitative Aussage über den Einfluss einer Beobachtung auf die Vorhersage

Beispiel: Baum vs. lineare Regression



Abschließende Bemerkungen

- Bäume führen offensichtlich auch eine Form von (sparser) Modellwahl aus, gehen jedoch iterativ vor
- Bäume oder gar Stümpfe (stumps), d.h., Bäume, die nur aus Wurzel und Blättern bestehen, sind populäre Baselearner für Boosting (siehe Kapitel Boosting)
- Der Algorithmus, der Klassifikations- und Regressionsbäume wie in diesem Kapitel angegeben berechnet, wird als CART bezeichnet
- Konkurrierend mit CART: C5.0, eine verbesserte Variante des Algorithmus C4.5, der jedoch nicht notwendigerweise binäre Splits verwendet

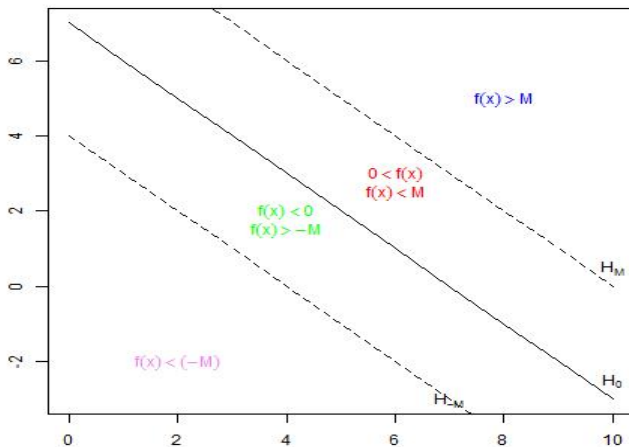
9. Support Vector Machines

- Angenommen, wir haben Trainingspunkte $(X_1, Y_1), \dots, (X_n, Y_n)$ eines binären Klassifikationsproblems mit o.E. $Y_i \in \{-1, 1\}$
- Bereits kennengelernt: LDA und QDA mit dem Ziel, eine Hyperfläche zwischen die Daten zu legen und so zu trennen
- QDA ist bereits sehr flexibel, aber brauchte Modellannahmen
- Nahe den Entscheidungsgrenzen ist die Vorhersage oft nicht so gut
- Gibt es einen flexibleren Weg, mit dem man die Punkte noch besser trennen kann?

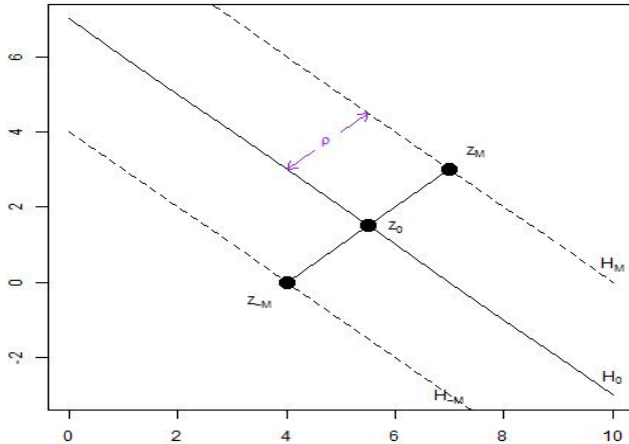
Idee von SVCs (Support Vector Classifiers)

- Sei H^0 die Hyperfläche, die durch die Daten gelegt wird
- Die Hyperflächen seien dabei allgemein definiert durch
$$H^c := \{x \mid f(x) = b + w^T x = c\}.$$
- Ziel: Finde M , sodass keine Daten zwischen H^M und H^{-M} liegen
- Dieses Prinzip führt zu **harten SVCs ("hard SVC")**
- Genauer: Die Distanz (margin) ρ zwischen H^M und H^0 (äquivalent: zwischen H^0 und H^{-M}) soll maximiert werden

Visualisierung für $p=2$



Visualisierung für $p=2$



- Die obigen Bedingungen lassen sich formulieren durch

$$f(X_i) \geq M \text{ für } Y_i = 1, \quad f(X_i) \leq -M \text{ für } Y_i = -1,$$

also kurz $Y_i(b + X_i^T w) \geq M$

- Falls $b + X_i^T w = \pm M$, so heißt das Paar (X_i, Y_i) **Trägervektor (support vector)**
- Da wir f o.E. mit M skalieren können, nehmen wir im Folgenden immer $M = 1$ an

Bemerkung 8.1

Die Distanz ρ ist gegeben durch

$$\rho = \frac{1}{\|w\|}.$$

Die Maximierung von ρ unter den geforderten Nebenbedingungen liefert folgendes Optimierungsproblem:

$$\max_{w,b} \left(\frac{1}{\|w\|} \right) \quad \text{s.t.} \quad Y_i(X_i^T w + b) \geq 1 \quad \forall i$$

Optimierungsproblem für harte SVCs

Satz 8.2

Das Optimierungsproblem

$$\min_{w,b} \left(\frac{1}{2} \|w\|^2 \right) \quad \text{s.t.} \quad Y_i(X_i^T w + b) \geq 1 \quad \forall i \quad (26)$$

hat die eindeutige Lösung

$$\hat{w} = \sum_i \hat{\alpha}_i Y_i X_i = \sum_{\text{support vectors}} \hat{\alpha}_i Y_i X_i. \quad (27)$$

Die $\hat{\alpha}_i$ sind dabei die Lösung des dualen Problems (Wolfe-Dual)

$$\max_{\alpha} \left(\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j Y_i Y_j X_i^T X_j \right) \quad \text{s.t.}$$

$$\sum_i \alpha_i Y_i = 0, \quad \alpha_i \geq 0 \quad \forall i = 1, \dots, n, \quad \alpha_i [Y_i(X_i^T w + b) - 1] = 0 \quad \forall i = 1, \dots, n.$$

Der Achsenabschnitt $\hat{b}$ kann mit Hilfe eines beliebigen Trägervektors bestimmt werden durch

$$X_i^T \hat{w} + \hat{b} = 1 \quad \text{für} \quad Y_i = 1 \Leftrightarrow \hat{b} = 1 - X_i^T \hat{w}. \quad (28)$$

- Wie man in dem Theorem sieht, ist $\hat{\alpha}_i$ genau dann von Null verschieden, wenn (X_i, Y_i) ein Trägervektor ist
- D.h., SVCs ermöglichen eine Form von **Zeilenwahl** (nicht Modellwahl!) für Klassifikation
- Wir bekommen durch die Lösung also den Klassifizierer

$$\hat{Y}^{neu} := \text{sign}(\hat{f}(X^{neu})) = \text{sign}((X^{neu})^T \hat{w} + \hat{b})$$

Eigenschaften von trennenden Hyperflächen

Satz 8.3

Die Lösung des SVC-Optimierungsproblems hat folgende Eigenschaften:

- a) Die Parameter $\hat{\mathbf{w}}$ und $\hat{\mathbf{b}}$ sind eindeutig und damit auch die Hyperfläche.
- b) Die Darstellung von $\hat{\mathbf{w}}$ in Termen von Trägervektoren ist nicht eindeutig.
- c) Das Maximum des dualen Problems nimmt folgenden Wert an:

$$\max(W(\alpha)) = \frac{1}{2} \sum_i \hat{\alpha}_i = \frac{\|\hat{\mathbf{w}}\|^2}{2}.$$

- d) Es gilt die Ungleichung

$$W(\alpha) < W(\hat{\alpha}) = \frac{1}{2} \frac{1}{\rho^2(\hat{\mathbf{w}})}.$$

Definition 8.4

Aufgrund der in c) angezeigten Nicht-Eindeutigkeit der Darstellung von $\hat{\mathbf{w}}$ sind die jeweils verwendeten Trägervektoren auch nicht eindeutig. Trägervektoren, die in jeder Darstellung von $\hat{\mathbf{w}}$ auftauchen, heißen **essentielle Trägervektoren**.

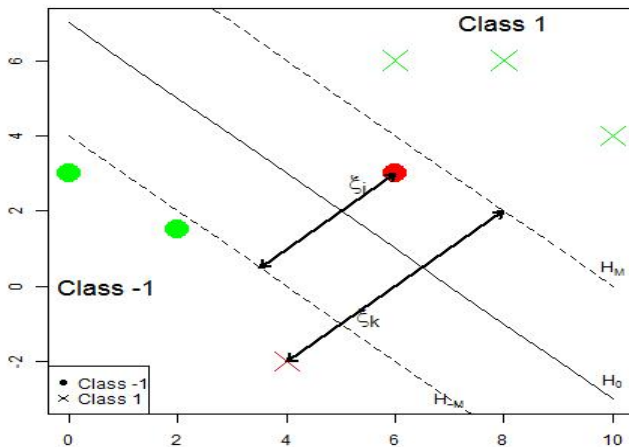
Idee von weichen SVCs

- Der bisherige Ansatz der harten SVCs war offenbar noch nicht sinnvoll, da die Daten in der Regel nicht perfekt trennbar sind
- Das bedeutet, am Ende werden Datenpunkte auf der falschen Seite der trennenden Hyperfläche liegen
- Idee: Führe **Slack-Variablen** in folgender Form ein:

$$\xi_i = \begin{cases} 0, & \text{falls } X_i \text{ auf der richtigen Seite liegt} \\ \text{dist}(X_i, H^M) & \text{sonst, falls } Y_i = 1 \\ \text{dist}(X_i, H^{-M}) & \text{sonst, falls } Y_i = -1 \end{cases}$$

- Dadurch erhalten wir einen **Strafterm**, den wir mit einem **Kostenparameter C** versehen
- Das resultierende Optimierungsproblem bezeichnen wir als weiches SVC-Optimierungsproblem

Visualisierung des weichen SVC



- Das Optimierungsproblem ist konkret gegeben durch

$$\min_{\mathbf{w}, b} \left(\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \right) \quad (29)$$

$$\text{s.t. } Y_i(\mathbf{X}_i^T \mathbf{w} + b) \geq 1 - \xi_i \text{ und } \xi_i \geq 0 \quad \forall i = 1, \dots, n$$

- Je höher der Kostenparameter C gewählt wird, desto kleiner wird die Distanz
- Bei $C = \infty$ erhalten wir wieder die harte SVC

Optimierungsproblem des weichen SVC

Satz 8.5

Das Optimierungsproblem 29 hat die eindeutige Lösung

$$\hat{\mathbf{w}} = \sum_{\text{support vectors}} \hat{\alpha}_i \mathbf{Y}_i \mathbf{X}_i.$$

Der Achsenabschnitt $\hat{\mathbf{b}}$ kann analog wie beim harten SVC durch einen beliebigen Trägervektor berechnet werden. Die $\hat{\alpha}_i$ sind die Lösung des dualen Problems

$$\max_{\alpha} \left(\sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j \mathbf{Y}_i \mathbf{Y}_j \mathbf{X}_i^T \mathbf{X}_j \right)$$

$$\text{s.t. } \sum_i \alpha_i \mathbf{Y}_i = \mathbf{0}, \quad 0 \leq \alpha_i \leq C \quad \forall i = 1, \dots, n,$$

$$\alpha_i [\mathbf{Y}_i (\mathbf{X}_i^T \mathbf{w} + \mathbf{b}) - 1 + \xi_i] = 0, \quad \gamma_i \xi_i = 0 \quad \forall i = 1, \dots, n$$

SVC als regularisiertes Risikominimierungsproblem

Bemerkung 8.6

Das regularisierte Problem

$$\min_{w,b} \left(\sum_i L^{\text{hinge}}(Y_i, f(X_i)) + \frac{\lambda}{2} \|w\|^2 \right)$$

mit dem Regularisierungsparameter $\lambda := C^{-1} > 0$ und dem **Hinge-Verlust**

$$L^{\text{hinge}}(Y_i, f(X_i)) := \max(1 - Y_i f(X_i), 0) := [1 - Y_i f(X_i)]_+,$$

ist äquivalent zum weichen SVC-Problem.

Bemerkungen zum weichen SVC

- Alle Trainingspunkte, die auf der falschen Seite der Hyperfläche liegen, sind Trägervektoren
- Die trennende Hyperfläche ist nach wie vor linear

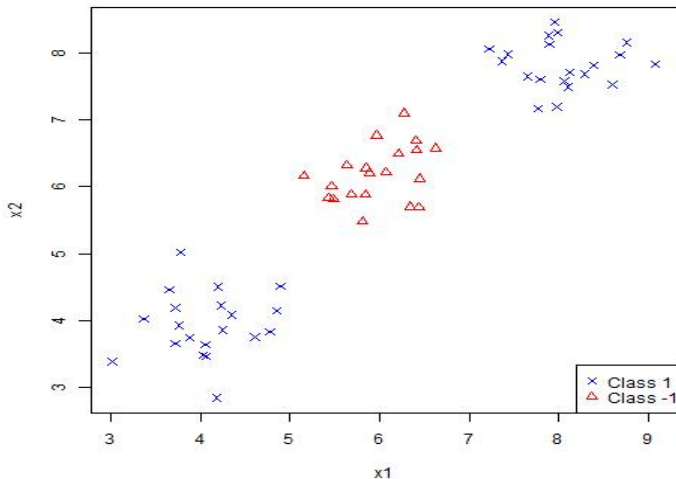
Satz 8.7

Falls $\mathcal{X} \subset \{\mathbf{X} \in \mathbb{R}^p \mid \|\mathbf{X}\| \leq K\}$, so gilt für die VC-Dimension h der Familie $\mathcal{F}_A$ aller trennenden Hyperflächen mit $\|\mathbf{w}\| \leq A$, dass

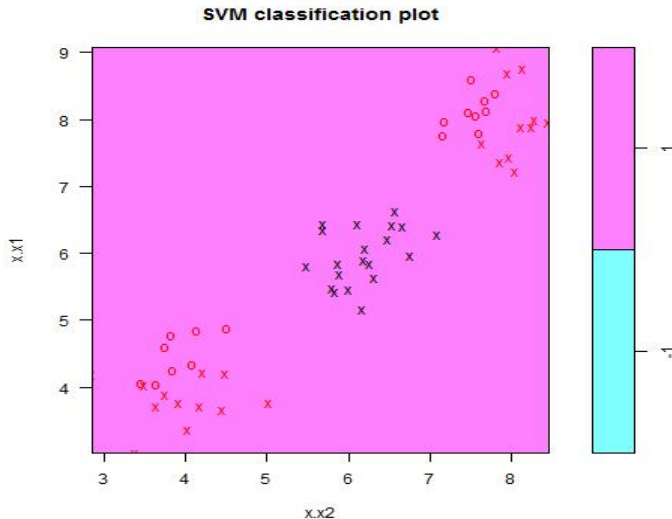
$$h \leq \min(\lfloor A^2 K^2 \rfloor, p) + 1.$$

Vapnik: Statistical Learning Theory, 1998, Thm. 10.3

Beispiel: Linear nicht-separable Daten



Ergebnis einer weichen SVC



Einführung einer nichtlinearen Transformation

- Analog wie bei LDA und QDA: Wenn wir die Daten nicht durch eine lineare Hyperfläche trennen können, versuchen wir, eine nichtlineare Hyperfläche zu konstruieren
- Vorgehen wie bei nichtlinearer Regression: Transformiere die X –Werte durch eine nichtlineare Funktion $h : \mathcal{X} \rightarrow H, h(x) := (h_1(x), h_2(x), \dots)$, wobei H ein hoch-dimensionaler (auch möglich: unendlich-dimensionaler) Hilbertraum sei
- Damit definieren wir die trennende Hyperfläche durch

$$H^0 := \{X \mid f(X) := w^T h(X) + b = 0\}$$

- Das weiche Optimierungsproblem lautet analog

$$\min_{w,b} \left(\frac{1}{2} \|w\|^2 + c \sum_i \xi_i \right)$$

$$\text{s.t. } Y_i(h(X_i)^T w + b) \geq 1 - \xi_i \text{ und } \xi_i \geq 0 \forall i = 1, \dots, n$$

- Analog erhalten wir die Lösung

$$\hat{w} = \sum_{\text{support vectors}} \hat{\alpha}_i Y_i h(X_i)$$

sowie den Achsenabschnitt $\hat{b}$

- Die Klassifikationsregel ist

$$\hat{Y}^{neu} = \text{sign}(h^T(X^{neu})\hat{w} + \hat{b})$$

Bemerkungen zu nichtlinearen Hyperflächen

- Wir bezeichnen in der obigen Situation den Raum H als **feature space**
- Die Hyperfläche ist offenbar linear in H , aber die Projektion auf $\mathcal{X}$ ist i.A. nichtlinear
- Problem: Bei hoch- oder gar unendlich-dimensionalen Räumen H würden wir immense Probleme bei der Berechnung bekommen, wenn wir h explizit definieren müssten
- Darüber hinaus stellt sich die Frage, wie die Funktion h überhaupt gewählt werden soll

- Beobachtung: Wir haben

$$\hat{f}(X^{neu}) := h^T(X^{neu})\hat{w} + \hat{b} = \sum_{\text{support vectors}} \hat{\alpha}_i Y_i h^T(X_i) h(X^{neu}) + \hat{b}$$

- Die Auswertung von $\hat{f}$ benötigt also nur die **inneren Produkte** $\langle h(X_i), h(X^{neu}) \rangle$
- **Wir brauchen die Ausdrücke $h(X_i), h(X^{neu})$ also gar nicht explizit zu kennen!**

Definition 8.8

Ein **Kern K bzgl. einer Funktion h** ist definiert als symmetrische Funktion $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ mit

$$K(x, y) := \langle h(x), h(y) \rangle.$$

Bemerkung 8.9

Cauchy-Schwarz: $K^2(x, y) \leq K(x, x)K(y, y)$

Reproducing Kernel Hilbert Spaces

Lemma 8.10

Seien $K_1, K_2 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, $\mathcal{X} \subset \mathbb{R}^p$ Kerne. Dann sind auch folgende Funktionen Kerne:

a) $K_1 + K_2$ b) $K_1 K_2$ c) $a K_1, a \in \mathbb{R}_+$

d) $K(x, z) := f(x)f(z), f : \mathcal{X} \rightarrow \mathbb{R}$

e) $K_3(\varphi(x), \varphi(z)), K_3 : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}$ Kern, $\varphi : \mathcal{X} \rightarrow \mathbb{R}^m$

f) $x^T B z, B \in \mathbb{R}^{n \times n}$ spsd.

Definition 8.11

Ein **RKHS** $\mathcal{H}_K$ ist ein durch einen Kern K induzierter Hilbertraum mit einem Skalarprodukt $\langle \cdot, \cdot \rangle_{\mathcal{H}_K}$ so, dass die **reproduzierende Eigenschaft**

$$f(x) = \langle f(y), K(x, y) \rangle_{\mathcal{H}_K} \quad \forall f(y) \in \mathcal{H}_K$$

erfüllt ist.

Lemma 8.12

In einem RKHS gilt

$$\langle K(y, x), K(z, x) \rangle_{\mathcal{H}_K} = K(y, z) \quad \forall x, y, z \in H.$$

Reproducing kernel Hilbert spaces

Satz 8.13 (Mercer)

Sei K eine symmetrische, stetige Funktion, $K \in L_2(\mu \otimes \mu)$ für ein sigma-finites Maß μ auf $\mathbb{B} \cap \mathcal{X}$. Dann hat K die Entwicklung

$$K(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{\infty} \lambda_i \varphi_i(\mathbf{x}) \varphi_i(\mathbf{x}')$$

mit Koeffizienten $\lambda_i \geq 0$ genau dann, falls für alle $\psi \in L_2(\mu)$ gilt, dass die Ungleichung

$$\iint K(\mathbf{x}, \mathbf{x}') \psi(\mathbf{x}) \psi(\mathbf{x}') d\mathbf{x} d\mathbf{x}' \geq 0$$

erfüllt ist. Die λ_i und die φ_i sind dann Eigenwerte und Eigenfunktionen des zugehörigen Integraloperators

$$T : L_2(\mathcal{X}) \rightarrow L_2(\mathcal{X}), (Tf)(\cdot) := \int K(\cdot, t) f(t) dt.$$

Beispiel 8.14

Die folgenden Abbildungen K erfüllen die Bedingungen aus dem Satz von Mercer, sind also (Mercer-)Kerne:

a) Der Polynom-Kern

$$K(x, x') = (c + \gamma x^T x')^d$$

mit $d \in \mathbb{N}$, $c, \gamma \geq 0$.

b) Der radiale Kern

$$K(x, x') = \exp(-\gamma \|x - x'\|_2^2)$$

mit einem $\gamma > 0$.

c) Der sigmoide Kern

$$K(x, x') = S(v x^T x' + c)$$

mit $c \geq v$ und einer sigmoiden Funktion S .

Satz 8.15

Jeder Mercer-Kern erzeugt einen RKHS $(H, K, \langle \cdot, \cdot \rangle_{\mathcal{H}_K})$.

Satz 8.16

Das SVC-Optimierungsproblem kann geschrieben werden als

$$\begin{aligned} \min_{f \in \mathcal{H}_K} & \left(\sum_{i=1}^n L(y_i, f(x_i)) + \frac{\nu}{2} \|f\|_{\mathcal{H}_K}^2 \right) \\ &= \min_{b, \theta} \left(\sum_{i=1}^n \left[1 - y_i \left(b + \sum_{k=1}^{\infty} \theta_k \varphi_k(x) \right) \right]_+ + \frac{\nu}{2} \sum_{k=1}^{\infty} \frac{\theta_k^2}{\lambda_k} \right) \end{aligned}$$

mit Eigenwerten λ_k aus der Mercer-Entwicklung des Kerns K .

Satz 8.17 (Representer Theorem)

Sei $\mathcal{H}_K = (\mathbf{H}, \mathbf{K}, \langle \cdot, \cdot \rangle_{\mathcal{H}_K})$ ein RKHS. Für $\mathbf{f} \in \mathcal{H}_K$ und eine Verlustfunktion L hat das Optimierungsproblem

$$\min_{\mathbf{f} \in \mathcal{H}_K} \left(\sum_{i=1}^n L(y_i, \mathbf{f}(x_i)) + \frac{\nu}{2} \|\mathbf{f}\|_{\mathcal{H}_K}^2 \right)$$

eine **eindeutige Lösung** $\hat{\mathbf{f}}$ mit **endlich-dimensionaler Darstellung**

$$\hat{\mathbf{f}}(x) = \sum_{i=1}^n \hat{\alpha}_i K(x, x_i).$$

Das Representer Theorem garantiert uns also, dass das SVC-Problem mit Kernen trotz Projektion der Regressoren in einen potentiell unendlich-dimensionalen Hilbertraum trotzdem endlich-dimensional darstellbar ist!

Beispiel mit einem polynomialen Kern

Beispiel 8.18

Sei $p = 2$ und sei K ein Polynom-Kern mit Parametern $c = 1, \gamma = 1, d = 2$.
Dann gilt

$$\begin{aligned} K(x, x') &= (1 + x^T x')^2 = (1 + x_1 x'_1 + x_2 x'_2)^2 \\ &= 1 + 2x_1 x'_1 + 2x_2 x'_2 + x_1^2 (x'_1)^2 + x_2^2 (x'_2)^2 + 2x_1 x'_1 x_2 x'_2. \end{aligned}$$

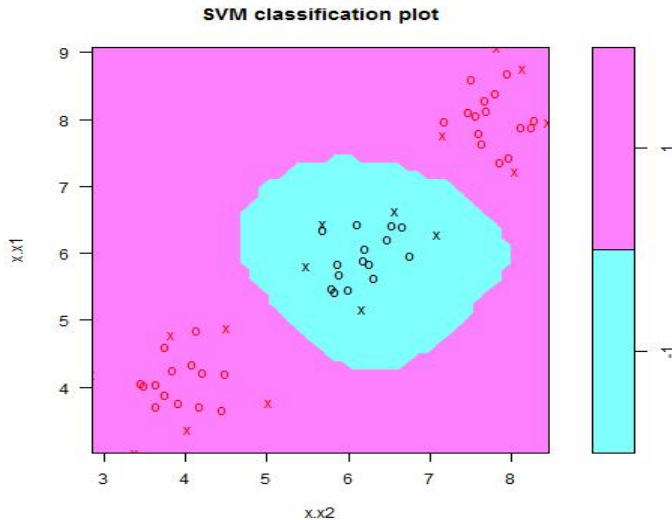
Der Kern repräsentiert also die Transformation

$$h(x) = \begin{pmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \end{pmatrix}$$

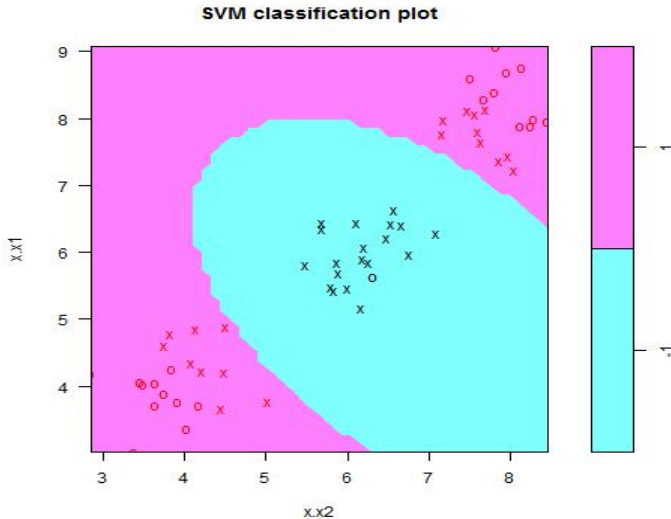
mit

$$K(x, x') = h^T(x)h(x') = \sum_{m=1}^6 h_m(x)h_m(x').$$

SVM mit radialem Kern



SVM mit polynomialem Kern



- Kerne werden auch für lokale Risikominimierung, d.h., für Risikominimierung in festen Umgebungen, verwendet
- Sei dazu $K^{hard}(x, x_0, c) := I(\|x - x_0\| < c/2)$ die harte Vicinity-Funktion und $K^{soft}(x, x_0, c) := \exp(-\|x - x_0\|^2/c)$ die weiche Vicinity-Funktion

Definition 8.19

Sei

$$\mathcal{K}(x_0, c) := \int K(x, x_0, c) dF(x).$$

Dann ist das **lokale Risikofunktional** gegeben durch

$$L(c, x_0) := \int L(y, f(x)) \frac{K(x, x_0, c)}{\mathcal{K}(x_0, c)} dF(x).$$

- Werden für jedes x_0 sowohl f als auch c optimiert, spricht man von **lokaler Approximation**. Verwendet man immer das gleiche c , von **semilokaler Approximation**
- Die Verwendung der quadratischen Verlustfunktion und der Funktion $f(x, \theta) := \theta$ führt durch die semilokale Approximation auf den **Nadaraya-Watson-Moving-Average-Schätzer**

$$\hat{\theta} = \sum_i \frac{K(X_i, x_0, c)}{\sum_j K(X_j, x_0, c)} Y_i$$

einer Variante des kNN

- Weiteres Beispiel: Kerndichteschätzer mit Gaußkernen führen zu Summanden $\varphi((t - X_i)/h)$ mit Bandweite h

- SVCs produzieren lineare trennende Hyperflächen zwischen binären Daten
- Nichtlineare trennende Hyperflächen können durch Transformation der Regressoren berechnet werden
- Da lediglich innere Produkte für die Berechnungen benötigt werden, kann man sich auf Kerne zurückziehen
- Sobald solche Kerne benutzt werden, spricht man von **Support Vector Machines (SVMs)**
- Bisher noch nicht: Wie geht man vor, wenn man $K > 2$ Klassen hat?

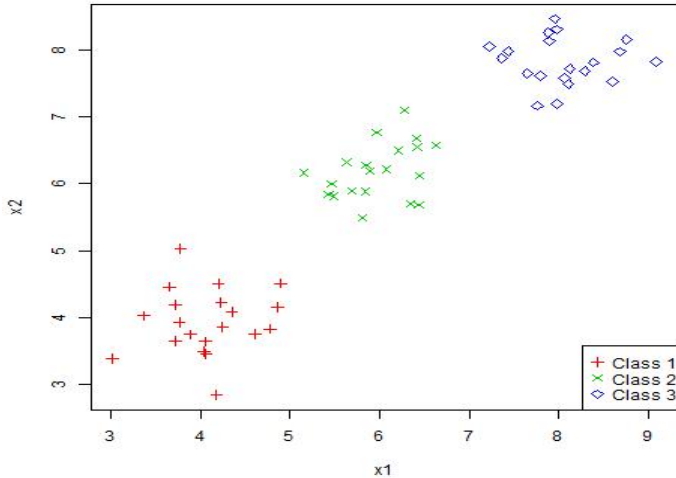
Mehr als 2 Klassen

- Idee wie bei Logit oder LDA: Vergleiche jeweils zwei Klassen
- Zwei Möglichkeiten: **one vs. one** und **one vs. all**
- **One vs. one (ovo):** Berechne für alle $\binom{K}{2}$ Paare von zwei Klassen die trennende Hyperfläche. Wende auf eine neue Beobachtung alle $\binom{K}{2}$ korrespondierenden Klassifikationsregeln an und verwende die **majority vote** für die Klassifikation der neuen Beobachtung
- **one vs. all (ova):** Berechne für $k = 1, \dots, K$ die trennende Hyperfläche zwischen jeweils Klasse k und allen anderen („Klasse $\neg k$ “). Nenne die die Hyperfläche definierende Funktion jeweils $\hat{f}^k$
- Achtung: Es kann passieren, dass man für alle k die Beobachtung in Klasse $\neg k$ einordnen würde
- Die ova-Klassifikationsregel ordnet die Beobachtung daher in die Klasse k^* mit

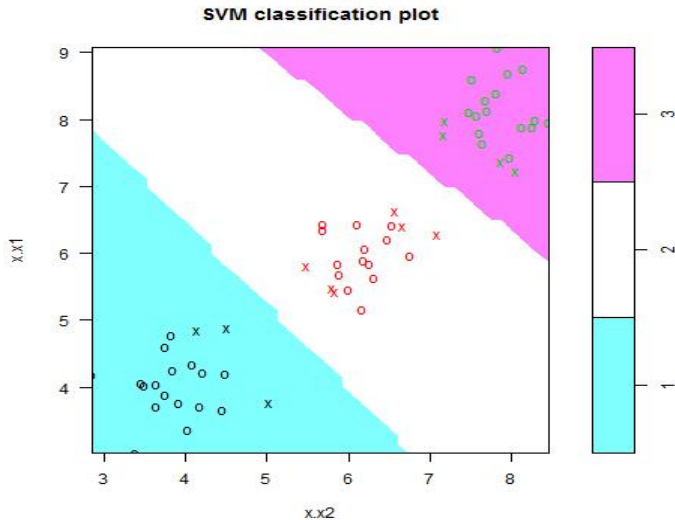
$$k^* = \operatorname{argmax}_k(\hat{f}^k(x))$$

ein, d.h., die Klasse, in die x am ehesten gehört

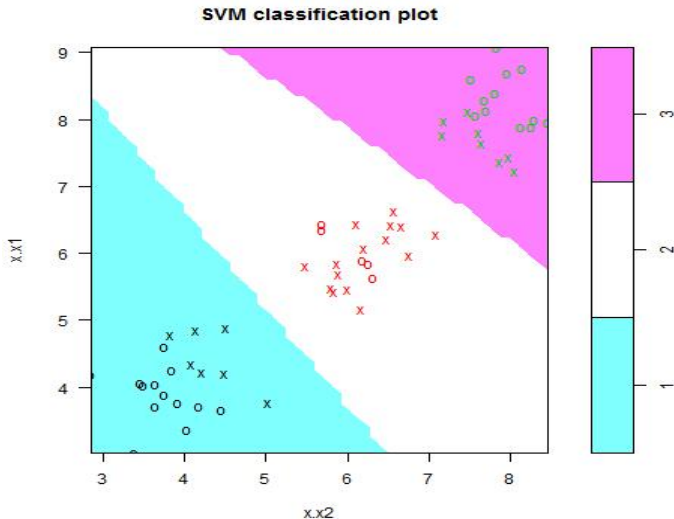
Beispiel mit $K = 3$



Weiche SVC



SVM mit einem radialen Kern

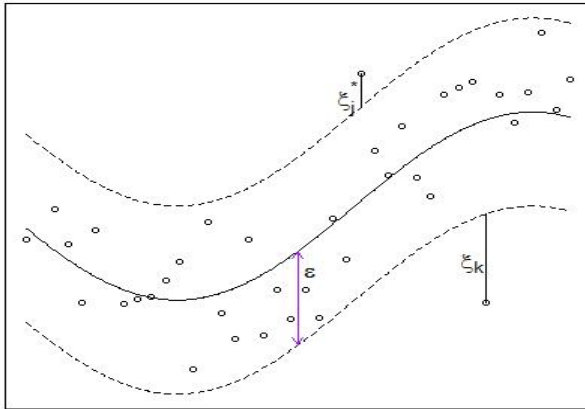


Support Vector Regression (SVR)

- SVMs können sogar für Regression, also für die Approximation von Funktionen, verwendet werden
- Idee: Berechne eine Hyperfläche, sodass möglichst viele Beobachtungen in einem ε -Schlauch um diese liegen
- Da wir nicht erwarten können, dass alle Beobachtungen innerhalb des ε -Schlauches liegen, führen wir wie bei der weichen SVC Slack-Variablen ξ_i, ξ_i^* ein
- Konvention: $\xi_i^* \neq 0$, falls Y_i oberhalb des ε -Schlauches liegt, $\xi_i \neq 0$, falls Y_i unterhalb liegt
- Formal:

$$\xi_i^* = (Y_i - \mathbf{w}^T \mathbf{X}_i - \mathbf{b} - \varepsilon)_+, \quad \xi_i = (\mathbf{w}^T \mathbf{X}_i + \mathbf{b} - Y_i - \varepsilon)_+.$$

Visualisierung der SVR-Idee



- Dies führt zunächst auf das Optimierungsproblem

$$\begin{aligned} \min_{w,b} \left(\frac{1}{2} \|w\|^2 + C \sum_i \xi_i + \xi_i^* \right) \quad \text{s.t.} \\ f(X_i) - Y_i \leq \varepsilon + \xi_i, \quad Y_i - f(X_i) \leq \varepsilon + \xi_i^*, \quad \xi_i, \xi_i^* \geq 0. \end{aligned} \quad (30)$$

- Mit der ε -**insensitiven Verlustfunktion**

$$L^{\varepsilon-ins}(z) := \begin{cases} |z| - \varepsilon, & |z| \geq \varepsilon \\ 0, & |z| < \varepsilon \end{cases},$$

können wir obiges SVR-Optimierungsproblem äquivalent als regularisiertes Minimierungsproblem auffassen, konkret

$$\min_{w,b} \left(\sum_{i=1}^n L^{\varepsilon-ins}(f(X_i) - Y_i) + \frac{\lambda}{2} \|w\|^2 \right).$$

Satz 8.20

Das Optimierungsproblem (30) hat die eindeutige Lösung

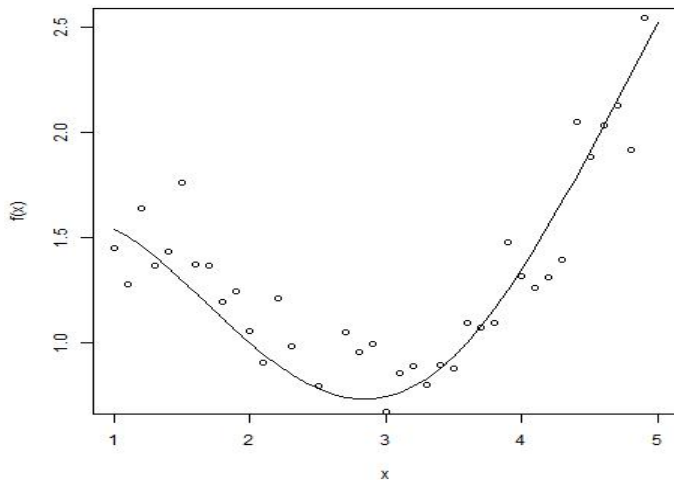
$$\hat{\mathbf{w}} = \sum_{\text{support vectors}} (\hat{\alpha}_i^* - \hat{\alpha}_i) \mathbf{X}_i$$

und $\hat{\mathbf{b}}$ kann wieder durch einen beliebigen Trägervektor berechnet werden.
Die Koeffizienten $\hat{\alpha}_i, \hat{\alpha}_i^*$ sind die Lösung des dualen Problems

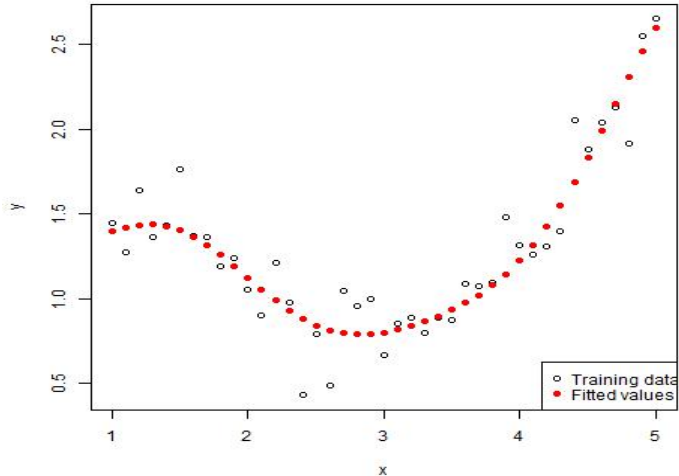
$$\begin{aligned} \max_{\alpha, \alpha^*} & \left(\sum_i Y_i (\alpha_i^* - \alpha_i) - \varepsilon \sum_i (\alpha_i^* + \alpha_i) - \frac{1}{2} \sum_i \sum_j (\alpha_i^* - \alpha_i) (\alpha_j^* - \alpha_j) \mathbf{X}_i^T \mathbf{X}_j \right) \\ \text{s.t.} & \sum_i \alpha_i^* - \alpha_i = 0 \quad \text{und} \quad 0 \leq \alpha_i, \alpha_i^* \leq C \end{aligned}$$

- Die Trägervektoren sind die Paare $(\mathbf{X}_i, Y_i)$, sodass Y_i außerhalb des ε -Schlauches liegt, d.h., falls $\max(\xi_i, \xi_i^*) > 0$
- Der Kern-Trick funktioniert analog

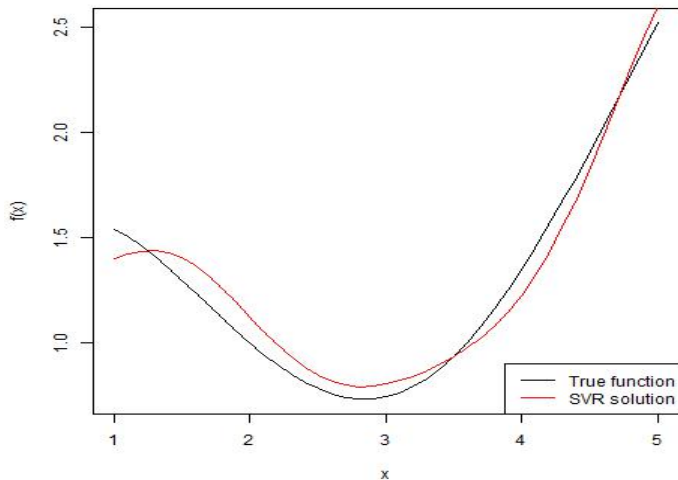
Einfaches Beispiel: Die wahre Funktion



Gefittete Werte des SVR-Modells



Wahre Funktion vs. SVR-Lösung



- (Christmann, Steinwart 2007) zeigen, unter welchen Bedingungen SVMs konsistent sind. Wesentlich ist dabei, dass $\mathcal{X} \subset \mathbb{R}^p$ kompakt ist und dass der Kern K universell ist, d.h., K ist stetig und dessen erzeugter RKHS ist dicht in $(\mathcal{C}(\mathcal{X}), \|\cdot\|_\infty)$
- (Hable 2011) zeigt, unter welchen Bedingungen SVMs (für Regression und Klassifikation) asymptotisch normal sind
- Man kann zeigen (Jaggi 2013), dass Lasso und SVM in dem Sinne äquivalent sind, als dass man zu jedem Lasso-Problem ein SVM-Problem konstruieren kann, das die gleiche Lösung hat und umgekehrt
- (Dumpert 2017) partitioniert $\mathcal{X}$ in disjunkte Regionen $\mathcal{X}_i$ und berechnet dort jeweils **lokalisierte SVMs**
- (Hastie 2004) zeigen, dass die SVM-Regularisierungspfade stetig im Dualraum sind, da die Verlustfunktion stückweise linear und der Strafterm quadratisch ist $\leadsto$ `svmpath`

Kern-Trick für hochdimensionale Daten?

- Bisher war der Kern-Trick dahingehend sehr hilfreich, als dass man sich von einer hochdimensionalen Transformation h auf die Kernmatrix, also die Matrix der paarweisen Skalarprodukte, zurückziehen konnte
- Die Dimension der Kernmatrix ist allerdings $n \times n$, d.h., für eine sehr hohe Anzahl an Beobachtungen werden die Auswertung und der Speicherverbrauch teuer. Insgesamt benötigt man $\mathcal{O}(pn^2)$ Operationen für die Berechnung.
- Daher wird versucht, die Kernmatrix geeignet zu approximieren
- (Rahimi, Recht, 2007) versuchen, über zufällige Fourier-Basisfunktionen eine Transformation $z : \mathbb{R}^p \rightarrow \mathbb{R}^D$, $D < p$, zu finden, sodass $K(x, y) \approx z^T(x)z(y)$, d.h., man verwendet letztendlich keinen Kern mehr bei der Berechnung, sondern nur die transformierten Spalten
- Weitere Ansätze: Zufälliges Ziehen von Beobachtungen und Ausführung eines stochastischen Gradientenverfahrens auf solchen Unterstichproben

- Der RKHS-Strafterm ist letztendlich ein l_2 -Strafterm. Somit ist es ein natürlicher Ansatz, daraus einen l_1 -Strafterm zu machen
- (Zhu et. al., 2004) hat diesen Ansatz gemacht, den (Zou, 2007) verbessert haben, indem sie einen gewichteten Strafterm wie beim adaptiven Lasso gewählt haben, mit Gewichten, die durch die Koeffizienten einer gewöhnlichen SVM berechnet werden
- Da hochkorrelierte Variablen wie beim Lasso durch eine l_1 -Regularisierung Probleme machen und i.A. nur eine dieser Variablen gewählt wird, haben (Wang et. al., 2006) einen EN-Strafterm eingeführt und in (Wang et. al., 2008) eine huberisierte Hinge-Verlustfunktion verwendet, um diese SVM noch zu robustifizieren
- Nichtkonvexe Strafterme wie SCAD oder MCP wurden in (Zhang et. al., 2016) für eine fast exponentiell wachsende Anzahl an Prädiktoren studiert. Sie zeigen Orakel Eigenschaften solcher SVMs.

- Bisher hatten wir den Shatter-Koeffizienten, also die Anzahl aller durch eine Funktionenklasse $\mathcal{F}$ gegebenen Möglichkeiten, eine Stichprobe in zwei Klassen aufzuteilen
- Die maximale Stichprobengröße, die wir in allen Varianten trennen konnten, war die VC-Dimension der Klasse $\mathcal{F}$
- Die VC-Dimension einer Klasse von reellwertigen Funktionen konnten wir durch die Pseudo-Dimension, also die VC-Dimension der Familie aller Subgraphen, beschreiben
- Bei SVMs ist die Situation noch anders, da wir sogar **mit Distanz trennen wollen**
- Die Erweiterung der VC-Dimension für solche Fälle nennt sich **fette Shatter-Dimension**

Definition 8.21

Seien die Klasse $\mathcal{F}$ sowie $S := \{X_1, \dots, X_n\} \subset \mathcal{X}$ gegeben. Die höchste Zahl n , für die $\mathbf{c} := (c_1, \dots, c_n) \in [0, 1]^n$ existiert, sodass für alle $\mathbf{e} \in \{0, 1\}^n$ ein $f \in \mathcal{F}$ existiert mit

$$f(X_i) \geq c_i + \varepsilon, \text{ falls } e_i = 1, \quad \text{und} \quad f(X_i) \leq c_i - \varepsilon \text{ für } e_i = 0,$$

heißt **fette** ε -**Shatterdimension**, kurz $\text{fat}_\varepsilon(\mathcal{F})$. Können beliebig lange Vektoren mit Margin 2ε getrennt werden, ist $\text{fat}_\varepsilon(\mathcal{F}) := \infty$.

Bemerkung 8.22

Äquivalent kann dies definiert werden über $\mathbf{b} := (b_1, \dots, b_m) \in \{-1, 1\}^n$ anstatt der $\mathbf{e}$ -Sequenz und der Bedingung

$$b_i(f(X_i) - c_i) \geq \varepsilon.$$

Definition 8.23

Die ε -**Pseudodimension** von $\mathcal{F}$ ist gerade $\text{fat}_0(\mathcal{F})$. Äquivalent ist mit den Begriffen aus Definition 8.21 dies gerade die maximale Länge eines Vektors, sodass

$$b_i(f(X_i) - c_i) \geq 0, \quad \text{also} \quad \text{sign}(f(X_i) - c_i) = b_i$$

erfüllt ist.

- Anschaulich: Genügt es, dass f das korrekte Vorzeichen hat, so reicht die Forderung $b_i f(X_i) \geq 0$. Im Sinne der Pseudo-Dimension soll aber der Funktionswert möglichst um $c_i \in [0, 1]$ von der Null weg sein, d.h., für $c_i = 0 \forall i$ hätten wir hier die übliche VC-Dimension
- Die fette ε -Shatterdimension verlangt, dass sogar für $c_i = 0$ der Funktionswert $f(X_i)$ mindestens ε von der Entscheidungsgrenze entfernt liegt, und möglichst soll auch noch $c_i > 0$ sein

Lemma 8.24

Für Vektorräume $\mathcal{F}$ reellwertiger Funktionen ist die Pseudo-Dimension von $\mathcal{F}$ gerade die Dimension von $\mathcal{F}$. Offensichtlich gilt immer die Ungleichung

$$\mathbf{fat}_\varepsilon(\mathcal{F}) \leq \mathbf{Pdim}(\mathcal{F}).$$

Lemma 8.25

Enthält $\mathcal{F}$ nur Indikatorfunktionen und ist $\varepsilon \leq \frac{1}{2}$, so gilt $\mathbf{VC}(\mathcal{F}) = \mathbf{fat}_\varepsilon(\mathcal{F})$.

Die Covering-Zahl lässt sich auch in Termen der fetten Shatterdimension nach oben abschätzen, analog zu den Abschätzungen der Entropie durch die VC-Dimension und der Packzahl durch Pollards Pseudo-Dimension (die VC-Dimension für reellwertige Funktionen)

Satz 8.26

Sei

$$\mathcal{F}_{\sin} := \{f : \mathbb{R} \rightarrow \mathbb{R}, f(x) := \sin(\alpha x) \mid \alpha \in \mathbb{R}\}.$$

Dann gilt:

a) Für $0 < \varepsilon < 1$ ist $\text{fat}_{\varepsilon}(\mathcal{F}_{\sin}) = \infty$.

b) Es ist $\text{fat}_1(\mathcal{F}_{\sin}) = 1$.

c) Für

$$\mathcal{F}_{\sin}^{A,M} := \{f : [0, M] \rightarrow \mathbb{R}, f(x) := \sin(\alpha x) \mid \alpha \in [-A, A]\}$$

gilt, dass

$$\text{fat}_{\varepsilon}(\mathcal{F}_{\sin}^{A,M}) \leq \left\lfloor \frac{MA}{2\varepsilon} \right\rfloor + 1$$

für $0 < \varepsilon < 1$.

Definition 8.27

Es sei $\mathcal{F}$ eine Familie von Entscheidungsregeln. Dann ist

$$DIS(\mathcal{F}) := \{x \in \mathcal{X} \mid \exists f, g \in \mathcal{F} : f(x) \neq g(x)\}$$

die **Region of Disagreement**.

Definition 8.28

Die größte Zahl s , sodass paarweise verschiedene $X_1, \dots, X_s \in \mathcal{X}$ und Entscheidungsregeln $h_0, \dots, h_s \in \mathcal{F}$ existieren, sodass

$$DIS(\{h_0, h_i\}) \cap \{X_1, \dots, X_s\} = \{X_i\} \quad \forall i = 1, \dots, s,$$

heißt **Sternendimension** von $\mathcal{F}$. Existiert keine solche Zahl, ist $s := \infty$. Die Menge $\{X_1, \dots, X_s\}$ heißt dann **Sternmenge** von $\mathcal{F}$.

- Man beachte, dass natürlich die h_i alle verschieden von h_0 sein müssen, da ansonsten offensichtlich $DIS = \emptyset$ gelten würde.
- Man kann die Sternendimension als eine Erweiterung der VC-Dimension ansehen. Hier wird nicht mehr „nur“ die Länge der größtmöglichen Stichprobe gesucht, die auf alle denkbaren Arten von einem 0/1-Klassifizierer getrennt werden kann, sondern nach der Länge einer Stichprobe gesucht, sodass jede Instanz durch die Klasse der binären Entscheidungsregeln vereinzelt werden kann.

Satz 8.29

Sei s die Sternendimension und h die VC-Dimension von $\mathcal{F}$. Dann gilt

$$h \leq s.$$

Beispiel 8.30

- a)** Sei $\mathcal{F} := \{x \mapsto f_t(x) := 2I(x \in [t, \infty)) - 1 \mid t \in \mathbb{R}\}$. Dann ist $s = 2$.
- b)** Sei $\mathcal{F} := \{x \mapsto f_{a,b}(x) := 2I(x \in [a, b]) - 1 \mid -\infty < a \leq b < \infty\}$. Dann ist $s = \infty$.
- c)** Die Klasse der linearen trennenden Hyperflächen hat Sternendimension ∞ .

Hanneke, Yang: Minimax Analysis of Active Learning, 2015

10. Boosting

- Erinnerung: CD-Algorithmus für das Lasso: Gehe zyklisch durch alle Spalten und berechne jeweils ein „Simple Lasso“, also den Lasso-Schätzer bezüglich jeweils einer einzigen Spalte, wobei nicht die originalen Y -Werte, sondern die aktuellen partiellen Residuen gefittet werden
- Boosting verfolgt eine ähnliche Idee: Berechne sogenannte Baselearner (auch „weak models“) und kombiniere diese geeignet zu einem besseren Modell
- Diese Baselearner basieren oft ebenso wie die einfachen Lasso-Modelle beim CD-Algorithmus auf einer einzigen Spalte
- Boosting kann zur empirischen Minimierung vieler verschiedener Verlustfunktion verwendet werden, und zwar sowohl für Klassifikation als auch für Regression

Forward Stepwise vs. Forward Stagewise

- LAR ist ein Spezialfall von **Forward Stepwise Regression**. Während bei LAR die aktuellen Koeffizienten im k -ten Schritt nur so weit gegen den **gemeinsamen** KQ-Koeffizienten $\hat{\beta}^{(k,LS)}$ laufen, bis eine andere Variable mehr mit dem aktuellen Residuum korreliert ist, wird bei der Forward Stepwise Regression die Aufdatierung $\hat{\beta}^{(k)} = \hat{\beta}^{(k-1)} + \hat{\beta}^{(k,LS)}$ ausgeführt (da $\hat{\beta}^{(k,LS)}$ nur auf einer Teilmenge der Prädiktoren basiert, nehmen wir mit der Schreibweise implizit schon an, dass die übrigen Komponenten Null sind)
- Bei der **Forward Stagewise Regression** wird genau so wie bei Forward Stepwise Regression die am meisten mit dem aktuellen Residuum korrelierte Variable gewählt, aber es wird **nur der zugehörige Koeffizient aufdatiert, alle anderen Koeffizienten bleiben in dem Schritt unverändert**
- FStep wird auch als Orthogonal Matching Pursuit, FStage als Matching Pursuit bezeichnet

Forward Stepwise vs. Forward Stagewise

- Im k -ten Schritt bei Forward Stagewise Regression wählen wir also Variable j_k mit

$$j_k = \operatorname{argmax}_{j=1,\dots,p} (|\operatorname{Corr}(r^{(k-1)}, X_j)|), \quad r^{(k-1)} = Y - X\hat{\beta}^{(k-1)}$$

und machen die Aufdatierung

$$\hat{\beta}_j^{(k)} = \hat{\beta}_j^{(k-1)} + \hat{\beta}^{(k,LS)}, \quad j \in \{0, j_k\}, \quad \hat{\beta}_j^{(k)} = \hat{\beta}_j^{(k-1)}, \quad j \notin \{0, j_k\},$$

wobei $j = 0$ zum Intercept gehört

- Der Intercept ist bei beiden Methoden nicht notwendig

- Sowohl bei FStage als auch bei FStep werden **aggressive Aufdatierungen** vorgenommen, da man dem neuen Koeffizienten mit Vorfaktor 1 zu dem aktuellen Koeffizienten hinzuaddiert
- Es ist allerdings so, dass Algorithmen, die vorsichtige Aufdatierungen vornehmen für den Preis, dass die Anzahl der Iterationen höher ist, besser sind
- Dies wird durch Lernraten (auch Schrittweiten genannt) ν erreicht. Ein konkreter Algorithmus ist **Incremental Forward Stagewise Regression** FS_ν : Wähle im k -ten Schritt Variable j_k wie bei FStage und mache den Aufdatierungsschritt

$$\hat{\beta}_{j_k}^{(k)} = \hat{\beta}_{j_k}^{(k-1)} + \nu \text{sign}(\text{Corr}(X_{j_k}, r^{(k-1)})).$$

- Man kann die Lernraten auch adaptiv wählen, also ν_k im k -ten Schritt. Dann kürzen wir den Algorithmus mit FS_{ν_k} ab.

L_2 –Boosting mit komponentenweisen KQ-Modellen

Initialisierung: Daten (X, Y) , Lernrate $\nu \in]0, 1]$, Anzahl m_{iter} der Iterationen und Parametervektor $\hat{\beta}^{(0)} = \mathbf{0}_{p+1}$;

Berechne $\bar{Y}$ und die Residuen $r^{(0)} := Y - \bar{Y}$;

for $k = 1, \dots, m_{iter}$ **do**

for $j = 1, \dots, p$ **do**

 Fitte das aktuelle Residuum $r^{(k-1)}$ durch ein einfaches lineares Regressionsmodell bezüglich Variable j ;

 Berechne die Residuenquadratsumme ;

end

 Nehme die Variable j_k , dessen einfaches Modell $\hat{\beta}_{j_k} \in \mathbb{R}^{p+1}$ die kleinste Residuenquadratsumme lieferte ;

 Aufdatierung $\hat{\beta}^{(k)} = \hat{\beta}^{(k-1)} + \nu \hat{\beta}_{j_k}$;

 Berechne die aktuellen Residuen $r^{(k)} = Y - X\hat{\beta}^{(k)}$;

end

Algorithm 1: Component-wise least squares Boosting

In R: Funktion `glmboost` im Paket `mboost`

Eigenschaften der Algorithmen

- FStep braucht höchstens p Schritte, um die LS-Lösung zu erreichen und LAR braucht für $p > n - 1$ höchstens $(n - 1)$ Schritte, wohingegen LARS und FStage i.A. deutlich mehr Schritte benötigen
- Das Lasso kann als Grenzfall von FS_ν für $\nu \rightarrow 0$ betrachtet werden
- Auf moderat großen Datensätzen sind Lasso und L_2 -Boosting von der Geschwindigkeit her ähnlich, ebenso Lasso, LAR und FS_ν . Auf großen Datensätzen ist L_2 -Boosting effizienter als LARS
- L_2 -Boosting und FS_ν selektieren jeweils dieselbe Variable, aber die Aufdatierung der Koeffizienten und daher auch die der Residuen ist unterschiedlich
- L_2 -Boosting kann als Spezialfall von FS_{ν_k} betrachtet werden, wenn man den Betrag des Koeffizienten in jedem Schritt von L_2 -Boosting in die Lernrate eingehen lässt

Lasso vs. L_2 –Boosting

- Lasso und L_2 –Boosting sind zwar ähnlich, jedoch gibt es einige signifikante Unterschiede
- Lasso selektiert in der Regel höchstens eine von hoch korrelierten Variablen, während L_2 –Boosting oft nach der Selektion einer dieser Variablen in späteren Iterationen auch die anderen Variablen selektiert
- Lasso kann maximal $\min(n, p + 1)$ Variablen selektieren, während L_2 –Boosting (in Abhängigkeit von der Anzahl der Iterationen) auch mehr Variablen selektieren kann
- Für das Lasso gab es Aussagen zur Variablenselektionskonsistenz, wohingegen (Vogt, 2018) gezeigt hat, dass man Fälle finden kann, in denen L_2 –Boosting zuverlässig die falschen Variablen auswählt
- L_2 –Boosting, FS_ν und FS_{ν_k} sind Spezialfälle des Problems
$$\min_r (\|X^T r\|_\infty) \quad \text{s.t. } r \in P_{res} := \{r \mid r = Y - X\beta \exists \beta \in \mathbb{R}^p\}.$$
- Lasso ist äquivalent zur Lösung des regularisierten Problems

$$\min_{r \in P_{res}} \left(\|X^T r\|_\infty + \frac{1}{2\delta} \|r - Y\|_2^2 \right)$$

für einen Regularisierungsparameter $\delta > 0$ (vgl. Freund, Grigas, Mazumder 2017)

- Beim L_2 –Boosting haben wir sowohl die Lernrate ν als auch die Anzahl der Iterationen m_{iter} anzugeben
- Die Schrittweite kann im Prinzip beliebig in $]0, 1]$ gewählt werden, jedoch sollte sie klein sein. Default ist $\nu = 0.1$ in der Implementation
- Alternativ kann man für jede Iteration die Schrittweite durch Line Search optimieren, d.h., man wählt ein Gitter von verschiedenen Werten für ν und wählt dasjenige Element, sodass die Aufdatierung bzgl. dieser Schrittweite die kleinste Residuenquadratsumme liefert. Für L_2 –Boosting ist diese Technik aber nicht notwendig

- Anstelle einer festen Anzahl an Iterationen kann auch ein Konvergenzkriterium (hinsichtlich MSE) herangezogen werden, d.h., man iteriert so lange, bis die MSE-Differenz eine Schranke unterschreitet
- Um (durch eine Wahl von m_{iter}) eher abubrechen („early stopping“), werden AIC-basierte Kriterien zugrunde gelegt. Z.B. kann man sehr lange iterieren, anschließend prüfen, wo man hätte stoppen können und letztendlich den Algorithmus noch einmal laufen lassen. Dies kann auch adaptiv realisiert werden

- Bisher haben wir lediglich den L_2 –Verlust optimiert. Boosting ist aber in der Lage, auch andere Verlustfunktionen iterativ zu minimieren
- Dazu sei L eine im zweiten Argument **konvexe, differenzierbare** Verlustfunktion, wobei auch Nicht-Differenzierbarkeit in endlich vielen Punkten zugelassen ist
- Eigentlich werten wir immer $L(Y_i, f(X_i))$ für alle $i = 1, \dots, n$ aus. Wir machen aber die funktionale Betrachtung, dass $L = L(y, f)$ ist, d.h., die Differenzierbarkeits-Bedingung führt darauf, dass wir die Verlustfunktion nach f ableiten
- Diese funktionalen Ableitungen führen auf **funktionale Gradienten**
- **(Funktionales) Gradienten-Boosting** kann als Analogon zum üblichen Gradientenverfahren angesehen werden, nur folgen wir hier iterativ dem steilsten funktionalen Gradienten (**funktionales Gradientenverfahren**)

Generisches funktionales Gradienten-Boosting

Initialisierung: Daten (X, Y) , Lernrate $\nu \in]0, 1]$, Anzahl m_{iter} der Iterationen und Offset

$$\hat{f}^{(0)}(\cdot) \equiv \operatorname{argmin}_c \left(\frac{1}{n} \sum_i L(Y_i, c) \right)$$

for $k = 1, \dots, m_{iter}$ **do**

Berechne die negativen Gradienten und werte sie an dem aktuellen Modell aus:

$$U_i = -\partial_f L(Y_i, f)|_{f=\hat{f}^{(k-1)}(X_i)}$$

für alle $i = 1, \dots, n$;

Fasse $U = (U_i)_i$ als Response auf und berechne ein einfaches Modell:

$$(X_i, U_i)_i \xrightarrow{\text{Baselearner}} \hat{g}^{(k)}(\cdot)$$

mit einem zuvor gewählten Baselearner ;
Aufdatierung

$$\hat{f}^{(k)}(\cdot) = \hat{f}^{(k-1)}(\cdot) + \nu \hat{g}^{(k)}(\cdot)$$

end

Identifikation mit L_2 –Boosting

- Als Baselearner haben wir beim L_2 –Boosting wie auf Folie 251 die einfachen KQ-Modelle. Die Spaltenselektion ist im Algorithmus auf Folie 257 nur durch $\xrightarrow{\text{Baselearner}}$ maskiert worden
- Der Offset kann als initialer Achsenabschnitt aufgefasst werden. Genau wie bei der Wahl der $\hat{c}_k$ für Regressionsbäume ist hier das arithmetische Mittel $\bar{Y}$ die richtige Wahl, was die Berechnung von $r^{(0)}$ auf Folie 251 erklärt
- Fassen wir den L_2 –Verlust als $L_2(\mathbf{y}, \mathbf{f}) = (\mathbf{y} - \mathbf{f})^2$ auf, so haben wir offensichtlich

$$U_i = -\partial_{\mathbf{f}} L_2(Y_i, \mathbf{f})_{\mathbf{f}=\hat{\mathbf{f}}^{(k-1)}(\mathbf{X}_i)} = Y_i - \mathbf{f}^{(k-1)}(\mathbf{X}_i),$$

was genau den Residuen beim L_2 –Boosting entspricht, da $\hat{\mathbf{f}}^{(k-1)}(\mathbf{X}_i) = \mathbf{X}_i \hat{\boldsymbol{\beta}}^{(k-1)}$ ist

- $\hat{\mathbf{g}}^{(k)}$ ist das simple Modell, welches wir zuvor durch den Koeffizienten $\hat{\beta}_{j_k}$ repräsentiert haben

Wichtige Boosting-Algorithmen für Regression

- In `mboost` sind sogenannte `family`-Objekte implementiert, die als Inputargument in `glmboost` oder der allgemeineren Funktion `mboost` eingegeben werden können. Diese Objekte beinhalten unter Anderem eine Verlustfunktion und einen Gradienten. Der Offset ist bei einigen Verlustfunktionen explizit darstellbar, teilweise wird er auch direkt durch `optimize` berechnet
- L_2 -Boosting in `glmboost`: Eingabe des Inputarguments `family=Gaussian()`, welches aber der Default ist
- Nebst L_2 -Boosting, welches $E[Y|X]$ nähert, kann auch wie bei der Quantilsregression ein bedingtes τ -Quantil genähert werden, indem die Check-Verlustfunktion verwendet wird $\leadsto$ `family=QuantReg(tau=...)` oder für $\tau = 0.5$ `family=Laplace()` (hier spricht man von L_1 -Boosting)
- Die robustere Huber-Verlustfunktion erhält man durch `family=Huber()`

Wichtige Boosting-Algorithmen für Klassifikation

- BinomialBoosting ist das Boosting-Analogon des Logit-Modells. Eigentlich würde man die negative binomiale Likelihood optimieren wollen, also

$$-y\ln(p) - (1 - y)\ln(1 - p),$$

jedoch ist diese erstmal nicht von der Form $L(y, f)$. Man setzt (beachte: $Y_i \in \{-1, 1\}$)

$$p = \frac{e^f}{e^f + e^{-f}} \Leftrightarrow f = \frac{1}{2} \ln \left(\frac{p}{1 - p} \right),$$

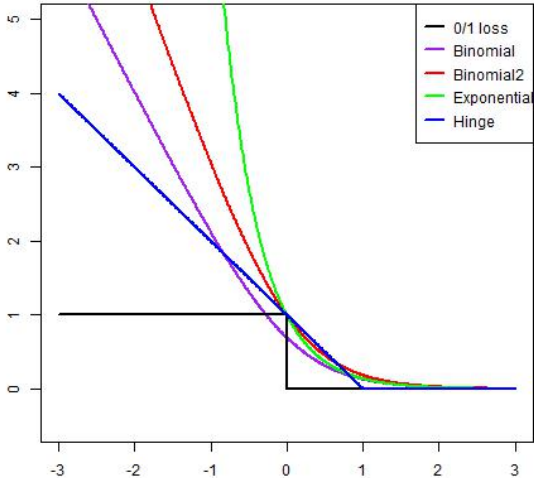
was aber zu

$$L^{\text{binom}}(y, f) = \log_2(1 + e^{-2yf})$$

umskaliert wird, da dies ein konvexer Surrogate für die Misklassifikations-Verlustfunktion ist. In **R**: `family=Binomial()`

- MultinomialBoosting generalisiert BinomialBoosting für mehrklassige Responses. Eingabe durch `family=Multinomial()`
- Weitere Varianten für Regression und Klassifikation sind verfügbar, darüber hinaus auch Methoden für Zähldaten, ordinale Daten und zensierte Daten

Surrogates für den Misklassifikationsverlust



- AdaBoost war der erste Algorithmus von Typ Boosting, folgte aber ursprünglich einer anderen Intention, nämlich iterativ einfache Modelle zu berechnen und anhand des Fehlers (hier im Sinne von Missklassifikation) Gewichte zu bestimmen, um für die folgenden Baselearner Zeilengewichte zu vergeben und am Ende die einfachen Modelle geeignet zu aggregieren
- Man konnte später zeigen, dass AdaBoost tatsächlich ein Gradienten-Boosting mit der Verlustfunktion $L(y, f) = \exp(-yf)$ ist. In R: `family=AdaExp()`

Initialisierung: Daten (X, Y) , Anzahl m_{iter} der Iterationen;

Setze $w_i^{(0)} = n^{-1}$ für alle $i = 1, \dots, n$;

for $k = 1, \dots, m_{iter}$ **do**

 Berechne einen Klassifizierer $\hat{g}^{(k)}$ basierend auf den $w_i^{(k-1)}$;

 Berechne

$$err^{(k)} := \frac{\sum_i w_i^{(k-1)} I(Y \neq \hat{g}^{(k)}(X_i))}{\sum_i w_i^{(k-1)}}, \quad \alpha^{(k)} := \ln \left(\frac{1 - err^{(k)}}{err^{(k)}} \right)$$

 und datiere die Gewichte auf durch

$$\tilde{w}_i := w_i^{(k-1)} \exp(\alpha^{(k)} I(Y_i \neq \hat{g}^{(k)}(X_i))), \quad w_i^{(k)} = \frac{\tilde{w}_i}{\sum_i \tilde{w}_i}$$

end

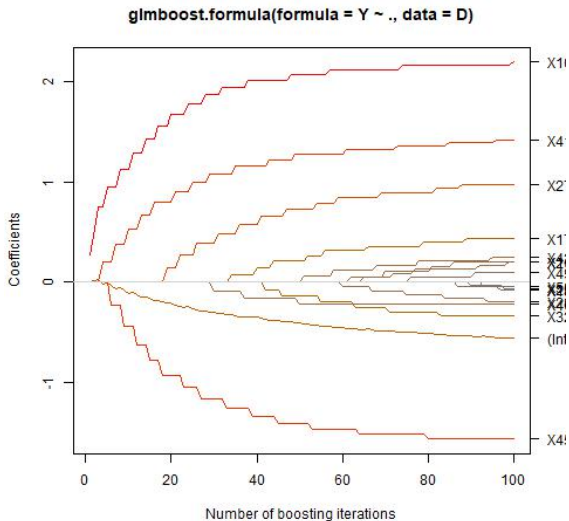
Ein neuer Punkt X^{neu} wird anhand der majority vote klassifiziert, d.h.,

$$\hat{Y}^{neu} = \text{sign} \left(\sum_{k=1}^{m_{iter}} w^{(k)} \hat{g}^{(k)}(X^{neu}) \right).$$

Funktionsweise von `glmboost`

- `glmboost` bedeutet, dass lineare Baselearner verwendet werden (`bo1s`)
- Einzugeben sind mindestens entweder ein `formula`-Argument und der Datensatz oder die Modellmatrix und der Response-Vektor
- Ferner lassen sich durch `weights` Zeilengewichte vergeben und durch `family` werden der Offset, die Verlustfunktion und der Gradient (als Funktion von $\mathbf{Y}$, $\hat{\mathbf{Y}}$ und dem Gewichtsvektor $\mathbf{w}$) übergeben
- Um die Schrittweite ν oder die Anzahl der Iterationen einzustellen, muss zunächst `control=boost_control()` als Inputargument eingegeben werden. In den Klammern kann dann die Lernrate durch Eingabe eines Wertes für `nu` oder die Anzahl der Iterationen durch Eingabe von `mstop` verändert werden
- Durch Anwendung von `plot` auf das `glmboost`-Objekt erhält man Koeffizientenpfade
- Der Output beinhaltet u.A. die Koeffizienten und den Offset (der eigentlich zum Intercept zählt)
- Durch `attributes(varimp(...))$selfreqs` (in die Klammern gehört das `glmboost`-Objekt) werden die Selektionshäufigkeiten der Variablen ausgegeben

Koeffizientenpfade



- Bisher: In `glmboost` hatten wir lineare Baselearner, also einfache lineare Regressionsmodelle.
- Für andere Baselearner brauchen wir `mboost` oder `gamboost`. Letzteres ist für generalisierte additive Modelle (GAMs), d.h.,

$$g(\mu) = \beta_0 + \sum_{j=1}^p f_j(X),$$

wobei g wieder eine Linkfunktion ist und f_j sind geeignete Funktionen (beim GLM hatten wir $f_j(X) = X_{\cdot j} \beta_j$)

- In `mboost` und `gamboost` können wir ebenso ausgewählte Variablen mit linearen Baselearnern versehen, indem wir `bo1s(...)` aufrufen. Zusätzliche Argumente sind möglich, z.B. `lambda` für eine Ridge-Penalisierung oder `intercept`, wobei `TRUE` bedeutet, dass jeder Baselearner einen Intercept berechnet (d.h., in `glmboost` hätten wir hier immer `FALSE`)
- In `bo1s` sind auch gruppenweise Updates möglich, indem man in `bo1s` mehrere Variablen eingibt. Zudem sind variierende Koeffizienten, mit Hilfe des Arguments `by` möglich. Ein variierender Koeffizient einer Variablen v bzgl. w bedeutet, dass v den Koeffizienten β_w erhält.

- `bbs` verwendet als Baselearner P-Splines, die eine regularisierte Form von B-Splines darstellen. Es können manuell u.A. noch die Anzahl der Knoten pro Variable (`knots`) und der Grad jedes Splines (`degree`) modifiziert und zyklische Splines durch `cyclic=T` (durch Eingabe von `boundary.knots`, die aber den Wertebereich der Variablen abdecken müssen), sowie variierende Koeffizienten ermöglicht werden
- Stümpfe (also Bäume, die nur einen Split vollziehen) als Baselearner sind verfügbar durch `btrees`. Für solche Baselearner empfiehlt sich aber, die Funktion `blackboost` im gleichen Paket zu verwenden
- Auch möglich für Bäume als Baselearner ist die Verwendung der Funktion `gbm` im gleichnamigen Paket
- Weitere Baselearner sind u.A. zufällige Effekte (`brandom`), bivariate P-Splines (`bspatial`) und radiale Basisfunktionen (`brad`). Es können auch verschiedene Baselearner für verschiedene Prädiktoren gewählt werden
- Eine sehr gute Übersicht: (Hofner et. al.: Model-based Boosting in R, 2014)

- Ein wichtiger Baselearner für L_2 -Boosting (Bühlmann, Yu, 2003) sind Smoothing Splines, d.h., kubische Splines, die die Lösung von

$$\hat{f}_j(\mathbf{X}) = \operatorname{argmin}_f \left(\sum_{i=1}^n (Y_i - f_j(X_{i,j}))^2 + \lambda \int (f''(x))^2 dx \right)$$

darstellen

- Der Strafterm bestraft letztendlich die Krümmung der Funktion, soll also erreichen, dass die Funktion nicht allzu stark gekrümmt ist, also nicht zu „wild“ verläuft
- Da die Spur der resultierenden Hutmatrix den Freiheitsgraden entspricht, sind die Freiheitsgrade und der Strafparameter gekoppelt
- In `btrees` kann man daher entweder `df` oder `lambda` eingeben. Empfohlen wird eine geringe Anzahl von Freiheitsgraden (Default ist 4), da dann eine geringe Varianz gegeben ist, wenngleich ein hoher Bias, der aber durch die Boosting-Iterationen schrittweise verringert wird

11. Ensemble-Methoden

- Kapitel 2-6: Klassische Verfahren, die den gesamten Trainingsdatensatz nutzen und alle Spalten verwenden
- Nachteil: Overfitting, fehlende Interpretierbarkeit, Versagen oder hohe computationelle Kosten für $p > n$
- Kapitel 7,10: Regularisierte Verfahren, die eine Spaltenauswahl ermöglichen
- Führt zu Sparsity, besserer Qualität der Modelle auf Testdaten und stellte die Interpretierbarkeit der Modelle wieder her. Boosting gab zudem Einblicke in die Wichtigkeit der Variablen
- Kapitel 9: Zeilenauswahl (Trägervektoren) und ggf. Spaltenselektion bei anderen Straftermen als $\|f\|_{\mathcal{H}_K}^2$
- Kapitel 8: Splits werden an einer Variablen vollzogen, daher kann die Wichtigkeit von Variablen quantifiziert werden, aber Bäume allein sind schlecht interpretierbar hinsichtlich des Einflusses der einzelnen Variablen, sehr instabil und i.A. nicht geeignet für wahre lineare Zusammenhänge

- Stochastik: Für i.i.d. ZVen $X_1, \dots, X_B$ mit $E[X_1] = \mu$ und $Var(X_1) = \sigma^2$ ist

$$E[\bar{X}_B] = \mu, \quad Var(\bar{X}_B) = \frac{\sigma^2}{B}.$$

- Können wir das auch für Lernalgorithmen nutzen? Wie bekommen wir aus einem Datensatz mehrere Modelle durch den gleichen Lernalgorithmus?
- Kreuzvalidierung: Wir haben durch Subsampling (alternativ auch Bootstrapping) aus einem Datensatz Trainings- und Testdatensätze erzeugt, um den Testfehler approximieren zu können
- Warum sollte dieses Vorgehen auf eine Aufteilung des ganzen Datensatzes beschränkt sein?
- Idee: Ziehe auch aus dem Trainingsdatensatz geeignet neue Trainingsdatensätze und berechne auf jedem ein Modell

- Die obige Strategie, bei der aus dem Trainingsdatensatz neue Trainingsdatensätze gleicher Größe durch Bootstrap gezogen werden, nennt sich **Bagging** (Akronym für „Bootstrap aggregating“)
- Wir haben also auf jedem Trainingsdatensatz $D^{(b, \text{train})}$, $b = 1, \dots, B$, ein Modell $\hat{f}^{(b)}$ berechnet. Bei Klassifikation folgen wir wieder der majority vote, bei der Regression ist unsere Vorhersage für eine neue Beobachtung X^{neu}

$$\hat{Y}^{\text{neu}} = \frac{1}{B} \sum_b \hat{f}^{(b)}(X^{\text{neu}}).$$

- Der erwartete Bias sollte bei allen Modellen gering sein, da dieser lediglich gemittelt wird. Daher ist Bagging besonders geeignet für Modelle mit geringem Bias, aber hoher Varianz, also insbesondere für (große) Bäume
- Es gilt:

$$E_D[E_f[(Y - f_D(X))^2]] \geq E_F[(Y - E_b[f_{D^{(b)}}(X)])^2],$$

wenn f_D meint, dass das Modell auf dem Datensatz D basiert (das Superskript „train“ haben wir hier unterdrückt)

Varianten des Bagging

- Zieht man keine Bootstrap-Stichproben, sondern Subsamples (also ohne Zurücklegen und mit kleinerer Zeilenzahl), so spricht man von Subbagging
- Jackknifing ist eine spezielle Form des Subsampling, bei der immer eine Beobachtung weggelassen wird
- Speziellere Varianten: Arcing (adaptive resampling classifying; Breiman 1996), wo Gewichte adaptiv gemäß der Performance des vorangegangenen Klassifizierers gewählt werden oder Magging (Meinshausen, Bühlmann, 2015), wo für inhomogene Daten die Gewichte optimal gewählt werden

Stabilisierung der Prädiktorwahl

- Bagging in der ursprünglichen Form dient der Reduzierung der Varianz von Schätzern
- Sobald Modellselektion nötig ist, können wir analog zum Bagging und Subbagging mit Hilfe von Bootstrap- oder Unterstichproben dafür sorgen, dass die selektierte Menge an Prädiktoren **stabil** in dem Sinne ist, dass bei einer Anwendung des Modellwahl-Algorithmus auf Beobachtungen aus der gleichen Verteilung wie die gesehenen möglichst die gleichen Variablen auswählen würden
- Für Lasso- oder Boosting-Modelle nennt sich dieses Vorgehen **Stability Selection** (Bühlmann, Meinshausen, 2010 und Hofner et. al., 2014)
- Bei Bäumen wird noch geschickter verfahren, als einfach nur einen Wald zu berechnen und diesen zu aggregieren. Genauer wird noch eine Zufälligkeit eingebaut, sodass das Vorgehen dann **Random Forest** heißt (Breiman 2001)

Stability Selection für Lasso-Modelle

- Bei Lasso-Modellen wird die Komplexität durch den Strafparameter λ gesteuert. Daher bezeichnen wir die bzgl. λ selektierte Indexmenge der Prädiktoren als $\hat{S}(\lambda)$
- Da λ durch Kreuzvalidierung bestimmt wird, benötigen wir ein Gitter $\Lambda \subset \mathbb{R}_{\geq 0}$, was auf die Familie $\{\hat{S}(\lambda) \mid \lambda \in \Lambda\}$ führt
- Wir wollen nun die Wahrscheinlichkeit

$$\Pi_j(\lambda) := P(j \in \hat{S}(\lambda))$$

nähern, was natürlich durch einen einzigen Lasso-Durchlauf nicht möglich ist

- Stattdessen ziehen wir B (Default 100) Subsamples auf den Trainingsdaten mit $\lfloor n/2 \rfloor$ Zeilen und berechnen jeweils die Lasso-Modelle. Diese liefern uns dann für jedes λ eine Approximation $\hat{\Pi}_j(\lambda)$.
- Das stabile Modell ist dann

$$\hat{S}^{stable} := \{j \mid \max_{\lambda}(\hat{\Pi}_j(\lambda)) \geq \pi_{thr}\}$$

für einen zuvor gewählten Cutoff π_{thr}

Stability Selection für Lasso-Modelle

- π_{thr} wird hier zwischen]0.6, 0.9[gewählt. Laut (Bühlmann, Meinshausen, 2010) hat die konkrete Wahl wenig Einfluss auf das finale Modell
- Als Analogon zu Regularisierungspfaden, die die Entwicklung der Koeffizienten in Abhängigkeit von λ visualisieren, stellen **Stabilitätspfade** die Entwicklung der Selektionshäufigkeiten für jede Variable in Abhängigkeit von λ dar
- In dem Sinne wird also jede Variable in das stabile Modell aufgenommen, deren Stabilitätspfad irgendwo den Cutoff überschritten hat
- Zwar könnte man auch durch einen höheren Strafparameter ein Modell erhalten, das genau so groß ist wie dasjenige, das die Stability Selection ausgibt, jedoch bietet das Subsampling noch den weiteren Vorteil, dass dadurch auch Modelle erfasst werden können, die ein einfaches Lasso mit dem Gitter Λ nie sieht

Stability Selection für Lasso-Modelle

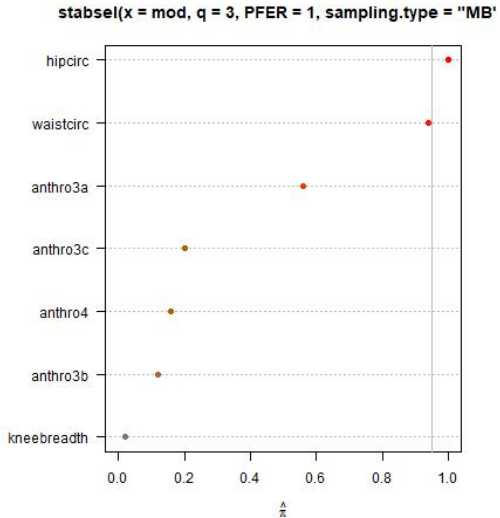
- Die erwartete Anzahl der false positives (auch bekannt als per-family error rate; PFER) lässt sich nach oben in Abhängigkeit von der Anzahl q der in $\hat{S}^{stab}$ enthaltenen Variablen und π_{thr} beschränken
- Es gibt noch eine weitere Variante der Stability Selection: Partitioniere die Daten in zwei disjunkte Teile mit $\lfloor n/2 \rfloor$ Zeilen und nähere die Wahrscheinlichkeit $\Pi_j^{simult,\lambda}$, dass Variable j gleichzeitig vom Lasso bzgl. beider Unterdatensätze gewählt wurde (s. auch Shah, Samworth, 2013)
- Implementiert ist die Stability Selection im R-Paket `stabs` (später)

Stability Selection für Boosting-Modelle

- Sehr ähnliches Vorgehen: Erzeuge wieder B Unterstichproben mit $\lfloor n/2 \rfloor$ Beobachtungen
- Auf jeder wird ein Boosting-Modell berechnet, aber nicht mit einer festen Iterationszahl, sondern nur solange, bis q Variablen selektiert wurden (d.h., die minimale Iterationszahl ist q)
- Die durch Boosting ausgegebenen Selektionshäufigkeiten werden nicht verwendet, sondern es wird wie bei der Stability Selection für Lasso-Modelle nachgesehen, in wie vielen der B Boosting-Modelle eine Variable j aufgetaucht ist
- Durch Vorgabe eines Cutoffs (laut Hofner in $]0.5, 1[$ zu wählen) erfolgt die Bestimmung der stabilen Prädiktormenge analog
- Stabilitätspfade sind definiert als Selektionshäufigkeiten jeder Variable in Abhängigkeit von der Iterationszahl m (beim Zeichnen der Stabilitätspfade ist also q hinfällig)
- Implementiert in `stabs` als Funktion `stabse1`, die auch verfügbar ist im Paket `mboost`

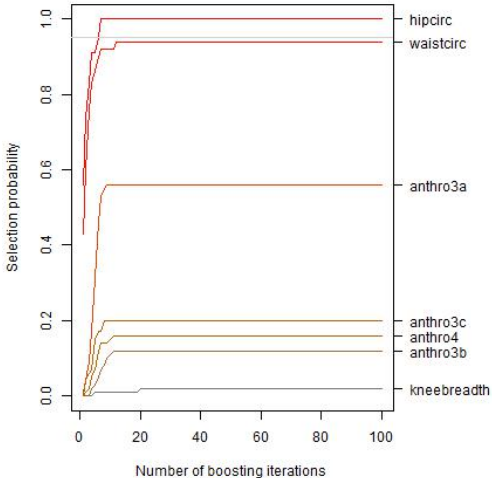
- In der Funktion `stabsel` müssen mindestens drei Argumente eingegeben werden: Ein zuvor berechnetes Boosting-Modell oder ein einfaches Lasso-Modell (ein `glmnet`-Objekt, kein `cv.glmnet`-Objekt!) sowie zwei der drei Argumente `q`, `PFER` und `cutoff`
- Da `PFER`, `q` und π_{thr} miteinander gekoppelt sind, wird das verbleibende Argument automatisch berechnet
- Mit `sampling.type='MB'` erhalten wir das „Meinshausen-Bühlmann-Ziehungsprinzip“ (ist Default)
- Anwendung von `plot` auf das `stabsel`-Objekt ergibt einen Stability-Plot, in dem der Cutoff sowie die aggregierten Wichtigkeiten jeder Variablen geplottet werden. Die Stabilitätspfade werden geplottet, wenn `type='paths'` eingegeben wird
- Vorsicht: Es gibt für `q` und `PFER` keine Empfehlungen. Insbesondere bei großen Datensätzen mit geringer SNR kommt es häufig vor, dass gar keine Variable selektiert wird

Stabilitäts-Plot; Boosting



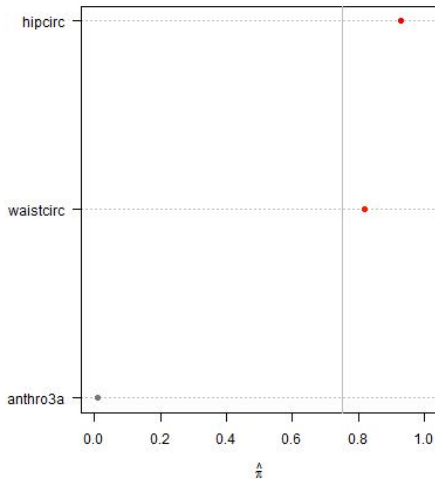
Stabilitäts-Pfade; Boosting

`stabsel(x = mod, q = 3, PFER = 1, sampling.type = "MB")`



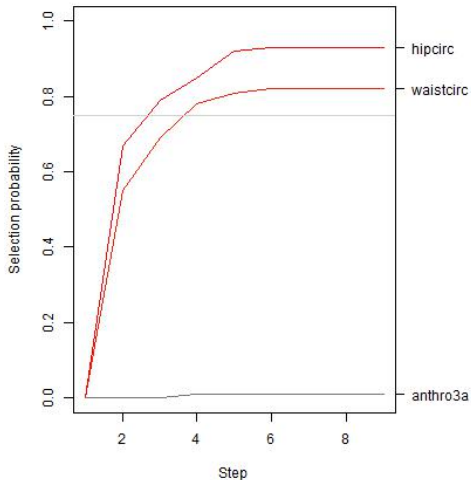
Stabilitäts-Plot; Lasso

`stabsel(x = x, y = y, fitfun = ..1, cutoff = 0.75, PFER = 1)`



Stabilitäts-Pfade; Lasso

`stabel(x = x, y = y, fitfun = ..1, cutoff = 0.75, PFER = 1)`



- Eine naive Erweiterung von Bäumen im Sinne der obigen Stability Selection wäre, Bootstrap- oder Unterstichproben zu ziehen und auf jeder einen Baum zu berechnen
- Das gibt es auch und nennt sich „Bagged Trees“
- Jeder Baum erhält etwa zwei Drittel der Bootstrap-Stichproben für das Training und das andere Drittel dient als Validierungsdatensatz, um den Out-of-Bag-Fehler zu bestimmen
- Eine neue Vorhersage folgt dann entweder der majority vote aller Bäume für Klassifikation oder ist das arithmetische Mittel der Vorhersage aller Bäume für Regression
- Nachteil: Sehr schwierig zu interpretieren, nach wie vor hohe computationelle Zeit bei großem p , Bäume sind korreliert

- Seien $X_1, \dots, X_n$ identisch verteilt mit $E[X_1] = \mu$ und $\text{Var}(X_i) = \sigma^2 > 0$, aber korreliert, und zwar so, dass $\text{Corr}(X_i, X_j) = \rho > 0$ für alle $i \neq j$. Dann ist

$$E[\bar{X}_n] = \mu, \quad \text{Var}(\bar{X}_n) = \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2.$$

- Idee: Versuche, die Korrelation der Bäume zu verringern, indem für jeden Split nur $m < p$ Variablen zufällig gezogen werden und dann die beste Split-Variable in der Menge dieser m Variablen gesucht wird. Daher wird dieses Verfahren **Random Forests** genannt
- Anschaulich: Wenn in den p Prädiktoren wenige sehr gute Prädiktoren enthalten sind, so wählt jeder der gebaggten Bäume genau diese Variablen aus, sodass alle gebaggten Bäume sehr ähnlich sein werden und somit hochkorreliert sind, also die Varianzreduktion durch Bagging nicht mehr signifikant wäre. Durch die De-Korrelation der Bäume wird die Stärke des Bagging wesentlich besser ausgenutzt

- Gebaggte Bäume sind der Spezialfall für $m = p$. In der Regel wählt man bei Random Forests $m \approx \sqrt{p}$ für Klassifikation und $m \approx p/3$ für Regression
- Bestimmung des Out-of-Sample-Fehlers für eine Instanz (X_i, Y_i) , indem alle Bäume, bei denen diese Instanz nicht in den entsprechenden Bootstrap-Stichproben enthalten war
- Außerdem kann so die Wichtigkeit der Prädiktoren quantifiziert werden: Für Regressionsprobleme wird bei jedem Split berechnet, wie viel die letztendlich gewählte Variable hinsichtlich der Verringerung der Residuenquadratsumme bringt, bei Klassifikation wird die Verbesserung hinsichtlich des Gini-Index berechnet. Die beste Variable bzgl. des gebaggten Bäume ist am Ende diejenige, bei der die kumulierte Verringerung in all ihren Splits am höchsten war
- Es gibt sogenannte Importance-Plots, die genau diese Wichtigkeit auftragen, und zwar so standardisiert, dass für jede Variable die Verringerung des Verlustkriteriums anteilig an der bzgl. der besten Variable berechnet wird
- In R: Paket `randomForest`

- RandomForests ziehen zufällig aus allen Spalten, d.h., bei gruppierten oder blockweisen Daten wird diese Struktur gar nicht berücksichtigt
- Insbesondere problematisch bei Datensätzen zu Patienten, bei denen Blöcke von Informationen zu verschiedenen medizinischen/biologischen Teilgebieten vorliegen („multi omics data“)
- Idee: Ziehe zunächst zufällig Blöcke, per Default mit einer Wahrscheinlichkeit von 0.5 pro Block, und ziehe dann zufällig Variablen aus den gezogenen Blöcken. Ist die Größe der Blöcke jeweils p_m , so werden $\sqrt{p_m}$ Variablen gezogen
- Um eventuelle Nachteile kleinerer Blöcke gegenüber großen zu verringern, können noch Split-Gewichte vergeben werden, d.h., die Verbesserung des Verlustkriteriums wird zunächst mit Gewichten multipliziert und anschließend die beste Split-Variable bestimmt
- Implementiert in `blockForest` (s. auch Hornung, Wright, 2018)

- Problem beim Lasso: Bei hochkorrelierten Variablen wählt das Lasso in der Regel höchstens eine dieser Variablen aus, was bei Daten aus genetischen Anwendungen („microarray data“) manchmal nicht angemessen ist. Außerdem ist bei solchen Daten n so klein und p so groß, dass die Restriktion, dass das Lasso dann höchstens n Variablen auswählt, ebenfalls zu stark ist
- Das Random Lasso (Wang, 2011) erzeugt in der ersten Stufe B Bootstrap-Stichproben und damit Koeffizienten $\hat{\beta}^{(b,init)}$ berechnet. Anschließend wird die **aggregierte Wichtigkeit jeder Variable** bestimmt durch

$$\hat{v}(\{j\}) := \frac{1}{B} \left| \sum_b \hat{\beta}^{(b,init)} \right|.$$

- In der zweiten Stufe werden wieder B Bootstrap-Stichproben gezogen. Gemäß dem **Maß** $\hat{v}$ werden anschließend zufällig Variablen gezogen und auf diesen wieder ein Lasso berechnet. Die Koeffizienten der B Lassos aus der zweiten Stufe werden gemittelt, um die finalen Koeffizienten zu erhalten

12. Ranking

Die Position von Ranking

■ Ranking vs. Klassifikation:

- ▶ Klassifikation würde nur zwischen Klasse -1 und Klasse 1 unterscheiden
- ▶ Ranking versucht, eine Ordnung (ggf. nur für die Instanzen in Klasse 1) zu finden
- ▶ $\rightsquigarrow$ Ranking-Probleme sind komplizierter, aber informativer als Klassifikationsprobleme

■ Ranking vs. Regression:

- ▶ Das Ziel der Regression ist die korrekte Vorhersage der Y_i
- ▶ Für das Ranking müssen wir die Y_i nicht richtig vorhersagen, sondern nur ihre Ränge
- ▶ $\rightsquigarrow$ Ranking-Probleme sind nicht so strikt wie Regressionsprobleme, aber auch weniger informativ

Arten von Ranking-Problemen

■ Systematisiert nach ihrem Ziel:

- ▶ Lediglich die K wichtigsten Instanzen zu finden, sind schwache Rankingprobleme
- ▶ Das harte Rankingproblem versucht, die Ränge aller Instanzen richtig vorherzusagen
- ▶ Lokalisierte Ranking-Probleme sind ein Kompromiss mit dem Ziel, die richtigen K Instanzen zu finden und nur das Ranking dort richtig vorherzusagen

■ Systematisiert nach den Response-Werten:

- ▶ Y binär $\rightsquigarrow$ binäres Rankingproblem (bipartite ranking problem)
- ▶ Y kann d verschiedene Werte annehmen $\rightsquigarrow d$ -partite ranking problem
- ▶ Y ist stetig $\rightsquigarrow$ stetiges Rankingproblem

- Betrugsdetektion (hier: Risk-based auditing): Welche Steuererklärungen sind die auffälligsten, d.h., wo ist Betrug am ehesten zu erwarten?
- Alternativ: Für welche Steuererklärungen wäre der erwartete Schaden am größten?
- Dokumentenranking (z.B. Suchmaschinen): Welche Dokumente sind am relevantesten?
- Medizin: Was für Patienten leidet am ehesten unter einer bestimmten Krankheit?

- Eine Funktion $r : \mathcal{X} \times \mathcal{X} \rightarrow \{-1, 1\}$ so, dass $r(x, x') = 1$ bedeutet, dass x besser ist als x' , heißt **Ranking-Regel** (Clémenton et. al., 2008)
- Eine Scoring-Funktion $s : \mathcal{X} \rightarrow \mathbb{R}$ ist mit einer Ranking-Regel durch die Äquivalenz

$$r(x, x') = 1 \Leftrightarrow s(x) - s(x') \geq 0$$

verbunden

- Die Scoring-Regel muss so bestimmt werden, dass der entsprechende Ranking-Verlust für das konkrete Problem minimiert wird

- Harter Ranking-Verlust:

$$L_n^{\text{hard}}(s) := \frac{1}{n(n-1)} \sum \sum_{i \neq j} I((Y_i - Y_j)(s(X_i) - s(X_j)) < 0)$$

- Weicher Ranking-Verlust:

$$L_n^{\text{weak}, K}(s) = \frac{1}{n} \sum_{i=1}^n I(\tilde{Y}_i^{(K)}(s(X_i) - \hat{Q}(s, 1 - u^{(K)})) < 0)$$

für das empirische Quantil $\hat{Q}(s, 1 - u^{(K)})$ mit $u^{(K)} = K/n$ und $\tilde{Y}_i := 2I(\text{rk}(Y_i) \leq K) - 1$, wenn die Ränge absteigend sind

- Da jeder Fehler in dem Sinne, dass eine der tatsächlich besten K Instanzen nicht erkannt wird, automatisch dazu führen muss, dass stattdessen eine der anderen $n - K$ Instanzen als eine der besten vorhergesagt wird, genügt es, lediglich den Verlust für K Instanzen zu berechnen
- Der weiche Ranking-Verlust kann also auch geschrieben werden als

$$L_n^{\text{weak}, K}(s) = \frac{2}{n} \sum_{i \in \text{Best}_K} I(Y_i(s(X_i) - \hat{Q}(s, 1 - u^{(K)})) < 0),$$

wobei Best_K die Menge der tatsächlichen K besten Instanzen sei.

- Lokalisierter Ranking-Verlust:

$$L_n^{loc,K}(s) := \frac{n_-}{n} L_n^{weak,K}(s) + \frac{1}{n(n-1)} \sum_{i \neq j} I((s(X_i) - s(X_j))(Y_i - Y_j) < 0, \min(s(X_i), s(X_j)) \geq \hat{Q}(s, 1 - u^{(K)}))$$

- Interpretation: Wir wollen erreichen, dass wir die K besten Instanzen finden und zudem diese auch noch richtig ranken. n_- sei dabei als die Menge der schlechten Instanzen, also $n_- = n - K$, zu verstehen.
- Wenn wir die richtigen K besten Instanzen finden, aber deren Ranking nicht korrekt ist, ist der erste Summand des lokalisierten Rankingverlustes Null, der andere nicht (letzterer ist maximal $K(K-1)/n/(n-1)$)
- Sagen wir K völlig falsche Instanzen als die besten voraus, ranken diese aber untereinander alle richtig, so ist der zweite Summand Null, der erste aber nicht (ersterer ist $2K(n-K)/n^2$)

- Sei

$$\mathbf{Perm}(1 : n) := \{\pi \mid \pi \text{ ist eine Permutation von } \{1, \dots, n\}\}$$

und seien $\pi, \hat{\pi} \in \mathbf{Perm}(1 : n)$ die wahre bzw. eine geschätzte Permutation. Dann kann der harte Ranking-Verlust äquivalent geschrieben werden als

$$L_n^{\text{hard}}(\pi, \hat{\pi}) = \frac{2}{n(n-1)} \sum_{i < j} I((\pi_i - \pi_j)(\hat{\pi}_i - \hat{\pi}_j) < 0).$$

- Wie in der üblichen Schreibweise ist auch das eine spezielle U-Statistik der Ordnung 2
- Man kann auch anstatt des paarweisen Vergleichs m Instanzen gleichzeitig vergleichen, was auf eine U-Statistik der Ordnung m führen würde. Laut (Cléménçon et. al., 2008) ist es aber einfacher, paarweise Vergleiche auszuführen, was auch mehr Anwendungen hat
- Das harte Rankingproblem ist ein globales Problem

- (Clémençon et. al., 2008) haben Resultate hergeleitet, in denen sie für empirische Minimierer

$$\hat{r}_n := \operatorname{argmin}_{r \in \mathcal{R}} (L_n^{\text{hard}}(r))$$

das Risiko $L(\hat{r}_n) = P((Y - Y')\hat{r}_n(X, X') < 0 | D_n)$ studieren, wobei D_n die Daten mit n Zeilen seien

- Dabei verwenden sie Lemma 0.6, um das Exzess-Risiko nach oben abzuschätzen und können die rechte Seite durch Symmetrisierung und McDiarmids Ungleichung (Satz 4.4) durch Rademacher-Mittel beschränken
- Durch existierende Resultate zur Relation von Rademacher-Mitteln und der VC-Dimension wird das Exzess-Risiko am Ende in Termen der VC-Dimension der Klasse $\mathcal{R}$ der Ranking-Regeln beschränkt
- Es werden unter Zusatzannahmen noch bessere Raten bewiesen durch Techniken der Lerntheorie, die über diese Vorlesung hinausgehen (Haussler-Theorem, Rademacher-Chaos, Theorie von U-Prozessen...)

Ranking vs. ordinale Regression

- Ordinale Regression ist eigentlich Klassifikation, aber mit geordneten Klassen, d.h., o.E. seien die Klassen $1, 2, \dots, K$ mit $1 \leq 2 \leq \dots$
- Bei der ordinalen Regression und auch beim Logit-Modell (letzteres hat nicht notwendigerweise geordnete Klassen, aber durch „ $\neg 0 = 1$ “ und „ $\neg 1 = 0$ “ eine besondere Struktur) wird eine reellwertige Scoring-Funktion s berechnet und die Klassifikation gemäß geeigneter Thresholds für die Scores vorgenommen
- Insbesondere wird durch die Scores also ein Ranking der Instanzen induziert
- Unterschied: Bei Rankingproblemen ist es das Ziel, dass die Ordnung der Scores möglichst korrekt ist, was bei ordinaler Regression nicht notwendig ist, da wir **andere Verluste** (Klassifikationsverluste) haben. Daher ist es bei der ordinalen Regression schon ausreichend, wenn wir die Scores so berechnet haben, dass wir geeignete Thresholds finden können, um perfekt zu klassifizieren

Rankingprobleme mit diskreter Response

- Bereits in Kapitel 5 angesprochen: Binäre Rankingprobleme lassen sich als gewichtete binäre Klassifikation auffassen, wodurch sich anstatt der Minimierung einer Ranking-Verlustfunktion eher eine AUC-Maximierung anbietet
- Analog: Rankingprobleme mit Responses mit d verschiedenen Werten können durch VUS-Maximierung gelöst werden
- Für beide Arten von Rankingproblemen gibt es bereits viele verschiedene Ansätze, die im Folgenden vorgestellt werden
- Klassische Modelle (Plackett-Luce, Mallows) berechnen eine Wahrscheinlichkeit auf der Menge $\text{Perm}(1 : n)$ der Permutationen (hier nicht)

Rankingprobleme mit diskreter Response:

Plug-in-Ansatz

- Idee: Wir berechnen bei der Klassifikation durch z.B. Logit- oder Boosting-Modelle bereits eine Scoring-Funktion und damit ein Ranking
- Standardisiert auf $[0, 1]$ approximieren wir also die Wahrscheinlichkeit $\eta(\mathbf{x}) = P(Y = 1 | X = \mathbf{x})$
- Beim **Plug-in-Ansatz** wird diese Scoring-Funktion für binäre Ranking-Probleme verwendet
- Genauer wird dabei η als zu schätzende Funktion betrachtet, d.h., implizit kann man den Plug-in-Ansatz als einen Regressionsansatz verstehen, bei dem man $\hat{\eta}$ schätzt
- Nachteil: Laut Clémenton nicht gut für hochdimensionale Daten. Außerdem wird dadurch nur eine L_1 -Optimierung der ROC-Kurve erreicht, d.h.,

$$\int_0^1 |ROC^*(\alpha) - ROC_s(\alpha)| d\alpha \rightarrow 0$$

für die optimale ROC-Kurve ROC^* und die von der Scoring-Funktion s induziert ROC-Kurve ROC_s

Rankingprobleme mit diskreter Response:

Ranking-SVM

- Idee: Fasse für jede Kombination $(i, j), i \neq j$, das Ranking der Instanzen (X_i, Y_i) und (X_j, Y_j) als binäres Klassifikationsproblem auf, d.h., $Y_i = 1$ und $Y_j = 0$, falls X_i höher gerankt ist als X_j und umgekehrt, und löse diese Subprobleme durch eine SVM
- Dies kann als eine einzige Zielfunktion in Termen des Koeffizientenvektors und einer Doppelsumme an Slack-Variablen geschrieben werden, inklusive $n(n - 1)$ Nebenbedingungen (und den Nebenbedingungen, dass die Slack-Variablen alle nicht-negativ sein sollen) (s. Herbrich et. al., 1999; Joachims, 2002)
- Diese kann gemäß Bemerkung 8.6 nach (Clémentçon, 2013) geschrieben werden als

$$\frac{1}{n(n-1)} \sum_{i \neq j} L^{\text{hinge}}(Y_i - Y_j, f(X_i) - f(X_j)) + \lambda \|f\|_{\mathcal{H}_K}^2,$$

wobei $\mathcal{H}_K$ wieder ein RKHS bzgl. Kern K sei

Rankingprobleme mit diskreter Response: RankBoost

p-Norm-Push

- Idee: Verwende im Sinne von AdaBoost einen exponentiellen Surrogate-Verlust (s. Freund et. al., 2003)
- Minimiere also die Zielfunktion

$$\frac{1}{n(n-1)} \sum_{i \neq j} \exp((Y_i - Y_j)(f(X_i) - f(X_j)))$$

- Dies wird aber nicht durch ein Gradienten-Boosting gelöst, sondern wie bei AdaBoost durch iterative Ungewichtung der Beobachtungen
- Es wurde in (Rudin, Shapire, 2009) gezeigt, dass AdaBoost und RankBoost unter schwachen Voraussetzungen bereits äquivalent sind
- Erweiterung: p-Norm-Push (Rudin, 2009), der

$$\frac{1}{n(n-1)} \sum_{i \neq j} (\exp((Y_i - Y_j)(f(X_i) - f(X_j))))^p$$

für $1 \leq p < \infty$ optimiert

- Für höhere p soll dadurch mehr Gewicht auf die besten Instanzen gelegt werden

Rankingprobleme mit diskreter Response: Weitere Ansätze

- Idee: Versuche, die Differenzen der Y_i durch die Differenzen der X_i zu erklären
- Das führt auf den Algorithmus **RankRLS**, der die Verlustfunktion

$$\frac{1}{n(n-1)} \sum_{i \neq j} ((Y_i - Y_j) - (f(X_i) - f(X_j)))^2 + \lambda \|f\|_{\mathcal{H}_K}$$

optimiert (Pahikkala et. al, 2007)

- Es gibt auch einen Ansatz, der neuronale Netze verwendet (RankNet; Burges et. al., 2005)

Rankingprobleme mit diskreter Response: RankOver, TreeRank

- Alle vorgestellten Algorithmen maximieren laut (Cl  men  on et. al., 2013) die AUC, was zu einer L_1 –Approximation der optimalen ROC-Kurve f  hrt. Da aber sehr verschiedene ROC-Kurven die gleiche AUC liefern k  nnen, ist dieses Prinzip nicht ausreichend
- (Cl  men  on, Vayatis, 2008) und (Cl  men  on, Vayatis, 2010) schlagen daher Baum-artige Algorithmen (TreeRank, RankOver) vor, die die ROC-Kurve im L_∞ –Sinn optimieren sollen, d.h., in der Norm

$$\|ROC^*(\cdot) - ROC_s(\cdot)\|_\infty = \sup_{\alpha \in [0,1]} (|ROC^*(\alpha) - ROC_s(\alpha)|).$$

- Problem: Es gibt keine einfache empirische Variante dieser Gr   e, die sich optimieren l  sst

TreeRank und RankOver

- Idee: Partitioniere $\mathcal{X} = \bigcup_j C_j$ und konstruiere eine stückweise konstante Scoring-Funktion

$$s_N(x) := \sum_{j=1}^N a_j I(x \in C_j)$$

mit $a_1 \geq a_2 \geq \dots > 0$.

- Da die Scoring-Funktion s_N nur N verschiedene Werte annehmen kann, besteht die zugehörige ROC-Kurve aus N linearen Teilstücken
- Idee: Versuche, durch Hinzufügen von neuen Knoten zwischen den $(N + 1)$ bestehenden Knoten der ROC-Kurve derart, dass dadurch der AUC-Gewinn maximiert wird, die wahre ROC-Kurve immer weiter anzunähern
- Füge also in jeder Iteration $2^d - 1$ Knoten zu den bestehenden 2^d Knoten hinzu und erreiche dadurch rekursiv eine Partitionierung des Raumes $\mathcal{X}$. Die Scoring-Funktion wird so aufdatiert, dass am Ende das Blatt ganz links den höchsten Rang hat
- Analog versucht RankOver, den ROC-Raum zu partitionieren

Rankingprobleme mit diskreter Response

- Es gibt ferner noch Erweiterungen von TreeRank, z.B. eine gebaggte Version, eine Erweiterung vom RandomForest-Typ sowie hinsichtlich des Stützens des Baumes
- Laut Clémenton können diese Bäume auch das lokalisierte binäre Rankingproblem lösen, da man damit die **lokalisierte AUC**
$$LocAUC(s, \alpha) = P(s(X) \geq s(X'), s(X) \geq Q(s, 1 - u) | Y = 1, Y' = -1)$$
 optimieren kann
- Für Responses mit d Klassen gibt es den Algorithmus TreeRankTournament (Clémenton, Robbiano, 2014), der iterativ das VUS optimiert

Stetiges Rankingproblem

- Sei o.E. $Y \in [0, 1]$. (Cléménçon, Achab, 2018) identifiziert das stetige Rankingproblem als „Kontinuum binärer Rankingprobleme“ in dem Sinne, dass für jedes $y \in [0, 1]$ die Größe $s(X)|Y > y$ **stochastisch größer** sein soll als $s(X)|Y < y$, d.h.,

$$P(s(X) > t | Y > y) > P(s(X) > t | Y < y) \quad \forall t.$$

- Als Gütekriterium schlagen sie eine **integrierte ROC-Kurve** vor, d.h.,

$$IROC_s(\alpha) d\alpha := \int ROC_{s,y}(\alpha) dF_Y(y) d\alpha$$

für die ROC-Kurve $ROC_{s,y}$ bzgl. Scoring-Funktion s für das binäre Subproblem bzgl. y .

- Daraus wird die integrierte AUC

$$IAUC(s) := \int_0^1 IROC_s(\alpha) d\alpha$$

abgeleitet, die letztendlich das Verlustkriterium darstellt

- Die Regressionsfunktion, also $E[Y|X = x]$, ist eine optimale Ranking-Regel für das stetige Rankingproblem
- Sie selbst schlagen den Algorithmus CRank vor, der wiederum ein Baum ist und $\mathcal{X}$ partitioniert

Consensus Ranking

- Für aggregierte Bäume stellt sich bei Rankingproblemen die Frage, wie die Aggregation geeignet definiert werden kann
- Jeder Baum liefert bei der Vorhersage ein **partiell**es Ranking der Instanzen
- Sobald man mehrere partielle Rankings hat, ist es das Ziel, ein sogenanntes **Consensus Ranking** zu finden
- Kemeny-Ansatz: Seien $\pi^{(l)}$ die jeweils zu den partiellen Rankings korrespondierenden Permutationen, also $\pi^{(k)} \in \text{Perm}(1 : n)$ für alle $l = 1, \dots, L$. Sei weiter d eine Distanz zwischen zwei Permutationen, z.B. der harte Ranking-Verlust. Dann ist das Consensus Ranking die Lösung von

$$\min_{\pi \in \text{Perm}(1:n)} \left(\sum_l d(\pi^{(l)}, \pi) \right).$$

- Nachteil beim Consensus Ranking: Die prädiktive Information, die ggf. in den einzelnen L Modellen steckt, geht durch ein solches Verfahren verloren

13. Online Learning

- In vielen realen Anwendungen erhält man nacheinander neue Beobachtungen, die in das aktuelle Modell einzubeziehen sind
- Spam-Filter, um neue Arten von Spam detektieren zu können
- Überwachungsvideos, Tracking von Objekten/Personen
- Empfehlungsdienste
- Situationen, wo die Daten so groß sind, dass sie nicht in den Speicher passen und daher nicht als Ganzes für einen Lernalgorithmus verwendet werden können

Batch Learning vs. Online Learning

- Bisher: Wir hatten Daten $(X_1, Y_1), \dots, (X_T, Y_T)$ simultan vorliegen und haben ein Modell auf diesen T Instanzen berechnet, welches den Trainingsfehler

$$\frac{1}{T} \sum_t L(Y_t, f(X_t))$$

minimiert hat oder eine regularisierte Variante davon

- Diese Situation bezeichnet man als Batch Learning
- Wir bezeichnen im Folgenden $z_t := (X_t, Y_t) \in \mathcal{Z} = \mathcal{X} \times \mathcal{Y}$ und fassen $L(f, z_t)$ als $L(Y_t, f(X_t))$ auf. Ferner sei z^t die gesamte Information bis zum Zeitpunkt t , also $(X_1, Y_1), \dots, (X_t, Y_t)$
- Man kann sich das Online Learning als Spiel gegen die Natur vorstellen: Im einfachsten Fall hat man im Schritt t ein Modell f_t (in der Regel basierend auf den Daten z^{t-1}) und die „Natur“ „wählt“ $z_t \in \mathcal{Z}$. Wir erhalten anschließend den Verlust $L(f_t, z_t)$ und **beide** erhalten (f_t, z_t)
- Diesen Fall bezeichnet man als **adaptive environment**. Wenn die Natur schon im Vorfeld z^T auswählen muss, spricht man von **non-adaptive environment**

Verluste beim Online Learning

- Beim Online Learning erhalten wir also nacheinander („sequentiell“) neue Beobachtungen mit dem Ziel, das Modell geeignet aufzudatieren. D.h., wir haben für jeden „Zeitschritt“ ein eigenes Modell f_t und daher den Verlust

$$\frac{1}{T} \sum_t L(Y_t, f_t(X_t))$$

- Dabei ist für eine Familie $\mathcal{F}$ von Funktionen der **Regret** definiert durch

$$\text{Regret} = \frac{1}{T} \sum_t L(Y_t, f_t(X_t)) - \inf_{g \in \mathcal{F}} \left(\frac{1}{T} \sum_t L(Y_t, g(X_t)) \right).$$

- Genauer: Unsere berechnete Funktion variiert beim Online Learning, sodass wir im Folgenden die Menge Φ aller **Strategien** φ betrachten, wobei jede Strategie φ als Folge von Funktionen φ_t aufgefasst werden kann.
- Daher wird der Regret oft allgemeiner aufgefasst als

$$\text{Regret} = \frac{1}{T} \sum_t L(Y_t, f_t(X_t)) - \inf_{\varphi \in \Phi} \left(\frac{1}{T} \sum_t L(Y_t, \varphi_t(X_t)) \right).$$

- Zum Zeitpunkt t müssen wir $\hat{\beta}^t$ berechnen, sodass wir, sobald wir $\mathbf{z}_t$ sehen, $\mathbf{Y}_t$ möglichst gut durch $\mathbf{X}_t$ vorhersagen
- Da wir $\mathbf{Y}_t$ noch nicht kennen, bleibt uns nichts anderes übrig, als den Regret basierend auf den vorliegenden Beobachtungen zu minimieren, also

$$\hat{\beta}^t = \operatorname{argmin}_{\beta} \left(\frac{1}{t-1} \sum_{s=1}^{t-1} (\mathbf{X}_s \beta - \mathbf{Y}_s)^2 \right),$$

da wir den *inf*-Term ohnehin nicht kontrollieren können.

- Ggf. kann man hier noch eine Ridge-Regularisierung einbauen, dann liegt die Lösung immer noch in geschlossener Form vor für jedes t .

Minimax-Betrachtung

- Man kann das Spiel gegen die Natur als Minimax-Problem auffassen. Sei $\mathcal{P}$ eine Familie von Verteilungen auf $\mathcal{Z}$. Dann würde man im Batch-Fall das Minimax-Problem

$$\inf_{\hat{f} \in \mathcal{F}} (\sup_{f \in \mathcal{F}, P \in \mathcal{P}} (\mathbb{E}[L(\hat{f}, Z)] - \mathbb{E}[L(f, Z)])).$$

- Für Online Learning wird das Minimax-Problem in (Rakhlin et. al., 2010) geschrieben als

$$\inf_{q_1} \left(\sup_{z_1} \left(\mathbb{E}_{q_1} \left[\dots \inf_{q_T} \left(\sup_{z_T} \left(\mathbb{E}_{q_T} \left[\frac{1}{T} \sum_t L(f_t, z_t) - \inf_{f \in \mathcal{F}} \left(\frac{1}{T} \sum_t L(f, z_t) \right) \right] \right) \right) \right) \right) \right),$$

wobei V_T auch als der Wert des Online-Learning-Problems verstanden wird

Definition 12.1

Ein Problem heißt **online lernbar**, falls

$$\limsup_{T \rightarrow \infty} \left(\frac{V_T(L, \Phi)}{T} \right) = 0.$$

- Im einfachsten Fall bekommt man, nachdem man die Vorhersage $\hat{Y}_t$ gemacht hat, im nächsten Schritt Y_t sowie den Verlust $L(Y_t, \hat{Y}_t)$ zurück. Dies nennt sich **full-information setting**
- Es gibt aber auch die Situation, dass man nur den Verlust zurückbekommt, aber nicht Y_t selbst. Das nennt sich **partial feedback setting** oder **bandit setting**
- Beispiel: Bei der Klassifikation mit K Klassen sagen wir $\hat{Y}_t$ voraus und bekommen durch die Verlustfunktion die Antwort, dass die Vorhersage richtig oder falsch war. In diesem Fall spricht man von **multiarmed bandits**
- Dueling Bandits: In jedem Zeitschritt dürfen zwei Entscheidungen getroffen werden (z.B. zwei Klassen) und man erhält danach die Information, welche die bessere war
- Battling Bandits: Analog zu Dueling Bandits, aber in jedem Zeitschritt darf man mehr als zwei Entscheidungen treffen

- Angenommen, wir haben nur eine Sequenz von Y -Werten mit dem Ziel, bei Vorliegen von $Y_1, \dots, Y_t$ den nächsten Wert Y_{t+1} so vorherzusagen, dass der Verlust L minimiert wird

- Dies kann durch ein (Sub-)Gradientenverfahren gelöst werden, d.h.,

$$\hat{Y}_{t+1} = \hat{Y}_t - \eta \nabla L(\hat{Y}_t)$$

für eine Schrittweite η . Ggf. wird die rechte Seite noch geeignet auf $\mathcal{Y}$ projiziert

- Adaptive Verfahren wie AdaGrad (Duchi et. al., 2011) oder AdaSecant (Gulcehre et. al., 2017) versuchen, die Schrittweite adaptiv so zu wählen, dass sie dann groß ist, wenn die neue Beobachtung informativ ist, also nicht in das Muster der bisher gesehenen Beobachtungen passt

Online-Varianten von Algorithmen dieser VL

- Online Least Squares: Verwende die Sherman-Morrison-Formel, um $(X^T X)^{-1}$ aufzudatieren (Anagnostopoulos et. al., 2008)
- Online LDA/QDA: Basiert auf rekursiven und computationell effizienten Updates von Mittelwerten, Kovarianz, Prior-Wahrscheinlichkeiten etc. (Anagnostopoulos et. al., 2012)
- Dabei wird ein (adaptiver) Vergessensfaktor berechnet, der dafür sorgt, dass alte Beobachtungen nicht mehr so viel Gewicht bekommen
- Streaming Lassos (Pio Monti et. al., 2016): Datiere rekursiv den Regularisierungsparameter λ und dann die Koeffizienten auf (wobei die stückweise Linearität der Koeffizientenpfade verwendet wird)
- Online Group Lasso (Yang et. al., 2010)

Online-Varianten von Algorithmen dieser VL

- Online Boosting für binäre Klassifikation (Oza et. al., 2001; Leistner et. al., 2009; Chen et. al., 2012; Beygelzimer et. al., 2015): Aufdatierung der Gewichte durch die Performance der aktuellen schwachen Lerner bzgl. der jeweils neuen Instanz
- Online Boosting für Klassifikation mit mehreren Klassen mit Bandit-Feedback: (Chen et. al., 2014)
- Online Gradienten-Boosting (Beygelzimer et. al., 2015; Hu et. al., 2017) für Regression mit glatten Verlustfunktionen
- Adaptive RandomForest (Gomes, 2017)

- O.E. nehmen wir hier an, dass $Y_t \in \{-1, 1\}$ für alle t . Die Sequenz der schrittweise erhaltenen Y_t kann als ein **binärer Baum der Tiefe T** aufgefasst werden
- Genauer: Da die Natur als Gegenspieler angesehen wird, ist der Baum so zu verstehen, dass wenn $\hat{Y}_t = 1$ vorausgesagt wurde in Schritt t , die Natur sich dann für $Y_t = -1$ entscheidet und vice versa
- Wir bezeichnen einen **Pfad** der Länge t entlang des Baumes durch $\mathbf{e} = (\mathbf{e}_1, \dots, \mathbf{e}_t)$, wobei $\mathbf{e}_i \in \{-1, 1\}$ für alle i (man interpretiert $\mathbf{e}_i = -1$ so, dass man hier in den „linken“ Knoten geht und $\mathbf{e}_i = 1$, dass man in den „rechten“ Knoten geht)
- Der ganze Baum kann als Folge von Funktionen $\mathbf{v}_t$ repräsentiert werden, wobei $\mathbf{v}_t : \{-1, 1\}^{t-1} \rightarrow \mathcal{Z}$, d.h., $\mathbf{v}_t(\mathbf{e})$ ist zu verstehen als die Entscheidung der Natur für den nächsten Wert, wenn man dem Pfad $\mathbf{e}$ gefolgt ist

- Beim Batch Learning hat man direkt die ganze Sequenz $(X_1, Y_1), \dots, (X_T, Y_T)$ erhalten, ohne dass die Natur dazwischen eingreifen konnte. Somit ist dieser Fall der Spezialfall eines **constant level tree**, d.h., es ist $v_t(\mathbf{e}) \equiv z_t$ für alle t
- Man kann den Baum zu reellwertigen Bäumen erweitern, indem man an jedem Knoten die Komposition $h \circ v_t$ im Sinne von $h : \mathcal{X} \rightarrow \mathbb{R}$ betrachtet
- Die Wurzel des Baumes ist immer eine konstante Funktion $v_1(\mathbf{e}) \equiv z_1$.

Definition 12.2

Ein $\mathcal{Z}$ -wertiger Baum der Tiefe T wird durch eine Funktionenklasse $\mathcal{F} \subset \{-1, 1\}^{\mathcal{X}}$ getrennt (shattered), falls für alle $\mathbf{e} \in \{-1, 1\}^T$ ein $\mathbf{f} \in \mathcal{F}$ existiert, sodass $\mathbf{f}(\mathbf{v}_t(\mathbf{e})) = \mathbf{e}_t$ für alle $t = 1, \dots, T$ ist. Die maximale Tiefe eines Baumes, die durch $\mathcal{F}$ getrennt werden kann, heißt **Littlestone-Dimension** des Baumes, kurz $\text{Ldim}(\mathcal{F}, \mathcal{Z})$.

Lemma 12.3

Für einen Baum mit konstanten Werten entspricht die Littlestone-Dimension der VC-Dimension

Sequentielle fette Shatter-Dimension

Definition 12.4

Die **sequentielle fette ε -Shatter-Dimension** einer Funktionenklasse $\mathcal{F} \subset \mathbb{R}^{\mathcal{X}}$ ist die größte Tiefe T , sodass für jeden Baum der Tiefe T , also für alle $\mathbf{e} \in \{-1, 1\}^T$ ein $f \in \mathcal{F}$ existiert, sodass

$$e_t(f(v_t(\mathbf{e})) - s_t(\mathbf{e})) \geq \varepsilon/2 \quad \forall t = 1, \dots, T,$$

die s_t selbst wieder einen reellwertigen Baum Tiefe T repräsentieren, einen sogenannten **Zeugen der Trennung (witness to shattering)**.

Lemma 12.5

Für einen Baum mit konstanten Werten entspricht die sequentielle fette ε -Shatter-Dimension der üblichen fetten ε -Shatter-Dimension.

- Durch die Littlestone- und die sequentielle fette ε -Shatter-Dimension können Resultate der Lerntheorie für die VC- und die fette ε -Shatter-Dimension für die Situation des Online Learning übertragen werden
- Ferner gibt es sequentielle Varianten von Covering- und Packingzahlen, Rademacher-Komplexitäten etc.
- Es gibt somit auch sequentielles Chaining